






# Working Agents: Systems, Capabilities, and Optimization


## Abstract

Working agents extend foundation models from generating responses to carrying out digital work: constructing artifacts, changing business state, coordinating applications and people, and maintaining obligations over time. We organize the field around two independent axes. Work persistence asks how long a delegated responsibility stays open, from work that closes with an immediate response to standing obligations maintained across a service interval. Work-world openness asks how far the task world can be enumerated before execution begins, from closed task environments to workspaces whose material, relations, interfaces, authority, and other actors must be established while the work proceeds. Separating them removes a recurring confusion: a bounded question can arise in a wholly open workspace, and a fully enumerated environment can carry a persistent duty. The first axis yields five work forms, from response-level to workspace-level work. The second yields the capabilities reliable work requires: workspace understanding, perception, reasoning and decision making, planning and coordination, execution and state management, and verification, recovery, and replanning. We then synthesize how experience, model learning, and harness design improve those capabilities, and classify what benchmarks, metrics, judging procedures, and execution environments can actually establish about them, separating requested work from properties checked. Neither axis is a ladder of intelligence or autonomy. A role-separated file-format audit illustrates the object perspective with an explicit statistical denominator. Open challenges concern evolving work environments, persistent responsibility, trustworthy verification, and work-centric systems for organizing and scaling accepted work. This is a structured narrative review with an explicit source ledger, not an exhaustive systematic census or a new agent-performance experiment.


**Keywords:** working agents; model and harness; digital knowledge work; agent capabilities; workspaces; long-horizon agents; agent optimization; scaling.

## Contents


**1 Introduction** . . . . . **4**

**2 Foundation: Positioning and Defining Working Agents** . . . . . **6**
2.1 Work Persistence: From Immediate Closure to Persistent Responsibility . . . . . 6
2.2 Work-World Openness: From Closed Task Environments to Open Workspaces . . . . . 7
2.3 Formalizing Working Agents . . . . . 7
2.4 Working Agents and Related Agent Paradigms . . . . . 9

**3 Work Forms: What Work Do Agents Undertake?** . . . . . **10**
3.1 Response-Level Work . . . . . 10
3.2 Atomic-Action-Level Work . . . . . 11
3.3 Multi-Step-Level Work . . . . . 11
3.4 Workflow-Level Work . . . . . 12
3.5 Workspace-Level Work . . . . . 12

**4 Capabilities: What Enables Reliable Work?** . . . . . **13**
4.1 Perception and Understanding . . . . . 13
4.2 Reasoning and Decision Making . . . . . 16
4.3 Planning and Coordination . . . . . 17
4.4 Execution and State Management . . . . . 18
4.5 Verification, Recovery, and Replanning . . . . . 20

**5 Optimization: From Work Experience to System Capability** . . . . . **21**
5.1 Training Environments, Data, and Rewards . . . . . 21
5.1.1 Task and Environment Synthesis . . . . . 21
5.1.2 Trajectory Collection and Curation . . . . . 23
5.1.3 Reward and Verification Signals . . . . . 24
5.2 Model Optimization . . . . . 24
5.2.1 Mid-Training and Supervised Fine-Tuning . . . . . 25
5.2.2 Agentic Reinforcement Learning . . . . . 25
5.2.3 On-Policy Distillation . . . . . 26
5.2.4 Self-Improving Model Training . . . . . 27
5.3 Harness Optimization . . . . . 28
5.3.1 Interface and Tool Optimization . . . . . 28
5.3.2 Prompt and Workflow Optimization . . . . . 28
5.3.3 Memory and Skill Optimization . . . . . 29
5.4 Model–Harness Coupling and Joint Optimization . . . . . 30
5.4.1 Harness-Aware Model Training . . . . . 30
5.4.2 Learned Harness Design . . . . . 31
5.4.3 Joint Model–Harness Optimization . . . . . 31

**6 Evaluation and Resources: Assessing Working Agents** . . . . . **32**
6.1 Benchmarks and Task Construction . . . . . 32
6.1.1 Task Construction Strategies . . . . . 32
6.1.2 Domain Coverage and Work Requirements . . . . . 34
6.2 Evaluation Methodologies . . . . . 36
6.2.1 Evaluation Targets and Observable Evidence . . . . . 36
6.2.2 Judging Procedures . . . . . 37
6.3 Outcome-Aligned Metrics . . . . . 39
6.3.1 Task Success and Outcome Quality . . . . . 40
6.4 Open-Source Frameworks and Execution Environments . . . . . 41

**7 Frontiers: Scaling Working Agents Beyond Bounded Tasks** . . . . . **44**

7.1 Environment: Infrastructure for Generalization in Realistic Work . . . 46
7.1.1 Coverage across Domains, Occupations, and Workflows . . . 46
7.1.2 Realistic Workspaces and Decision-Relevant Noise . . . 46
7.1.3 Training Curricula and Discriminative Evaluation . . . 47
7.2 Persistence: Sustaining Delegated Responsibilities . . . 47
7.2.1 Proactive and Event-Driven Work . . . 47
7.2.2 Reliability under Delayed and Irreversible Effects . . . 48
7.2.3 Continual Adaptation without Losing Work Commitments . . . 48
7.3 Verification: From Subjective Quality to Value in Use . . . 49
7.3.1 Subjective Quality and Comparative Judgment . . . 49
7.3.2 Generalizable Criteria for Evolving Work . . . 49
7.3.3 Connecting Deliverable Quality to Downstream Value . . . 50
7.4 Work-Centric Systems: From Task Automation to Scalable Work Delivery . . . 50
7.4.1 Work as a First-Class System Object . . . 51
7.4.2 Human–Agent Work Organization and Accountability . . . 51
7.4.3 Net Productivity and Shifting Bottlenecks . . . 52

**8 Conclusion . . . 52**

# 1 Introduction

Foundation models are increasingly moving beyond generating responses toward carrying out consequential digital work. They are used to analyze financial data, create documents and presentations, operate enterprise applications, develop and maintain software, coordinate service requests, and support scientific research. In these settings, evaluating an agent requires looking beyond the quality of individual responses. Real-world tasks require agents to identify relevant objects, understand the dependencies between them, maintain consistency across multiple steps, and produce reliable results that support decision-making, business processes, and downstream activities. Thus, the research question extends beyond whether a model can generate a satisfactory output to how foundation models can be organized into systems that reliably carry out digital work.

Existing research has approached this problem from several directions. Tool-use research studies how models discover, select, and invoke external tools [1, 2]; web and computer-use research examines how agents interact with digital environments through browsers, graphical interfaces, and operating systems [3, 4]; office agents increasingly manipulate structured work products such as spreadsheets and presentations [5, 6]; and enterprise and software-engineering agents operate over application state, organizational processes, and code repositories [7, 8]. Together, these lines of work substantially expand the range of digital tasks that foundation models can participate in. At the same time, they differ in task structure, interaction interface, environment design, runtime support, and evaluation methodology. As a result, an improvement in end-to-end performance may reflect stronger model capabilities, more effective interfaces, richer information, better state organization, or more capable verification. A model- or interface-centric view alone is therefore insufficient to characterize the capabilities and limitations of these systems.

We use the term **working agent** to describe this system-level object of study. We examine how a foundation model, supported by a runtime harness, interacts with a digital work environment to produce acceptable work outcomes. The model provides learned knowledge, reasoning, and decision-making capabilities; the harness organizes context, tools, memory, execution control, and recovery; and the workspace contains stateful objects such as files, applications, database records, messages, tasks, permissions, and other actors. The work outcome may be a textual answer, a document, a spreadsheet, a presentation, code, a change in application or business state, the completion of a multi-stage process, or the maintenance of an ongoing work obligation.

This perspective changes the unit of analysis. Rather than asking only whether a model can predict the correct next action or whether an agent can invoke a particular tool, we ask how a complete working system connects observation, decision, execution, and verification. This requires understanding which objects and constraints matter to the work, which states change during execution, what constitutes an acceptable outcome, and what evidence is sufficient to establish that the outcome satisfies its intended purpose. A recurring question throughout this survey is therefore: which capabilities required for work should be provided by the model itself, and which are better supplied through tools, interfaces, memory, reusable procedures, runtime control, and verification infrastructure?

Working agents also operate under substantially different work conditions. We use two independent dimensions to describe part of this variation. **Work persistence** characterizes the extent to which delegated responsibility extends beyond immediate feedback. In some settings, responsibility closes when an answer is delivered or a local action is confirmed; in others, the system must wait for subsequent stages, human approval, external events, or new inputs and continue to carry the original obligation after those events occur. Work persistence therefore concerns how long responsibility remains open across feedback cycles and what conditions are required for it to close, rather than simply how long an execution lasts.

**Work openness** characterizes how much of the relevant task world can be determined before execution begins. In relatively closed task environments, the input material, relevant objects, interfaces, and rules can largely be specified in advance. In more open work worlds, the agent may need to discover relevant objects, interpret heterogeneous representations, respond to changing state, interact with other actors, and operate under permissions and authorization constraints. A task can be short-lived yet take place in a highly open environment, while a highly specified environment can support a responsibility that persists over a long interval. Work persistence and work openness are therefore not levels of task difficulty or intelligence, but complementary descriptors of the conditions under which work is performed.

Reliable work under these conditions requires capabilities that extend beyond single-turn language generation. An agent must establish an appropriate understanding of its workspace, identify task-relevant objects and dependencies, perceive and interpret information in heterogeneous representations, reason about goals and constraints, plan and coordinate actions, and execute those actions against actual environment state. Once actions affect the environment, the system must also distinguish intended operations from realized effects, maintain consistency across objects and stages, and verify, recover, or replan when assumptions fail or the environment changes. We therefore organize the core capabilities of working agents around workspace understanding, perception, reasoning and decision making, planning and coordination, execution and state management, and verification, recovery, and replanning.

These capabilities can be improved through different mechanisms. Some can be strengthened through data, interaction experience, and model optimization; others can be supported externally through better tool interfaces, structured environment representations, memory, reusable procedures, runtime controls, and verification mechanisms. Improving a working agent therefore does not necessarily mean modifying model parameters alone. Different systems may allocate the same functional requirement differently between model-internal competence and external support. Understanding this allocation provides a useful basis for comparing optimization methods and for studying how model and harness design jointly determine system behavior.

Evaluation raises a related systems problem. Working agents often produce more than a final textual response: their outcomes may include structured artifacts, environment-state changes, cross-stage dependencies, and behavior that must remain valid over time. Yet the work requested by a task and the properties actually observed by an evaluator are not necessarily the same. A task may require a correct, editable, and source-consistent workbook while an automated evaluator checks only selected cells; an enterprise task may require completion of a business process while the benchmark observes only a subset of state transitions. Evaluating a working agent therefore requires asking not only whether a task is declared complete, but also what evidence the evaluator inspects and what conclusions that evidence can support. An end-to-end score should accordingly be interpreted as a property of a particular model, harness, environment, and verification configuration rather than automatically attributed to the model alone.

Based on this perspective, this survey makes five contributions:

- **A work-centric view of agent research.** We organize research on tool use, computer use, office agents, enterprise agents, software-engineering agents, and scientific agents around the digital work they are expected to accomplish, emphasizing how model capabilities are converted into observable and verifiable work outcomes.
- **A system-level characterization of working conditions and execution.** We use work persistence and work openness to distinguish the temporal extent of delegated responsibility from the openness of the task world, and describe working-agent execution in terms of the roles of the task, model, harness, workspace, work outcome, and verification procedure.
- **A cross-domain account of capabilities required for reliable work.** We synthesize recurring requirements across domains, including workspace understanding, perception, reasoning and decision making, planning and coordination, execution and state management, and verification, recovery, and replanning.
- **An organization of working-agent optimization from work experience to system capability.** We examine how training environments, data, and rewards are constructed from work, how models and harnesses are optimized with them, and how the two are optimized jointly, with particular attention to whether a capability is better learned by the model or supplied by the harness.
- **An outcome-oriented view of working-agent evaluation.** We analyze what benchmarks, metrics, judging procedures, and execution environments actually inspect, distinguishing the work requested by a task from the properties that an evaluator can verify and the capability claims that such evidence can support.

This survey focuses on digital knowledge work across office productivity, professional services, enterprise operations, software engineering, and scientific research. Section 2 defines working agents, introduces the

relevant work conditions and system components, and relates working agents to neighboring agent paradigms. Section 3 examines the forms of work that agents undertake. Section 4 develops the capabilities required for reliable digital work. Section 5 reviews how training environments, data, and rewards support model optimization, harness optimization, and their joint optimization. Section 6 examines benchmarks, evaluation methodologies, outcome-aligned metrics, and execution environments. Section 7 discusses open challenges in open and evolving work worlds, persistent responsibility, trustworthy verification, and work-centric systems for organizing and scaling accepted work. Section 8 concludes the survey.

## 2 Foundation: Positioning and Defining Working Agents

As agents expand from question answering to tool use, environment interaction, and practical digital work, the conditions under which they operate are also changing. We describe this change along two independent but complementary dimensions: **work persistence** and **work openness**.

Work persistence describes the progression from immediate closure to persistent responsibility: the extent to which a delegated responsibility must remain active across subsequent feedback, stages, and events. Work openness describes the progression from closed task worlds to open work worlds: the extent to which an agent must operate in a task world that cannot be fully specified before execution begins.

The two dimensions characterize different properties of the work setting. A short-lived responsibility may arise in a highly open work world, while a largely predefined environment may support a responsibility that persists over an extended period. The following sections develop these two dimensions before formalizing the working-agent system and relating it to neighboring agent paradigms.

### 2.1 Work Persistence: From Immediate Closure to Persistent Responsibility

Many earlier language-model tasks operate within relatively short feedback loops. A system receives a question and relevant context, produces an answer, and can then be evaluated directly on the resulting output. Benchmarks such as DocVQA, FinQA, and TAT-QA exemplify settings centered on question answering and reasoning over provided material [9–11]. Even when the reasoning itself is complex, the work is typically organized around a relatively clear input–output cycle.

As models begin to use tools and act on external environments, the feedback loop becomes longer. Toolformer and API-Bank extend language-model behavior from text generation to tool selection and invocation [1, 2], while ToolSandbox and AppWorld further incorporate the resulting environment state into task execution and evaluation [12, 13]. The agent must therefore do more than choose an action: it must observe what actually happened and decide how to proceed from the resulting state.

As tasks further extend across multiple turns and stages, relevant feedback may arrive later. The consequences of earlier actions may only become visible in subsequent interactions, while users or other actors may provide new information or modify shared state during execution. MINT and the $\tau$-bench family illustrate longer feedback chains involving multi-turn interaction, user feedback, and shared environment state [14–16].

Long-horizon research extends this trajectory further by examining an agent's ability to sustain work over longer periods. The METR time-horizon analysis characterizes the length of human tasks that agents can complete at a given success rate [17], while RE-Bench studies open-ended research-engineering work under different time budgets [18]. Together, these settings reflect a broader shift from immediate response toward longer execution and feedback processes.

For example, an agent assigned to maintain a business report over a month must track source-data updates, revise affected analyses, and keep the report consistent with current records throughout that period.

We use **work persistence** to capture this progression. As persistence increases, the current response becomes less likely to constitute a natural endpoint of the work, and completion increasingly depends on later states, subsequent stages, other actors, or future events. At one end of the dimension, responsibility can close once the task's completion criteria are met. As the feedback loop extends, responsibility increasingly spans multiple interactions and waiting periods. At the continuous work end, the system must maintain work state and responsibility over an extended interval and continue acting as new events arrive.

Work persistence therefore characterizes the extent to which delegated responsibility extends with the feedback loop: from work that can close after a single round of feedback to work that must continue across stages, events, and time.

## 2.2 Work–World Openness: From Closed Task Environments to Open Workspaces

Alongside the extension of feedback loops, the task worlds in which agents operate are also becoming more open.

In many earlier benchmarks, the materials, objects, and interaction scope required for a task are largely specified before execution begins. Question-answering tasks directly provide the context to be analyzed, while API-based tool-use settings typically expose a relatively explicit set of tools and interfaces [2, 10]. In such environments, the agent primarily reasons or acts within a task space that has already been defined.

Web agents expand this boundary toward less predefined environments. WebArena places agents in functional websites, requiring them to obtain relevant information through navigation, retrieval, and interaction [3]. OSWorld extends this setting to realistic computer environments, where agents must work across applications, interfaces, and different types of digital objects [4].

As agents move into interconnected applications, enterprise systems, and professional workspaces, task-relevant information becomes increasingly difficult to provide completely before execution begins. AppWorld connects multiple applications and their states [13]; the $\tau$-bench family further incorporates users, policies, and shared business state into the interaction environment [15, 16]; and Workspace-Bench studies object discovery and file dependencies in professional workspaces, making the identification of relevant material and relationships part of the task itself [19].

We use **work openness** to capture this progression from closed task worlds to open work worlds. As openness increases, more of the task world must be established during execution: which objects are relevant, how those objects relate to one another, whether the current state has changed, which actions are currently available, and how other actors affect shared state. Completing the work therefore increasingly depends on the agent's ability to build and continuously update its understanding of the work world while acting within it.

Work openness characterizes the extent to which the relevant task world cannot be fully enumerated before execution and must instead be discovered, interpreted, and updated during the course of work. Sources of openness may include object discovery, representational heterogeneity, changing state, external actors, permissions and authority, and multiple viable solution paths.

## 2.3 Formalizing Working Agents

Answering, acting, and working emphasize different aspects of agent behavior. Answering produces an informational result; acting observes or changes an environment through an interface; working organizes observations, decisions, and actions around an explicit objective under relevant constraints and produces an acceptable result. For some forms of work, an answer itself may constitute the final result. For others, completion may involve producing a work artifact, changing workspace state, completing a process, or maintaining a responsibility over time.

We define a **working agent** as a system that uses a foundation model as its decision-making core and a runtime harness to organize observations and actions, interacting with a workspace under a given task, resources, permissions, and constraints in order to accomplish a work objective.

To describe such a system, we distinguish six interrelated elements: **Task**, **Model**, **Harness**, **Workspace**, **Work Outcome**, and **Verification**.

**Task** ($T$) specifies the work to be accomplished, including its objective, relevant constraints, resources and permissions, and the conditions used to determine whether the work has been completed satisfactorily. The task defines what the system is expected to accomplish and what constitutes a valid completion.

**Model** ($M$) is the learned reasoning and decision-making core of the system. It interprets available information, performs reasoning and judgment, and proposes responses or actions.

**Harness** ($H$) organizes the model's participation in the work process, including context construction, tool use, memory, execution control, feedback handling, and recovery. It connects the model's reasoning and decisions to actual execution and organizes task-relevant information from the workspace into context available to the model.

**Workspace** ($E$) is the stateful environment in which work takes place. It contains work objects such as files, applications, records, messages, and tasks, together with their relationships, available interfaces, permissions, and other actors. Workspace state may change as work proceeds.

**Work Outcome** ($O$) represents the result actually produced by the agent's execution. It may take the form of an answer, document, spreadsheet, presentation, code artifact, application or business-state change, a completed process, the maintenance of an ongoing work obligation, or another task-relevant result produced during execution.

**Verification** ($V$) evaluates the agent's performance using the requirements of the task and the evidence available from execution. It may assess whether the task has been completed, whether the work outcome is correct and complete, whether relevant constraints have been satisfied, and whether the resulting workspace state matches the intended effect. Verification may be performed by programs, tools, models, or human reviewers, and may produce task completion judgments, quality measures, diagnostic feedback, or other forms of assessment.

These six elements provide a system view of a working agent. The task specifies the work and its requirements; the model and harness organize execution; the workspace contains the objects and state on which the work operates; execution produces a work outcome; and verification evaluates task completion and work quality from observable evidence.

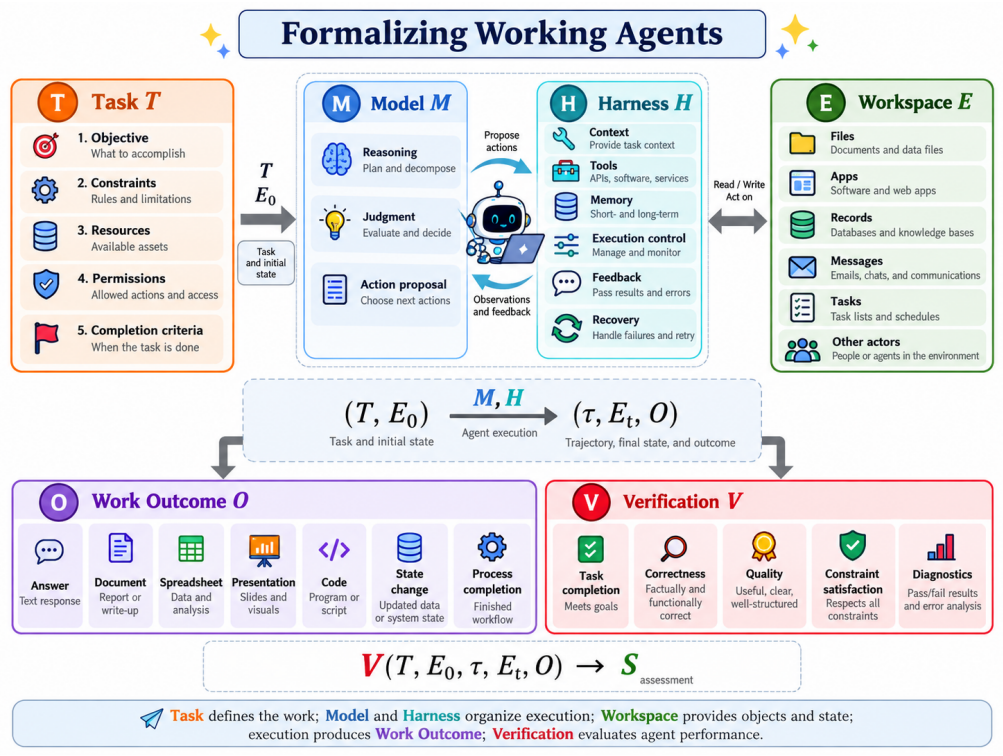



**Figure 1** Formalizing working agents: Task, Model, Harness, Workspace, Work Outcome, and Verification. The model and harness carry out the task in a workspace, producing an observable trajectory, a resulting workspace state, and a work outcome; verification assesses performance using the task requirements and available evidence.

Formally, given a task $T$ and an initial workspace state $E_0$, the model $M$ and harness $H$ carry out work in

the workspace:

$$(T, E_0) \xrightarrow{M,H} (\tau, E_t, O),$$

where $\tau$ denotes the observable trajectory of actions and feedback, $E_t$ denotes the workspace state after execution, and $O$ denotes the resulting work outcome. Different forms of work may produce different types of outcomes: an analytical task may primarily produce an answer, office work may produce a structured work artifact, while operational work may be expressed primarily through changes to workspace state.

Verification then evaluates the agent's performance:

$$V(T, E_0, \tau, E_t, O) \to S,$$

where $S$ denotes an assessment of the agent's performance on the task. It may capture task completion, outcome quality, correctness, constraint satisfaction, diagnostic information, or other task-relevant measures. Because different verification procedures have access to different evidence, what they can establish depends on the work outcome, workspace state, and execution records that they are able to inspect.

A working agent can therefore be viewed as a work system organized around a task: the model provides reasoning and decision making, the harness organizes execution, the workspace contains the objects and state involved in the work, execution produces a work outcome, and verification evaluates the agent's task completion and work quality based on observable evidence.

## 2.4 Working Agents and Related Agent Paradigms

Working agents are closely related to several established agent paradigms. Among them, **long-horizon agents**, **computer-use agents**, and **office agents** are particularly relevant to the work conditions and system settings considered in this survey. These lines of research extend agent capabilities along different dimensions, including execution horizon, workspace interaction, and the objects and outcomes involved in practical work. The working-agent perspective focuses on how these capabilities are organized around a task to form a complete work process.

**Long-horizon agents** study how agents maintain task progress, context, and state over extended execution and handle dependencies whose consequences may appear only in later stages. Existing work examines long-horizon performance through task duration, extended execution, and the preservation of state and prior information [17, 18]. This line of research is closely related to work persistence: as feedback loops extend, earlier information, decisions, and unresolved responsibilities must remain effective in subsequent stages of work.

**Computer-use agents** study how agents interact directly with environments through browsers, graphical user interfaces, and operating-system interfaces. WebArena and OSWorld, for example, require agents to complete tasks through navigation, interface manipulation, and interaction across applications [3, 4]. This line of research expands the workspace that an agent can directly observe and manipulate, connecting model decisions to actions over concrete objects and state.

**Office agents** focus more directly on common work objects and deliverables such as documents, spreadsheets, and presentations. Systems such as SheetCopilot study understanding, planning, and operations over spreadsheets, while PPTAgent focuses on the generation and editing of presentations [5, 6]. Office work often requires an agent not only to generate content, but also to understand existing work objects, perform modifications, preserve structure and relationships, and produce outcomes that can be further used and evaluated. This makes office agents particularly close to the working-agent setting considered in this survey.

These paradigms are not mutually exclusive. An office agent may operate through computer-use interfaces while also carrying out a long-horizon task, and a long-horizon agent may depend on graphical interfaces, tools, and structured work objects. A working agent is therefore not defined by a particular interface, work object, or execution duration. Instead, the working-agent perspective examines how a task is translated by

the model and harness into actions over a workspace, how those actions produce a work outcome, and how the agent's performance is assessed through verification.

**Table 1** Working agents and closely related agent paradigms.

| Related paradigm | Relationship to working agents |
|---|---|
| **Long-horizon agents** | Focus on task progress, state maintenance, and dependencies across extended execution, supporting work that spans delayed feedback and persistent responsibilities. |
| **Computer-use agents** | Focus on interaction through browsers, GUIs, and operating systems, enabling agent decisions to act directly on workspace objects and state. |
| **Office agents** | Focus on common work objects and outcomes such as documents, spreadsheets, and presentations, providing an important setting for studying agents that produce and modify practical work products. |

## 3 Work Forms: What Work Do Agents Undertake?

Section 2 characterizes the conditions under which Working Agents operate through the dimensions of Work Persistence and Work Openness. This section instead focuses on the scope and organization of the work delegated to an agent. As agents move from language generation toward practical work environments, the work they undertake can expand from producing responses from given information, to performing localized actions on concrete work objects, completing larger objectives through multiple actions, executing end-to-end workflows composed of multiple stages, and operating across Workspaces containing many related objects and dependencies.

Based on this expansion in work scope, we organize the work undertaken by Working Agents into five forms:

Response-Level Work → Atomic-Action-Level Work → Multi-Step-Level Work → Workflow-Level Work → Workspace-Level

These forms describe the primary organizational scope of work rather than execution time, the number of low-level operations, or the number of tool calls. As the scope of work expands, the objects and structures involved progress from given information and localized operations to complete objectives, interdependent work stages, and distributed objects and states within a Workspace.

### 3.1 Response-Level Work

**Response-Level Work** refers to work in which an agent produces an informational result from a given question and its associated context. The information required for the task is primarily provided through text, documents, tables, images, or other input modalities, and the agent interprets and reasons over this information to produce an answer, analysis, classification, or judgment without directly modifying external work objects or their states.

Classical reading comprehension and question answering benchmarks exemplify this form of work. SQuAD evaluates question answering over provided text passages, while HotpotQA further requires multi-hop reasoning across multiple supporting documents [20, 21]. DocVQA and ChartQA extend question answering to document images and charts, respectively [9, 22]. FinQA and TAT-QA focus on textual, tabular, and numerical reasoning over financial reports; LegalBench includes tasks involving legal rule understanding, application, and judgment; and PubMedQA and ScienceQA cover biomedical research questions and scientific questions, respectively [10, 11, 23–25].

Although these tasks differ in input modality, knowledge requirements, and reasoning structure, the work remains centered on understanding a supplied context, reasoning over it, and producing an informational response. Long documents, multiple evidence sources, numerical computation, or specialized knowledge can substantially increase the reasoning required without changing this basic work form.

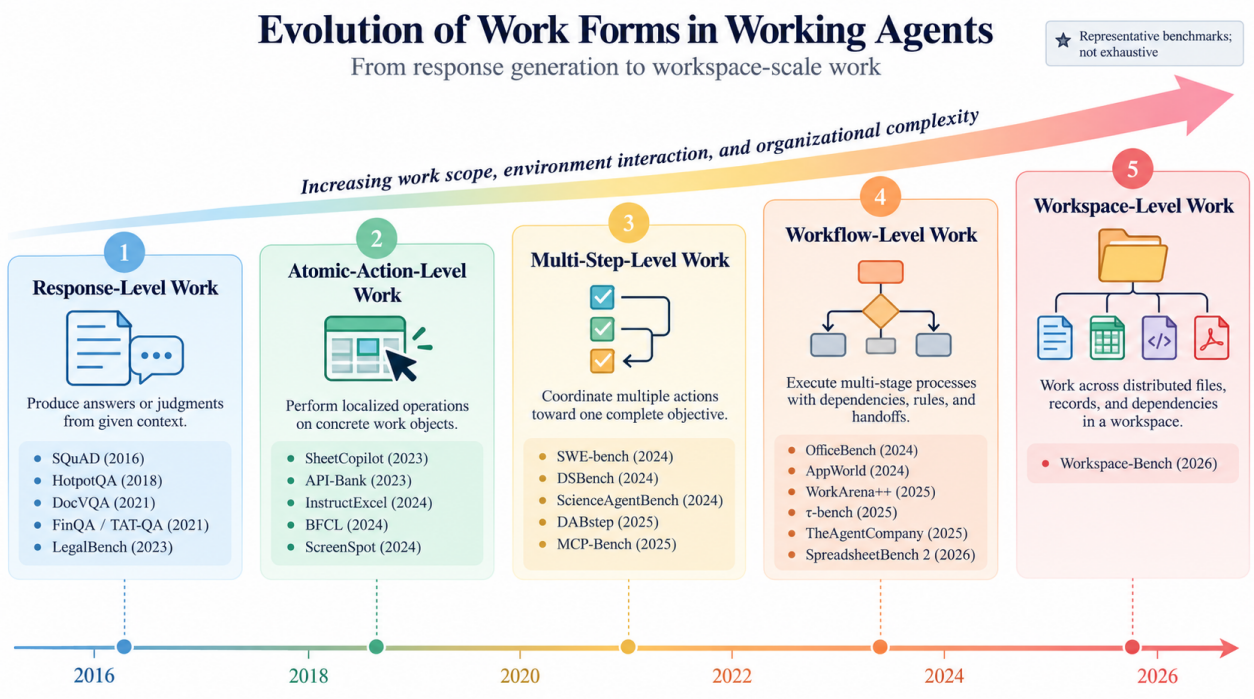



**Figure 2** Evolution of work forms in Working Agents. The five forms expand from responses based on supplied context to localized actions, multi-step objectives, end-to-end workflows, and work over distributed objects and dependencies in a Workspace. The listed benchmarks are representative rather than exhaustive.

## 3.2 Atomic-Action-Level Work

When an agent begins to directly manipulate concrete work objects, a localized operation with a clear objective can constitute an independent unit of work. We refer to this form as **Atomic-Action-Level Work**. An Atomic Action may involve modifying a spreadsheet cell, inserting or correcting a formula, adjusting a presentation element, updating a structured record, executing a clearly specified service operation, or performing a designated interaction with an application interface.

Here, atomic characterizes the local work objective rather than the number of low-level execution steps. For example, correcting an erroneous value in cell C7 can constitute a single Atomic Action even if the modification requires several mouse, keyboard, or API operations. Similarly, changing a particular field in a customer record or adjusting the position of a slide object represents a localized change with a clearly defined target and effect.

Spreadsheet manipulation provides direct examples of this form of work. SheetCopilot organizes spreadsheet functionality into a collection of composable atomic actions, while InstructExcel covers concrete Excel operations involving formulas, formatting, data manipulation, and worksheet editing [5, 26]. In structured service interaction, API-Bank and the Berkeley Function-Calling Leaderboard (BFCL) study function selection and argument generation for API execution and function calling [2, 27]. ScreenSpot and ScreenSpot-Pro focus on identifying and grounding localized interface targets for interaction [28, 29].

Supporting systems typically provide mechanisms for local object identification and action execution, enabling agents to apply a specified operation to the intended work object [5, 28, 30].

## 3.3 Multi-Step-Level Work

Many practical work objectives cannot be completed through a single Atomic Action, but instead require multiple interdependent actions to collectively produce a complete result. We refer to this form as **Multi-**

**Step-Level Work**. Here, the agent repeatedly acts toward a common objective, incorporates intermediate results into the current state, and uses them to determine subsequent actions.

For example, a data analysis task may involve reading data, preprocessing it, writing and executing code, inspecting intermediate results, and correcting errors. Resolving a software issue may similarly require locating relevant code, diagnosing the problem, modifying a repository, running tests, and iteratively refining the implementation based on feedback. The unit of work therefore extends beyond any individual local action to the complete objective achieved through a sequence of coordinated actions.

DABstep organizes document understanding, data processing, and code execution into multi-step data analysis tasks [31], while MCP-Bench evaluates realistic multi-step tasks that require planning, cross-tool coordination, and multi-hop execution [32]. SWE-bench is built around real software issues and repository modifications, DSBench and DataSciBench target complex data-analysis and data-science tasks, and ScienceAgentBench evaluates agents on data-driven scientific tasks [33–36]. Presentation and professional content creation tasks likewise involve coordinating content selection, organization, and presentation to produce a complete deliverable [37, 38].

Such multi-step execution commonly relies on planning, task decomposition, intermediate-state maintenance, and iterative execution to organize dependencies among actions and adapt subsequent steps to execution results [39, 40].

## 3.4 Workflow-Level Work

When work expands into an end-to-end process composed of multiple functionally meaningful stages, we refer to it as **Workflow-Level Work**. Different stages within a workflow perform distinct functions and are connected through data, state, prerequisites, business rules, or role relationships. The output of one stage may become the input to another or determine whether subsequent stages can proceed.

For example, a business process may include information collection, data entry, review, user confirmation, and follow-up processing, with approval, handoff, branching, validation, or repair introduced at different stages. The work therefore requires not only the execution of multiple actions but also the maintenance of dependencies and state transitions across distinct stages of the process.

WorkArena++ composes work components across enterprise applications into more complete and realistic workflows [41]. OfficeBench covers coordinated work across applications such as email, calendar, Word, and Excel [42], while AppWorld places agents in environments consisting of interconnected applications and complex control flows [13]. The $\tau$-bench and $\tau^2$-bench families incorporate users, policies, tools, and shared business state into the interaction process [15, 16]; TheAgentCompany further introduces coworker interaction and organizational collaboration [43]; and CRM benchmarks examine execution across records, policies, and business processes in customer relationship management settings [44, 45].

SpreadsheetBench 2 extends isolated spreadsheet operations toward generation, debugging, and visualization within end-to-end business workflows, including dependencies across multiple sheets [46]. Together, these settings illustrate how workflow structure emerges from the coordination of multiple stages and the dependencies that connect them.

Correspondingly, system methods have begun to represent and organize workflow structure explicitly, including dependency modeling, stage composition, and multi-agent coordination for constructing or optimizing complete execution processes [47, 48].

## 3.5 Workspace-Level Work

When work extends to environments containing large collections of heterogeneous objects, an agent must discover, organize, and use the content relevant to the current work within a broader Workspace. We refer to this form as **Workspace-Level Work**.

In Workspace-Level Work, the objects and relationships required to complete the work are not necessarily fully specified at the outset. Relevant information may be distributed across spreadsheets, documents, presentations,

source code, business records, messages, and historical versions, connected through data provenance, content references, version relationships, or other dependencies. The agent must therefore identify the relevant objects within the broader Workspace and establish the relationships among them that matter for the current work.

Workspace-Bench directly targets this workspace-level setting by placing agents in Workspaces containing large collections of heterogeneous files together with explicit and implicit file dependencies, and evaluating cross-file retrieval, contextual reasoning, and adaptive decision-making [19]. Object discovery, dependency identification, and task-relevant context construction thus become part of the work itself.

To operate over large and structurally complex work environments, systems increasingly employ mechanisms such as structural representations, selective retrieval, local summaries, and external state organization to identify and organize the Workspace content relevant to the current work [49, 50].

For example, updating a business report may require locating the authoritative workbook, tracing the corresponding source records, and identifying references to the same data in other documents or presentations. The scope of work is therefore defined by a set of distributed objects and their relationships within the Workspace rather than by a single pre-specified work object.

Taken together, these five forms describe an expansion in the scope of delegated work. The scope broadens from producing informational responses or making localized changes to completing objectives through multiple coordinated actions, carrying out end-to-end workflows, and operating across distributed workspace objects. As work spans multiple actions, stages, and objects, managing their dependencies becomes part of the work itself.

## 4 Capabilities: What Enables Reliable Work?

Section 3 distinguishes the forms of work that a Working Agent may be delegated. The present section instead asks what the system must be capable of doing to complete such work reliably. The key issue is whether an agent can interpret the working context, apply the knowledge and judgment required by the task, organize its actions, complete the requested changes, and correct mistakes when necessary. Such requirements emerge across office applications, software repositories, enterprise services, and professional workflows [8, 19, 41, 43].

Five closely connected capabilities characterize this process: Perception and Understanding, Reasoning and Decision Making, Planning and Coordination, Execution and State Management, and Verification, Recovery, and Replanning. They distinguish recognizing the work environment, making task-relevant decisions, organizing dependent work, realizing changes, and establishing that the work meets its requirements. Table 2 relates these capabilities to the five Work Forms of Section 3.

**Table 2** Capability demands across the five Work Forms

| Work Form | Perception and Understanding | Reasoning and Decision Making | Planning and Coordination | Execution and State Management | Verification, Recovery, and Replanning |
|---|---|---|---|---|---|
| Response-Level Work | Interpret supplied evidence and context | Derive supported answers and judgments | Organize local reasoning when needed | Produce an informational result | Check answer support and consistency |
| Atomic-Action-Level Work | Identify the target object and local state | Select an appropriate operation | Establish a valid local action path | Commit the intended local change | Confirm the target effect |
| Multi-Step-Level Work | Track intermediate evidence and state | Adapt decisions to intermediate results | Decompose a coherent objective | Maintain state across related actions | Check intermediate and final outcomes |
| Workflow-Level Work | Identify stages, rules, and shared state | Reason about prerequisites and policies | Coordinate stages, branches, and handoffs | Commit consequential state transitions | Validate stage transitions and workflow outcomes |
| Workspace-Level Work | Discover relevant objects, relations, versions, and authority | Reconcile evidence across objects | Plan over object and dependency structure | Preserve cross-object and cross-application consistency | Check lineage, freshness, and dependent outcomes |

### 4.1 Perception and Understanding

Perception and Understanding enables a Working Agent to observe its current work environment and interpret the native objects on which the delegated Task depends. Perception concerns the information accessible through files, interfaces, and application state; understanding identifies what that information represents and

how it can be used for the task. The relevant distinction is not simply between visual and textual inputs: an agent may read a value correctly but misunderstand its unit, target cell, active document section, or application state. Reliable perception must therefore be coupled to object-sensitive understanding [9, 19, 51].

## Work Environment Observation

Working Agents obtain task information from supplied context, tool-accessible applications, and persistent files. These access modes shape what information an agent can inspect and how it interacts with the work environment. Table 3 summarizes the principal input and access modes [19].

**Table 3** Input and access modes for working-agent tasks

| Mode | How work material is supplied or accessed | Typical work setting | Interpretation boundary |
|---|---|---|---|
| P — Prompt-only | The task material is contained in the prompt | Analyze supplied text or a directly represented table | Refers to the input setting; not every prompted agent is prompt-only |
| T — Tool-based | Information and state are exposed through calls or application interfaces | Search, query a database, inspect a CRM record | A tool can read information or change state; T alone does not specify which |
| F — File-based | Persistent files are provided for discovery, reading, editing, or production | Inspect a workbook or modify a repository | File content may be accessed through tools; F and T can coexist |

The information available to an agent also depends on the work setting. Table 4 compares representative benchmarks by their access modes, workspace configurations, and observed materials [4, 13, 19, 33, 42, 43].

**Table 4** Representative benchmark environments and their observation surfaces

| Benchmark | Workspace configuration | Observed work material | Evidence | Access mode |
|---|---|---|---|---|
| OneMillion-Bench (source comparison) | Prompt-supplied task setting | Observation details not specified in source comparison | [19] | P |
| CL-Bench (source comparison) | Prompt-supplied task setting | Observation details not specified in source comparison | [19] | P |
| OdysseyBench (source comparison) | Tool-access environment | Specific observation materials not recorded in source comparison | [19] | T |
| MMTB (source comparison) | Tool-access environment | Specific observation materials not recorded in source comparison | [19] | T |
| MultiAgentBench | Multi-agent interactive scenarios | Shared interactions and task observations | [52] | T |
| OfficeQA-Pro (source comparison) | Office document setting | Source files and office information (reported modality) | [19] | T + F |
| GDPval | Professional deliverable tasks | Task instructions and supplied work materials | [53] | T + F |
| WorkBench | Workplace databases and tools | Business records, messages, calendars, and task state | [54] | T |
| DocVQA | Supplied document | Document image and visual text | [9] | P |
| FinQA | Supplied evidence | Financial reports, text, and tables | [10] | P |
| LegalBench | Supplied legal text | Legal facts, rules, and arguments | [23] | P |
| WebArena | Interactive website | Webpage elements and site state | [3] | T |
| VisualWebArena | Visual web environment | Rendered webpage and visual elements | [55] | T |
| WorkArena / WorkArena++ | Enterprise web application | Enterprise forms, records, and task stages | [7, 41] | T |
| OSWorld | Desktop computer environment | GUI, application state, and local files | [4] | T + F |
| Windows Agent Arena | Windows desktop environment | Windows UI, documents, and application state | [56] | T + F |
| AndroidWorld | Mobile application environment | Android UI and application state | [57] | T |
| AppWorld | Connected API applications | Structured records and inter-application state | [13] | T |
| $\tau^2$-bench | Customer-service tools and user state | Policies, customer records, and tool observations | [16] | T |
| ToolSandbox | Stateful tool environment | Tool observations, prerequisites, and state | [12] | T |
| CRMArena / CRMArena-Pro | CRM business environment | Accounts, cases, orders, and policies | [44, 45] | T |

| Benchmark | Workspace configuration | Observed work material | Evidence | Access mode |
|---|---|---|---|---|
| OfficeBench | Multi-application office environment | Office objects and application operations | [42] | T + F |
| SWE-bench | Software repository | Source files, issues, and tests | [33] | T + F |
| Spider 2.0 | Enterprise data environment | Database schema, SQL dialects, and project files | [58] | T + F |
| ScienceAgentBench | Scientific data and code | Datasets, scripts, and computational outputs | [36] | T + F |
| MedAgentBench | Interactive electronic health record | Patient records and FHIR operations | [59] | T |
| TheAgentCompany | Organizational digital workplace | Repositories, shared files, messages, and tasks | [43] | T + F |
| Workspace-Bench | Multi-file professional workspace | Heterogeneous files and task-relevant sources | [19] | T + F |

In graphical work environments, an agent must identify interface elements that correspond to a requested operation. SeeClick trains visual grounding from screenshot-based supervision, while OmniParser parses screenshots into structured, interactable regions with associated descriptions [28, 60]. GUI-Actor aligns action queries with visual regions without requiring explicitly predicted coordinates, and Agent S2 combines interface grounding with broader computer-use control [30, 61].

## Work-Object Understanding

Work objects have semantics that extend beyond their text content. Documents require reading order, tables, footnotes, and visual layout [9, 62, 63]; spreadsheets couple displayed values to formulas, references, sheets, and number formats [51, 64]; presentations combine content with editable shapes, charts, and spatial hierarchy [37, 38]; repositories require file, symbol, and execution relationships [33, 65]; and business records are governed by entity identity and state constraints [13, 45]. Table 5 records the object types and access modalities represented in the supplied benchmark compilation.

**Table 5** Benchmark × file types and access modalities

| Benchmark | Word | Excel / tables | PPT | PDF | Data CSV / JSON | Code / terminal | GUI / multimodal | Web / API / tools |
|---|---|---|---|---|---|---|---|---|
| Finance | | | | | | | | |
| SpreadsheetBench | × | ✓ | × | × | × | × | × | × |
| BankerToolBench | × | ✓ | × | × | ✓ | × | × | ✓ |
| Finance Agent v2 | × | ✓ | × | ✓ | ✓ | × | × | ✓ |
| Finance/Law | | | | | | | | |
| PRBench | × | ✓ | × | ✓ | × | × | × | × |
| Law | | | | | | | | |
| LegalBench | × | × | × | × | × | × | × | × |
| Harvey LAB | ✓ | ✓ | ✓ | ✓ | × | × | × | ✓ |
| Legal Research (vals) | × | × | × | ✓ | ✓ | × | × | × |
| Healthcare | | | | | | | | |
| MedCode | × | × | × | × | ✓ | × | × | × |
| MedScribe | × | × | × | × | × | × | × | × |
| PSEBench | × | × | × | × | ✓ | × | × | × |
| MLCR-AA | × | × | × | ✓ | ✓ | × | × | × |
| Office | | | | | | | | |
| OfficeQA | ✓ | ✓ | ✓ | ✓ | × | × | × | × |
| PresentBench | × | × | ✓ | × | ✓ | × | × | × |
| PPTC | × | × | ✓ | × | × | × | × | × |
| OmegaUse-OfficeVal | ✓ | ✓ | ✓ | ✓ | × | × | × | × |
| OfficeBench | ✓ | ✓ | ✓ | × | × | × | × | ✓ |
| MMLongBench-Doc | × | × | × | ✓ | × | × | ✓ | × |
| GDP.pdf | × | × | × | ✓ | × | × | ✓ | × |
| PPTEval/PPTAgent | × | × | ✓ | × | × | × | × | × |
| General | | | | | | | | |
| GDPval | ✓ | ✓ | ✓ | × | × | × | ✓ | × |
| OmniaBench | ✓ | ✓ | ✓ | × | × | × | × | ✓ |
| Workspace-Bench | ✓ | ✓ | ✓ | × | × | × | × | ✓ |
| JOBBench | × | × | × | × | ✓ | × | × | ✓ |

| Benchmark | Word | Excel / tables | PPT | PDF | Data CSV / JSON | Code / terminal | GUI / multimodal | Web / API / tools |
|---|---|---|---|---|---|---|---|---|
| AutomationBench | × | × | × | × | × | × | × | ✓ |
| APEX-Agents | ✓ | ✓ | ✓ | ✓ | × | × | × | ✓ |
| tau²-bench | × | × | × | × | ✓ | × | × | ✓ |
| EnterpriseOps-Gym | × | × | × | × | ✓ | × | × | ✓ |
| OSWorld | × | × | × | × | × | ✓ | ✓ | ✓ |
| WorkArena | × | × | × | × | × | × | ✓ | ✓ |
| TheAgentCompany | ✓ | ✓ | × | × | × | ✓ | × | ✓ |
| AppWorld | × | × | × | × | × | ✓ | × | ✓ |
| MCP-Atlas | × | × | × | × | × | × | × | ✓ |
| Remote Labor Index | ✓ | ✓ | ✓ | × | × | ✓ | ✓ | ✓ |
| WildClawBench | × | × | × | × | × | ✓ | ✓ | ✓ |
| $OneMillion-Bench | × | × | × | ✓ | ✓ | × | × | ✓ |
| AgentIF-OneDay | ✓ | ✓ | ✓ | ✓ | × | × | × | × |
| CentaurBench | × | × | × | × | ✓ | × | × | ✓ |
| Business | | | | | | | | |
| CRMArena | × | × | × | × | ✓ | × | × | ✓ |
| CRMArena-Pro | × | × | × | × | ✓ | × | × | ✓ |
| EnterpriseBench | × | × | × | × | ✓ | ✓ | × | ✓ |
| CI-Work | × | × | × | × | ✓ | × | × | ✓ |
| Field | | | | | | | | |
| FieldWorkArena | × | × | × | × | × | × | ✓ | × |
| E-commerce | | | | | | | | |
| E-Commerce Bench | × | × | × | × | ✓ | × | × | ✓ |

Methods for native artifacts make these requirements concrete. SheetAgent couples spreadsheet reasoning and manipulation through iterative information retrieval and planning, while SpreadsheetAgent first constructs structural sketches and localized representations before task-conditioned spreadsheet reasoning and targeted structural verification [40, 49]. PPTAgent examines reference-slide structure and content schemas before producing editable slides through iterative editing actions, rather than treating presentation generation as text-only composition [6]. For code work, AutoCodeRover exploits program structure and iterative code search to locate issue-relevant methods and classes; SWE-agent similarly makes repository navigation and inspection available through an agent-oriented interface [8, 66]. These approaches illustrate that understanding a working object includes locating the parts relevant to an intended edit and preserving its native semantics.

A task may require the agent to locate the appropriate document, spreadsheet, repository file, or application record before working with it. It must then interpret the object's content and state in a form suitable for subsequent decisions and operations [13, 19].

## 4.2 Reasoning and Decision Making

Reasoning and Decision Making enables a Working Agent to interpret task requirements, apply relevant knowledge, and determine appropriate actions. It encompasses reasoning about objectives and constraints, using domain-specific knowledge to analyze work materials, and selecting actions that advance the task under current conditions.

### Task-Oriented Reasoning

Working requests generally specify a desired outcome rather than a complete procedure. Task-oriented reasoning involves interpreting that outcome, identifying the relevant constraints, and determining which information is needed to solve the task. In software maintenance, for example, an agent must connect an issue description to the program behavior and code locations that may explain it; in spreadsheet work, it must interpret the requested calculation in terms of the workbook's actual data and formulas.

Recent methods integrate this reasoning with observations obtained during work. SWE-agent uses repository inspection and test feedback to guide problem investigation, while SheetAgent incorporates intermediate spreadsheet information into its subsequent analysis [8, 40]. Data Interpreter similarly uses results from executed data-science operations to refine its analysis and develop the task solution [67].

### Domain Knowledge

Working Agents depend on knowledge specific to the professional setting in which a task is performed. Domain Knowledge includes not only terminology and factual background but also the definitions, standards, policies, and constraints used to decide what a valid work result means. In financial reporting, units and accounting definitions determine whether calculations are comparable; in clinical environments, patient-specific records and domain conventions shape the interpretation of findings and the admissibility of subsequent actions; and in CRM or service operations, organizational policy determines which state changes are appropriate [10, 15, 44, 59].

Table 6 groups representative benchmarks across nine professional domains: finance, law, healthcare, scientific research, data science, software engineering, office productivity, enterprise operations, and customer service.

**Table 6** Representative benchmarks and their domain-knowledge requirements

| Domain | Representative benchmarks | Required knowledge | Task setting | Evidence |
|---|---|---|---|---|
| Finance | FinQA; TAT-QA; ConvFinQA; Finance Agent Benchmark | Financial statements, accounting conventions, quantitative analysis, financial operations | Financial QA and executable finance work | [10, 11, 68, 69] |
| Law | LegalBench; LawBench | Legal concepts, statutory rules, case facts, legal interpretation | Legal reasoning and analysis | [23, 70] |
| Healthcare | MedAgentBench; HealthAgentBench | Clinical concepts, electronic health records, patient-specific constraints | Medical reasoning and EHR-based tasks | [59, 71] |
| Scientific Research | ScienceAgentBench; DiscoveryBench | Scientific concepts, hypotheses, experimental design, evidence analysis | Scientific inquiry and computational research | [36, 72] |
| Data Science | DSBench; Spider 2.0 | Statistics, data processing, database schemas, SQL | Executable analysis and data workflows | [34, 58] |
| Software Engineering | SWE-bench; Terminal-Bench | Program semantics, debugging, tests, development environments | Repository repair and terminal tasks | [33, 73] |
| Office Productivity | SpreadsheetBench; PPTC; PresentBench; OfficeBench | Spreadsheet formulas, document structure, presentation design | Native office artifacts and editing | [37, 38, 42, 64] |
| Enterprise Operations | CRMArena; CRMArena-Pro; WorkArena++; TheAgentCompany | Organizational processes, entities, enterprise policies, workplace context | Enterprise records and organizational work | [41, 43–45] |
| Customer Service | $\tau$-bench; $\tau^2$-bench | Customer policies, service procedures, account and case states | Customer dialogue and tool-mediated service | [15, 16] |

Methods for domain-specific work combine professional knowledge with task-level analysis. SheetAgent couples spreadsheet retrieval with numerical interpretation; Data Interpreter uses executed analysis to revise intermediate results; and CodeAct supports decisions informed by executable computations [40, 67, 74].

### Decision and Action Selection

After interpreting the task and analyzing the available information, an agent must select an action that advances the work. ReAct interleaves reasoning with environment interaction, allowing subsequent choices to depend on newly obtained observations; CodeAct expresses operations as executable code and uses their results to inform later actions [74, 75]. In software tasks, SWE-agent alternates between inspecting source files, editing code, and running tests according to its findings; in spreadsheet tasks, SheetAgent selects further operations based on intermediate analysis [8, 40].

Action selection also depends on the kind of progress needed at a given stage. An agent may inspect additional material, run a calculation, modify an artifact, or provide the requested result. The ability to make these choices in light of task progress is central to interactive working agents, whose next action cannot always be specified before execution begins.

## 4.3 Planning and Coordination

Planning and Coordination turns a work objective into a feasible organization of subgoals, actions, prerequisites, and interactions with other parties. A plan is more than an ordered list: it must preserve the original goal,

identify which steps depend on earlier results or approvals, and account for branches whose conditions become known only during execution. PlanBench, TravelPlanner, and WorkArena++ expose complementary aspects of these demands [41, 76, 77].

### Task Decomposition and Planning

Complex work needs decomposition that reflects real dependencies. Data Interpreter represents data-science tasks as hierarchical graphs and refines executable subproblems, allowing later work to use intermediate computations [67]. Agent S2 uses hierarchical planning over multiple timescales for computer-use tasks, so immediate actions can be revised while a broader goal is maintained [61]. SheetAgent similarly integrates task planning with iterative spreadsheet reasoning and manipulation [40]. These approaches illustrate different realizations of the same requirement: subgoals must remain linked to the final deliverable, and prerequisite information must be available when each step is taken.

For recurring or structurally similar tasks, recent methods also investigate how execution procedures can be learned and improved. Agent Workflow Memory extracts reusable workflows from earlier web-task trajectories, while AFlow searches for effective code-represented workflow structures using feedback from their execution [47, 78]. These approaches extend planning beyond a single task by improving how agents organize subsequent work.

### Coordination and Collaboration

Many work tasks involve contributions from multiple participants or systems. Coordination requires agents to distribute subtasks, exchange task-relevant information, and integrate results into the ongoing work. AutoGen supports collaboration through conversations among specialized agents, while MetaGPT organizes software development tasks through role-based cooperation and shared artifacts [79, 80].

In enterprise settings, coordination further involves interactions with colleagues, business applications, and organizational processes. TheAgentCompany provides tasks situated in a simulated workplace, and WorkArena++ includes composite enterprise tasks that connect multiple operations and stages [41, 43]. These settings require an agent to track the progress of dependent work and use the contributions of other participants when continuing its own tasks.

## 4.4 Execution and State Management

Execution and State Management concerns performing task-directed operations on work objects and applications while keeping track of the resulting work state. Working Agents may edit native artifacts, execute code, interact with graphical applications, or update structured business records, often across multiple dependent steps [4, 5, 8, 13].

### Action Execution

Executing work requires mapping an intended operation to the correct entity and a valid set of arguments. SheetCopilot and SheetAgent perform operations over spreadsheet structure; PPTAgent generates slides through editable artifact actions; SWE-agent exposes repository navigation, line editing, and tests through an agent-computer interface [5, 6, 8, 40]. For a broader action surface, CodeAct expresses composable operations as executable code, while OpenHands supports software-work agents that interact with files, terminals, and web resources [74, 81]. GUI-oriented methods, including Agent S2 and UFO2, address the translation of desktop intentions into interface actions and evolving system state [61, 82]. These methods differ in their action surfaces, but all make concrete work effects, rather than plausible action descriptions, the target of execution.

The operation types in Table 7 distinguish retrieval, generation, editing, extraction, conversion, analysis, dialogue, and state change in the supplied benchmark compilation. These are semantic operations that can co-occur in one Task; their frequencies or tool-call counts are not substitutes for the correctness of the resulting work.

**Table 7** Benchmark × operation types

| Benchmark | QA / retrieval | Generation | Editing | Extraction | Conversion | Analysis / calculation | Multi-turn dialogue | State change |
|---|---|---|---|---|---|---|---|---|
| Finance | | | | | | | | |
| SpreadsheetBench | × | ✓ | ✓ | × | × | ✓ | × | × |
| BankerToolBench | × | × | × | ✓ | × | ✓ | × | ✓ |
| Finance Agent v2 | ✓ | × | × | ✓ | × | ✓ | × | × |
| Finance/Law | | | | | | | | |
| PRBench | ✓ | ✓ | × | × | × | ✓ | × | × |
| Law | | | | | | | | |
| LegalBench | ✓ | × | × | × | × | × | × | × |
| Harvey LAB | × | ✓ | × | ✓ | ✓ | × | × | × |
| Legal Research (vals) | ✓ | ✓ | × | ✓ | × | × | × | × |
| Healthcare | | | | | | | | |
| MedCode | ✓ | × | × | ✓ | × | × | × | × |
| MedScribe | × | ✓ | × | × | × | × | × | × |
| PSEBench | ✓ | × | × | × | × | × | × | ✓ |
| MLCR-AA | ✓ | × | × | ✓ | × | ✓ | × | × |
| Office | | | | | | | | |
| OfficeQA | ✓ | × | × | × | × | × | × | × |
| PresentBench | × | ✓ | × | × | × | × | × | × |
| PPTC | × | ✓ | ✓ | × | × | × | × | × |
| OmegaUse-OfficeVal | × | ✓ | ✓ | ✓ | ✓ | × | × | × |
| OfficeBench | × | ✓ | ✓ | × | × | × | × | ✓ |
| MMLongBench-Doc | ✓ | × | × | × | × | × | × | × |
| GDP.pdf | ✓ | × | × | × | × | ✓ | × | × |
| PPTEval/PPTAgent | × | ✓ | × | × | × | × | × | × |
| General | | | | | | | | |
| GDPval | × | ✓ | × | ✓ | × | ✓ | × | × |
| OmniaBench | × | ✓ | ✓ | × | × | × | × | ✓ |
| Workspace-Bench | × | ✓ | × | ✓ | ✓ | × | × | × |
| JOBBench | × | × | × | × | × | ✓ | × | ✓ |
| AutomationBench | × | × | × | × | × | × | ✓ | ✓ |
| APEX-Agents | × | ✓ | × | ✓ | × | ✓ | × | ✓ |
| tau²-bench | × | × | × | × | × | × | ✓ | ✓ |
| EnterpriseOps-Gym | × | × | × | × | × | × | × | ✓ |
| OSWorld | × | × | ✓ | × | × | × | × | ✓ |
| WorkArena | × | × | × | × | × | × | ✓ | ✓ |
| TheAgentCompany | × | ✓ | × | × | × | × | ✓ | ✓ |
| AppWorld | × | × | × | × | × | × | ✓ | ✓ |
| MCP-Atlas | × | × | × | × | × | × | × | ✓ |
| Remote Labor Index | × | ✓ | × | × | × | ✓ | × | ✓ |
| WildClawBench | × | ✓ | × | × | × | × | ✓ | ✓ |
| $OneMillion-Bench | ✓ | × | × | ✓ | × | ✓ | × | × |
| AgentIF-OneDay | × | ✓ | ✓ | ✓ | × | × | × | × |
| CentaurBench | × | ✓ | × | × | × | ✓ | × | × |
| Business | | | | | | | | |
| CRMArena | ✓ | × | × | × | × | × | ✓ | ✓ |
| CRMArena-Pro | × | × | × | × | × | × | ✓ | ✓ |
| EnterpriseBench | × | ✓ | × | × | × | × | × | ✓ |
| CI-Work | ✓ | × | × | × | × | × | × | ✓ |
| Field | | | | | | | | |
| FieldWorkArena | ✓ | × | × | × | × | ✓ | × | × |
| E-commerce | | | | | | | | |
| E-Commerce Bench | × | × | × | × | × | × | ✓ | ✓ |

## State Management and Continuity

Once execution begins, the Agent must track the versions and identities of changed objects, completed subgoals, intermediate artifacts, unconfirmed effects, and obligations that remain open. The significance of memory in a Working Agent lies in its effect on subsequent behavior: recalling that a user requested an updated report does not establish that the current report was saved or delivered. MemGPT and Mem0 explore persistent memory, while LongMemEval and LoCoMo examine retained information use [83–86]. InfiAgent specifically externalizes long-running task state to a file-centric workspace snapshot and reconstructs a bounded reasoning context, illustrating an approach to preserving continuity without retaining an unbounded history in each

model call [50].

State continuity must be reconciled with the external Workspace. A timeout after a non-idempotent operation may conceal a successful commit; blind retry can duplicate a payment, message, or record. Conversely, a successful tool response does not guarantee that the intended artifact was saved to the correct path or version. AppWorld and ToolSandbox expose state-dependent action effects, while AgentDojo studies unintended or disallowed effects in tool-using agents [12, 13, 87]. Reliable execution therefore requires both producing the requested effect and avoiding changes to unrelated or unauthorized state.

## 4.5 Verification, Recovery, and Replanning

Verification, Recovery, and Replanning determines whether the work actually satisfies its requirements and what to do when it does not. Runtime verification is undertaken for the sake of ongoing work, rather than merely to produce a benchmark score: it may cause an output to be accepted, an error to be repaired, a pending effect to be reconciled, or a plan to be revised. The evidence used must match the kind of work performed, whether an analytical conclusion, a native artifact, a repository modification, or a business-state transition [13, 33, 38, 64].

### Work Verification

A generator's own reassessment can help, but it may repeat the same blind spots that produced the error. Self-Refine illustrates iterative revision from feedback, whereas CRITIC uses external tools to check and amend outputs [88, 89]. Work-object methods show why checking must be tailored to actual artifacts: PPTAgent couples generation with PPTEval's content, design, and coherence assessment; DeepPresenter renders intermediate slides and uses observations of the rendered artifact to guide revision; SpreadsheetAgent verifies intermediate spreadsheet structure through targeted inspection [6, 49, 90]. These approaches link checking to the artifact or environment rather than relying only on another language assertion.

Program checks provide another form of evidence, but their coverage is bounded by the properties they test. SWE-agent runs tests during software work; AutoCodeRover combines structure-aware search with fault localization and program modification; Agentless provides a contrasting, deliberately constrained localization–repair–validation baseline rather than an autonomous agent architecture [8, 66, 91]. These examples distinguish verification evidence from a guarantee of complete correctness: a test suite can pass without exercising an omitted requirement, and an application-state check can confirm a final value without establishing that every intermediate action was authorized.

### Error Recovery and Replanning

When a check fails, the Agent must identify the affected requirement and select an appropriate response. A bad formula may call for a local correction, a failing test may call for a small patch, and an uncertain non-idempotent commit calls for reconciliation rather than immediate replay. Reflexion and LATS illustrate feedback-guided improvement and search, while DeepPresenter demonstrates artifact-specific correction driven by rendered-state observations [90, 92, 93]. Effective recovery preserves already-correct work and avoids introducing regressions during repair.

Replanning is needed when the original route has become invalid, such as after a changed Task, a missing prerequisite, or an unexpected Workspace state. The Agent should resume from the current committed state rather than replaying its earlier trajectory. Verification, Recovery, and Replanning thus closes the working loop by checking real outcomes, diagnosing deviations, and directing only the necessary continuation of work.

Taken together, these five capabilities characterize the reliability of a Working Agent as a whole: observing and understanding the work environment, choosing supported actions, organizing dependent work, producing intended effects, and correcting errors against the current state of the Task and Workspace. Their specific realization may vary, but the capability requirement remains whether the delegated work is completed correctly and with appropriate control over its consequences.

# 5 Optimization: From Work Experience to System Capability

Optimization determines how experience changes the system that performs subsequent work. The work forms in Section 3 describe how that work is organized, while the capabilities in Section 4 identify the functions needed to carry it out. A failure within one function can have several causes. A spreadsheet agent may use the wrong value because its interface hid a formula, because it misunderstood the visible formula, or because the workbook changed after inspection. These causes suggest different interventions: expose the missing evidence, train the relevant decision, or refresh the state before acting. Choosing an update therefore begins with locating the failure in the work process.

Using the notation of Section 2.3, work experience comprises $(T, E_0, \tau, E_t, O, S)$: the task, initial workspace, interaction trajectory, resulting state, work outcome, and verification assessment. Different learning methods use different parts of this record. Demonstrations supply behavior to imitate; execution reveals the consequences of policy decisions; feedback can supervise a teacher or guide revisions to tools, prompts, and procedures. For comparing these methods, three questions matter: which evidence is available to the learner, which component changes, and which improvement in subsequent work is measured? The model and harness used to collect the experience also matter, because they determine the observations and actions from which the system learns.

The working conditions in Section 2 shape both transfer and retention. Openness introduces variation in objects, interfaces, dependencies, and other actors. Persistence requires the system to retain valid decisions and unresolved obligations as work continues. These conditions occur across work forms and affect both model learning and harness design. An update that improves new tasks may still require a different procedure for work already underway; Section 7.2.3 develops this compatibility problem. The target of optimization is consequently better work under specified requirements and costs, with the affected work state included in the assessment.

Figure 3 organizes the optimization loop. Section 5.1 examines how environments, trajectories, and feedback make work learnable. Section 5.2 compares complementary ways to internalize capabilities in model parameters. Section 5.3 examines how execution evidence improves external support. Section 5.4 distinguishes training an executor within a harness, training a separate harness designer, and jointly updating the executor and its support. Their comparison addresses the chapter's central question: which part of an observed capability gap should be addressed by learning, by an external mechanism, or by coordinated changes to both?

## 5.1 Training Environments, Data, and Rewards

Useful training experience preserves the relationship between a work requirement, the decisions made to satisfy it, and evidence about the resulting change. Task and environment synthesis determine which situations the learner encounters; trajectory curation determines which decisions it can learn from; verification determines which properties an update rewards. Their value therefore depends on their alignment. For cross-object reasoning, the important variation lies in the relationships a task requires and the evidence available to check their use. This section compares methods by the experience they construct and the learning signals they make available.

Table 8 distinguishes the mechanisms used to construct experience from the evidence retained for learning.

### 5.1.1 Task and Environment Synthesis

**Reconstructing work from existing materials.** Recorded changes and completed artifacts provide natural starting points for training tasks. daVinci-Env automates repository exploration, Docker construction, evaluation-script generation, and test analysis to turn software histories into executable environments; difficulty-aware filtering then selects tasks with useful learning signals [111]. Spreadsheet-RL extracts paired initial and target workbooks from online forums and builds an Excel runtime for their execution [105]. In both cases, task construction preserves the relationship between the original work object, the requested change, and an inspectable result.

**Grounding occupational tasks in workspaces.** Broader professional work requires constructing the context around those objects. NexForge starts from capability requirements and task-form distributions, then

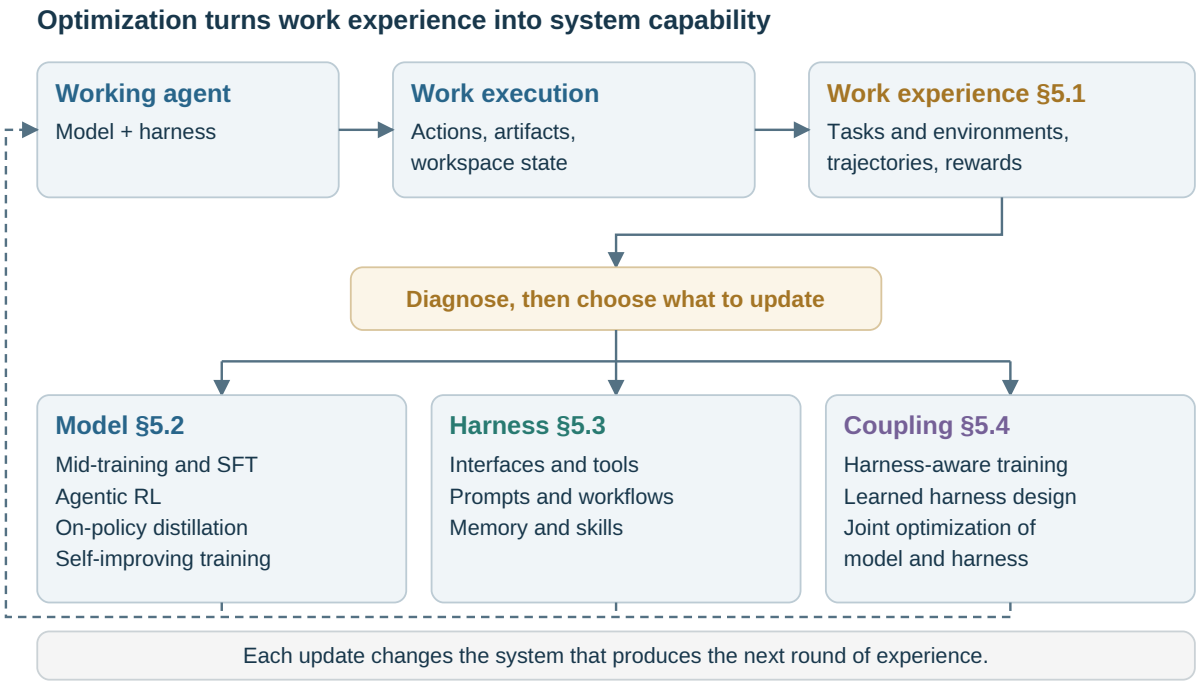



**Figure 3** From work experience to system capability. Execution and verification supply evidence for diagnosing a capability gap. Optimization can update the execution model, its external support, or their coupling. Evaluating the resulting system on work requirements closes the loop and supplies experience for subsequent updates.

**Table 8** Constructing work experience for optimization. Methods connect learning inputs to the work objects, state transitions, and acceptance criteria that make an execution useful for subsequent improvement.

| Learning input | Representative methods | Construction mechanism | Evidence retained for learning |
|---|---|---|---|
| Grounded tasks | NexForge, GraphForge, WorkGenesis [94–96] | Connect occupational requirements, real files, dependencies, and acceptance criteria | Source facts and dependencies connecting requests to work objects |
| Stateful environments | Agent World Model, EnvCraft, AgentMercury [97–99] | Generate executable tools, shared state, and business constraints | Shared state and constraints linking effects across services |
| Capability-driven tasks | Skill2Env [100] | Revise task blueprints and environments from solver evidence | Solver failures linked to revised task demands |
| Interaction records | EnterpriseLab, SEE [101, 102] | Compose verified tool dependencies or observed interface transitions | Action dependencies and observed recovery transitions |
| Curated experience | OpenThoughts-Agent, CurateEvo [103, 104] | Select and revise data using downstream learning and failure evidence | Downstream learning results and informative interaction segments |
| Executable outcome signals | Spreadsheet-RL, Anchor [105, 106] | Check recalculated workbooks or constrained business states | Recalculated values and constraints on the resulting state |
| Artifact-quality signals | AeSlides, SlideRL, CogEvol [107–109] | Evaluate layout, communicated content, and interaction behavior | Rendered properties and tested interaction behavior |
| Stage-level signals | VERA [110] | Restore intermediate workspaces and check stage requirements | Intermediate states linked to stage requirements |

retrieves or builds the materials and execution environments needed to instantiate them [94]. GraphForge assembles real files and represents their facts and cross-file dependencies in an evidence graph, from which it derives tasks and anchored evaluation criteria [95]. WorkGenesis uses occupational knowledge to retrieve public artifacts, supplies companion materials and work requests, and runs reference executions to locate and repair inconsistencies in tasks and rubrics [96]. These methods connect professional requirements to the evidence an agent must discover and use, giving workspace understanding a concrete role in training.

**Generating stateful business environments.** Cross-application work additionally requires executable transitions between states. EnvScaler synthesizes state definitions, tools, operating rules, and scenario-specific validators [112]. Agent World Model constructs database-backed environments with generated MCP interfaces, while EnvCraft uses dependencies within and across environments to compose executable task chains [97, 98]. AgentMercury constructs a business world before instantiating its tasks: entities, services, shared state, and cross-service invariants support multiple objectives in the same environment [99]. The distinction is the location of task dependencies: file-based tasks expose relationships among evidence and artifacts, whereas shared-state environments make one service's actions change what subsequent operations can validly do.

**Targeting capability gaps.** Environment generation can also be directed toward the competencies described in Section 4. Skill2Env translates reusable skills and capability demands into difficulty patterns and task blueprints, jointly constructing instructions, dependencies, workspace materials, and evaluators. Solver evidence drives iterative task hardening through coordinated changes to these components [100]. These construction routes address different gaps in experience. Reconstructed cases preserve concrete object transformations; generated workspaces expose chosen evidence and state dependencies; solver-directed synthesis concentrates practice on recurring failures. The useful unit of scale is thus a new learnable work situation, with corresponding execution and checks. Section 7.1.1 develops this connection between environment diversity and the capabilities that it can train.

### 5.1.2 Trajectory Collection and Curation

**Generating coherent interaction records.** Trajectories connect the constructed task to the decisions that complete it. EnterpriseLab derives tool dependency graphs from enterprise schemas, samples executable paths using identifiers returned by earlier calls, and generates task descriptions from those paths. Execution checks and diversity filtering select the resulting data [101]. TDScaling organizes MCP tool definitions into business clusters and uses scenario blueprints to guide simulated multi-agent interactions; feedback about domain coverage and reasoning patterns steers subsequent synthesis [113]. Their common objective is to retain dependencies between actions while covering different ways of organizing work.

**Collecting experience through exploration.** Exploration exposes states and transitions that task-first demonstrations may rarely visit. SEE builds a verified graph of mobile screens and elements through interaction, then composes paths into tasks, subgoals, and aligned action instructions. Its graph retains interruptions and recovery paths, allowing these events to appear in generated training trajectories [102]. The collected experience thus covers both progress toward an objective and the operations needed to resume progress after an unexpected state.

**Selecting experience for its learning role.** Successful demonstrations teach complete execution, while exploratory and recovery segments teach how to act under uncertainty. GraphForge admits teacher trajectories by checking their delivered artifacts against source-grounded criteria [95]. NexForge applies behavioral cleaning while retaining useful exploration and error recovery, including trajectories without a task-specific success certificate [94]. The selection unit follows the training objective: artifact acceptance supports outcome supervision, complete demonstrations support imitation, and recovery segments expose decisions after an error. Keeping observations and outcomes attached to the selected unit preserves the evidence needed to use it in each role.

**Adapting curation to the learner.** OpenThoughts-Agent evaluates task sources, mixtures, teachers, and rollout filters through downstream training. In its controlled experiments, minimum-turn filtering remains beneficial at a matched token budget, supporting the value of selecting sustained interactions within that setup [103]. CurateEvo makes curation itself adaptive: a code-writing agent revises augmentation, filtering, and

refinement procedures using development-set failures, then removes low-utility training turns. The resulting corpus supplies supervised examples, reinforcement-learning tasks, and inference-time memory [104]. These results favor evaluating a data filter through the learner it produces: a longer or cleaner trajectory is useful when its retained decisions address the learner's failures. Adaptive curation then revises that choice as the policy improves.

### 5.1.3 Reward and Verification Signals

**Checking completed state changes.** Verification turns the produced outcome into a learning signal. Spreadsheet-RL recalculates the edited workbook in Excel and compares the requested regions with a target workbook [105]. For business processes, Anchor compiles a solved constraint program into the instruction, initial ERP state, reference optimum, and state-based verifier. Its checks evaluate feasibility, objective quality, and traceable links between business records while allowing alternative valid plans [106]. These mechanisms ground feedback in what changed and whether the resulting state satisfies the work objective.

**Connecting judgments to source evidence.** Reports and other professional deliverables combine requirements about facts, coverage, and presentation. GraphForge ties rubric items to workspace evidence, combining deterministic artifact checks with an agent judge that inspects the delivered files [95]. WorkGenesis uses a reference deliverable to test whether each criterion is supported and decidable, then repairs defects in the task or rubric [96]. Agent World Model combines database changes with trajectory context to distinguish task progress, agent errors, and environment errors [97]. The comparison separates two needs: source grounding makes content judgments checkable, while execution context helps assign an unsatisfactory outcome to a decision or to the constructed environment. This distinction determines whether the next update should change behavior or repair training data.

**Evaluating rendered and interactive artifacts.** Visual work adds properties that become apparent when an artifact is viewed or used. AeSlides derives verifiable rewards from aspect ratio, whitespace, collisions, and visual balance [107]. SlideRL combines structural, rendering, and content scores with inverse-specification rewards: a judge reconstructs the intended brief from the produced deck [108]. CogEvol combines geometric rules with judgments of rendered content, and executes browser probes to assess interactive HTML. Its controlled reward comparison shows that adding interaction evidence improves the functionality rewarded during training [109]. Such signals extend optimization from producing valid files to producing artifacts that communicate and operate as required.

**Locating progress within a workflow.** Stage-level checks make feedback useful before final delivery. VERA restores intermediate workspace states, attaches binary rubric items to actions, execution effects, and artifacts, and verifies each training sandbox through code and interactive inspection. The resulting rewards measure stage completion while retaining mandatory requirements [110]. The verification mechanisms of Section 4.5 become distinct learning targets here: a layout score supervises composition, a state verifier supervises committed changes, and a stage check localizes progress within a workflow. These signals expose different kinds of failure in the work outcome and execution process, providing evidence for choosing a model or harness update. Sections 7.3.2 and 7.3.3 extend this connection to verification whose evidence and dependencies change during ongoing work.

## 5.2 Model Optimization

Model optimization is useful when recurring work failures require a better representation or decision policy. Its mechanisms differ in the supervision available for that change. Mid-training and supervised fine-tuning provide examples of relevant structure and behavior; reinforcement learning selects behavior through its consequences; on-policy distillation supplies guidance around student-generated behavior; and self-improving training turns execution evidence into subsequent supervision and tasks. These routes can be combined. The choice concerns the learning problem: what the model cannot yet represent or do, which states expose that weakness, and what feedback distinguishes a better decision.

Table 9 relates each route to the learning problem it addresses and the supervision it requires.

**Table 9** Complementary routes for model optimization. The distinction concerns how supervision is obtained and which behavior it teaches; a training pipeline can combine several routes.

| Route | Learning problem | Source of supervision | Representative methods |
|---|---|---|---|
| Mid-training and SFT | Learning recurring object structure and execution patterns | Reconstruction or imitation from prepared records and demonstrations | daVinci-Dev, TrajLong, CogEvol [109, 114, 115] |
| Agentic RL | Selecting actions by their consequences during execution | Outcome and intermediate rewards assigned to generated decisions | Spreadsheet-RL, ADMIRE, HiPER [105, 116, 117] |
| On-policy distillation | Guiding choices exposed by the current student policy | Teacher signals at student-induced states or for sampled responses | TCOD, LiteGUI, GHD [118–120] |
| Self-improving training | Converting recurring failures into subsequent learning opportunities | Feedback-conditioned targets, retrospective supervision, or new tasks | SDPO, RePro, Socratic-SWE [121–123] |

### 5.2.1 Mid-Training and Supervised Fine-Tuning

**Preparing representations for work.** Mid-training exposes models to the information dependencies that recur during execution. daVinci-Dev combines reconstructed repository histories, which retain issue descriptions, relevant files, and edits together, with trajectories collected through actual repository execution. This joins broad coverage of development patterns with grounded tool feedback [114]. Function-aware fill-in-the-middle training approaches the same dependency problem through code structure: it selects functions using program dependencies and their recoverability from surrounding code, then trains models to reconstruct them. Subsequent agent training benefits from the resulting ability to connect external results with their downstream use [124].

Workspace understanding also requires finding evidence across records and tracking objects through successive modifications. TrajLong compiles interaction histories into supervision for evidence retrieval, aggregation across sources, and reconstruction of changing file state. Its state-tracking tasks replay successful edits while preserving untouched content, making object continuity an explicit learning target. With the backbone, token budget, and downstream SFT held fixed, compiled supervision improves over raw combined trajectories on all seven benchmarks in its construction ablation [115]. The comparison shows why the training unit matters: raw histories expose the sequence of interactions, whereas compiled targets explicitly require recovering the evidence or object state needed for a later decision.

**Learning work-object production and interactive execution.** SFT learns work behavior from verified examples. CogEvol trains on structured slide scene graphs and interactive HTML derived from production experience. Its targets pair a user brief and output requirements with an artifact that has passed parsing, rendering, and task-specific checks, teaching the model how to express a request in an executable work product [109]. UI-Mate applies supervision to the decision turn: the current screenshot and interaction history condition a response containing reasoning and a GUI action. Its training mixture spans atomic operations, single-application workflows, and cross-application tasks, with sampling balanced by capability and task length [125]. Object-level targets teach what to produce, while trajectory targets teach how to reach and revise the required state through interaction.

These targets are suited to different recurring errors. Object reconstruction directly supervises lost or misread state; artifact targets supervise the representation of a deliverable; decision-turn demonstrations supervise an action conditioned on observations. Their common requirement is an informative target at the chosen unit of work. When good actions are better distinguished through execution than specified in advance, reinforcement learning supplies a complementary signal.

### 5.2.2 Agentic Reinforcement Learning

Agentic reinforcement learning updates the policy from its interactions with an executable work environment. The model selects an action, observes its effect, and receives feedback tied to task progress or completion. For

working agents, this connects the outcome signals of Section 5.1.3 to the decisions that produced a changed document, workbook, interface, or application state.

**Optimizing delivered work.** Spreadsheet-RL trains through a Python sandbox connected to Microsoft Excel. The agent inspects and modifies a starting workbook over multiple turns, and the final workbook is compared with the reference outcome. This allows policy optimization to learn sequences of operations whose usefulness is determined by the resulting spreadsheet [105]. SlideRL trains an agentic slide-generation workflow with structural, visual, and content feedback, including a reward based on reconstructing the requested specification from the deck [108].

Reinforcement learning can also optimize an artifact-producing component within a work system. AeSlides applies GRPO to verifiable layout properties of generated slides, while CogEvol optimizes rendered slides and interactive pages [107, 109]. For layered design documents, MiLDEAgent trains a multimodal reasoner to select layers for modification and formulate editing instructions for a fixed image editor. Rewards supervise layer decisions, instruction agreement, and response format, locating the trainable capability in object-specific planning [126].

**Assigning credit across decisions.** A final success signal leaves open which intermediate choices advanced the work. ADMIRE extracts ordered milestones from successful GUI trajectories and updates them as better execution patterns emerge. It concentrates rewards on useful steps in successful attempts and provides partial-progress signals for failed attempts, gradually reducing the milestone contribution as training proceeds [116]. HiPER addresses credit through policy structure: one model generates a subgoal, decides whether to retain or replace it, and selects primitive actions. Hierarchical value estimates connect action-level learning within a subgoal to progress at subgoal boundaries [117]. The two approaches supply intermediate guidance through different mechanisms: adaptive execution landmarks and learned temporal abstraction.

**Learning under variable execution cost.** Practical work generates trajectories of unequal length, often with long unsuccessful runs. UI-Mate centers outcome advantages over decision turns to account for this difference. Its optional process-credit mechanism identifies milestones, causal errors, recovery, and redundant steps, then redistributes the task-level signal over informative decisions. Adaptive task sampling further directs interaction toward tasks with useful learning signal [125]. The credit-assignment choice depends on what execution reveals. Observable intermediate achievements support milestone rewards; reusable subgoals motivate hierarchical decisions; variable trajectory lengths motivate allocating credit and sampling effort across turns and tasks. Their gains should be assessed through the work outcome and interaction cost that motivated the refinement. When a teacher can identify a useful continuation at a visited state, distillation offers a more direct source of action guidance.

### 5.2.3 On-Policy Distillation

On-policy distillation obtains supervision after the student has generated behavior, allowing a teacher to guide decisions that arise under the student's current policy. In working environments, those decisions include unexpected tool results, incomplete modifications, and alternative routes to the same outcome. Methods differ in whether student generation determines the visited environment states or the candidate response within a supplied interaction prefix, and in how guidance is scheduled and constructed.

**Guiding student-induced trajectories.** Revisiting DAgger collects teacher actions at every visited state while mixing teacher and student execution. One variant chooses the executor at each turn; another lets the student generate a prefix before the teacher takes over. Student control increases during training, bringing supervision closer to deployment behavior [127]. TCOD instead adjusts interaction depth, progressively extending student rollouts or replacing teacher prefixes with student execution [118]. Guided-OPD decays teacher-controlled turns to zero and combines teacher-turn imitation with distillation on student turns [128]. These approaches use temporary guidance to maintain informative interaction while training the student to act independently.

**Adapting the teacher signal.** The same workspace state can permit several successful actions. LiteGUI samples a student response at an annotated GUI state, selects the human-verified valid action that best matches it, and supplies that reference only to the teacher. The resulting distribution guides the student

toward a valid continuation while accommodating alternative ways to complete the step [119]. Gated Hindsight Distillation uses the next screenshot from an offline successful trajectory as additional teacher context. It applies distillation when the student's action is incorrect and the hindsight-conditioned teacher recovers the demonstrated action, extracting supervision from evidence revealed by the operation [120]. Both methods improve teacher guidance by conditioning it on information available during training.

Guidance can also vary with the student's behavior within a trajectory. SOD measures student–teacher divergence at successive tool-use steps and adjusts their distillation weights, attenuating supervision as divergence grows and restoring its weight when alignment improves [129]. This comparison separates state coverage from target quality. Teacher intervention during student rollouts addresses the states reached by an imperfect policy; reference selection and hindsight improve supervision at the states already supplied. When execution errors compound, rollout-based guidance exposes their downstream consequences. When several local actions are valid, adapting the teacher target avoids treating one demonstrated path as the only useful continuation. The teacher can also be an instance of the model given richer experience, connecting distillation to self-improving training.

### 5.2.4 Self-Improving Model Training

Self-improving training closes the loop between execution and the next model update. The agent's attempts become material for new supervision, revised curricula, or both. The durable gain lies in what subsequent executions can reuse: improved interpretation of feedback, better anticipation of errors, and more effective action policies.

**Turning feedback and hindsight into supervision.** SDPO conditions a copy of the current policy on textual feedback, such as execution errors, and distills its feedback-informed predictions into the acting policy [121]. Skill-SD summarizes previous attempts into compact descriptions of successful behavior, mistakes, and workflows; these descriptions condition a self-teacher during training, while the student learns from the ordinary task input [130]. Self-Retrospection Distillation instead uses completed attempts to supervise predictions of useful knowledge and potential pitfalls before interaction. Successful and failed trajectories provide complementary targets, including when equal rewards across a rollout group leave little relative policy-gradient signal [131]. These methods give accumulated experience a role beyond accepting or rejecting an entire attempt.

RePro learns a related capability directly within the interaction policy. After a supervised warmup, the agent acts while estimating progress, then reassesses that progress using the completed trajectory and its outcome. Policy optimization combines environmental feedback with retrospective progress differences and agreement between online and retrospective estimates. Experiments in WebShop and other interactive environments examine how learning to assess progress improves subsequent action selection [122]. Here the model learns to extract training signals from its own execution evidence.

**Adapting the next training tasks.** Socratic-SWE uses repository-solving traces to extract recurring failures and effective repair strategies. These skills guide new task generation in executable repositories; candidates pass execution checks and receive a gradient-alignment score estimating their usefulness for improving the solver. The updated solver supplies the next round of traces. Removing its skill registry lowers the reported third-iteration SWE-bench Verified result from 50.4% to 46.2%, showing the contribution of experience-guided task construction within that training setup [123]. Task generation and policy improvement thus become coupled through observed capability gaps.

The reusable information differs across these loops: feedback-conditioned distillation teaches how to act on discovered evidence, retrospective learning teaches how to anticipate progress and pitfalls, and task generation changes where the next policy update receives practice. Their common source of improvement is information that the acting policy did not exploit in its original attempt. Weight updates make this experience available through the policy on later tasks; external tools, context, and skills offer another place to retain it. The next section examines that choice before Section 5.4 considers coupled updates. Preserving active obligations during either form of adaptation is developed in Section 7.2.3.

## 5.3 Harness Optimization

Harness optimization is useful when a failure can be addressed by changing what the model observes, which operations it can invoke, or how execution is organized. A dependency omitted from context can be exposed through a different representation or tool description; a repeated prerequisite violation may call for an executable check; a reusable recovery procedure can become a skill. These interventions expose an important allocation choice: a system may improve by supplying information that its policy previously had to reconstruct or by enforcing constraints through the harness. The methods below optimize these external components from execution feedback, with completed work and its cost determining the value of a change.

Table 10 summarizes the components that feedback can improve.

**Table 10** Harness optimization converts execution feedback into reusable external support. Each target changes how subsequent work is observed, organized, or performed.

| Editable component | Optimization operation | Feedback used | Representative methods |
|---|---|---|---|
| Interfaces and tools | Filter observations, revise descriptions, repair arguments and check prerequisites | Tool traces, state differences, and execution failures | TACO, JTPRO, PRISM [132–134] |
| Prompts and workflows | Revise instructions, edit control code, compose and select harnesses | Task outcomes, failure replay, and artifact evaluation | GEPA, Meta-Harness, AutoDesign [135–137] |
| Memory and skills | Extract lessons, revise procedures, manage dependencies and context use | Successes and failures, scored rollouts, and skill interactions | ReasoningBank, SkillOpt, SkillSmith [138–140] |

### 5.3.1 Interface and Tool Optimization

**Observation and action representations.** Interface optimization changes the information available at a decision and the operations through which that decision affects the workspace. TACO addresses the observation side for terminal work: it discovers structured filtering rules from trajectories, refines them within a task, and shares effective rules across tasks. Rules specify which outputs to retain or compress, while explicit error signals remain available for recovery [132]. The optimization problem is selective information preservation: compression reduces the context consumed by execution while retained state and error evidence support the next decision.

**Tool descriptions and calling constraints.** Tool failures can originate in ambiguous descriptions, poorly specified arguments, or missing dependencies between calls. Trace-Free+ trains a description generator with a curriculum that moves from trace-conditioned examples toward description-only inputs, allowing learned interface improvements to transfer to unseen tools [141]. JTPRO uses execution-based reflection to jointly revise global instructions and per-tool descriptions, coordinating tool selection with argument construction [133]. These routes respectively learn reusable interface conventions and adapt a particular tool inventory from observed failures.

**Executable interface repair.** Some recurring errors are addressed at the tool boundary itself. PRISM clusters failures from tool sequences, state differences, and error traces, then routes candidate repairs to prompts, middleware, or both. Its middleware edits correct malformed arguments, return errors that support retry, or check prerequisites before allowing an operation [134]. These interventions operate at different points in a failure. Descriptions help the model choose and form a call; middleware changes the call or checks its prerequisites before the workspace is modified. When the relevant constraint is executable, a boundary check can enforce it on each invocation. When the difficulty is selecting among valid operations, the policy still needs an informative description and workspace evidence.

### 5.3.2 Prompt and Workflow Optimization

**From instructions to executable control.** Prompt optimization uses observed outcomes to change the guidance supplied to a model. GEPA reflects on trajectories containing reasoning, tool calls, and returned

results, proposes prompt revisions, and combines complementary improvements from a Pareto frontier [135]. Meta-Harness expands the editable object to harness code. Its proposer can inspect the source, scores, and execution traces of previous candidates through a filesystem, then submit a new implementation for evaluation [136]. This permits changes to context construction and control flow as well as instructions. MESH-Harness introduces modular reuse: role-specific implementations are recombined through bandit-guided search, and validation traces guide local edits [142]. The editable surface determines which failures can be repaired: prompts change guidance, control code changes the execution procedure, and modular search changes how procedures are composed. A missing instruction and a missing verification stage therefore call for different search spaces.

**Optimizing artifact-producing workflows.** AutoDesign applies this approach to transforming research papers into editable posters. Its inner loop constructs and revises an artifact using rendering, validation, and critic feedback; its outer loop identifies recurring failures across tasks and proposes bounded changes to the design harness. Candidate updates are evaluated against criteria grounded in reference artifacts and accepted through training and development checks [137]. The resulting improvement concerns a reusable production process: selecting source evidence, arranging content, rendering the output, and revising the design. The inner loop improves a particular deliverable; the outer loop tests whether the repair should become part of the procedure used for subsequent deliverables. This separates artifact revision from reusable workflow improvement.

**Learning from deployment and directing further search.** RHO selects challenging and diverse tasks from historical trajectories, re-solves them, and uses self-validation and pairwise preference to select harness updates [143]. VERSE strengthens the optimizer's ability to diagnose those failures by permitting draft tests, failure replay, and interventions on suspected steps; it updates both the executor harness and its own optimization procedures while keeping model weights fixed [144]. ActiveSaddler addresses which experiences should drive the next update. It abstracts recurring failures into optimization targets and reallocates effort according to their changing learning value [145]. For heterogeneous task streams, Adaptive Auto-Harness maintains multiple harnesses and routes incoming tasks among them [146]. Their choices concern distinct sources of variation: replay and intervention localize failures within a procedure, active selection changes which failures receive optimization effort, and routing supplies different procedures to different tasks. Routing is particularly relevant when task groups benefit from different configurations; a single revised harness is useful when the same repair transfers across those groups.

### 5.3.3 Memory and Skill Optimization

**Extracting useful experience.** Memory optimization determines what an agent should retain from completed work and how that information should influence later decisions. ReasoningBank distills strategies from self-judged successes and failures, retrieves relevant lessons for subsequent tasks, and uses diverse attempts to improve the experience available for consolidation [138]. ACE organizes accumulated experience as a playbook maintained through generation, reflection, and incremental curation [147]. Their retained object is a lesson or playbook entry rather than the full trajectory. This favors reuse of decision-relevant experience when only a small part of a previous execution is useful to the new task.

**Optimizing procedural artifacts.** Skills give this guidance a more explicit procedural form. SkillOpt treats a skill document as an editable optimization object: an optimizer proposes bounded additions, deletions, and replacements from scored rollouts, and validation determines which edits are retained [139]. Rejected edits also inform subsequent proposals. This allows a fixed executor to acquire a better procedure through repeated use and revision, with the resulting skill available for reuse across tasks and compatible harnesses.

**Managing a growing capability library.** As skills accumulate, optimization must also account for overlap, dependencies, and the context consumed by their use. SkillSmith estimates skill effects and interactions under a context budget, coordinates changes across dependent skills and tools, and moves reusable logic across the skill–tool boundary [140]. Stable operations can thereby become executable tools while skills retain the judgment and orchestration needed to use them. This suggests a practical division of retained experience: memory supplies relevant lessons, skills organize their application, and tools execute operations that can be specified and checked. Their value depends on the executor's ability to retrieve and use them, and on the

context and execution cost they add. Section 5.4 examines how training can adapt that executor together with its external support; Section 7.2.3 considers the further requirement of preserving active commitments as either changes.

## 5.4 Model--Harness Coupling and Joint Optimization

A model learns from the observations and actions its harness makes available; the value of a harness, in turn, depends on the model that uses it. This coupling takes three forms: training an executor within a given harness, training a designer to construct external support for an executor, and jointly adapting the executor and its harness. To distinguish their optimization targets, let $M_\theta$ be the execution model of Section 2.3, $H_h$ the harness with configuration $h$, and $D_\phi$ a separate designer when one is used. The configuration $h$ includes discrete objects such as prompts, tools, control rules, and skills. Harness-aware training updates $\theta$ under a supplied interface; learned design updates $\phi$ to produce useful configurations; joint optimization updates both $\theta$ and $h$ through feedback from their combined execution. Table 11 makes these update relations explicit.

**Table 11** Model–harness coupling: updated variables, feedback paths, and update schedules. $\theta$ denotes executor weights, $\phi$ designer weights, and $h$ external configuration. A learned designer can improve a frozen executor's harness without updating that executor.

| Methods | Updated and fixed variables | Feedback path | Update schedule |
|---|---|---|---|
| UI-Copilot [148] | Update $\theta$; supplied interface and copilot fixed | Tool-label and action rewards supervise the executor | Single-turn tool-choice and multi-turn action batches |
| JIT-Agent, CODESKILL, DMRL [149–151] | Optimize $\phi$; generate $h$; executor $\theta$ fixed | Downstream execution evaluates designs and trains the designer | Across harness candidates, skill edits, or operational experiments |
| UI-Mem, SkillRL [152, 153] | Update $\theta$ and external experience in $h$ | Memory guides rollouts; rollouts train the policy and refresh memory | Repeated policy and experience-update rounds |
| Fast-Slow Training [154] | Update $\theta$ and prompt population in $h$; freeze the other during each phase | Current policy evaluates prompts; revised prompts condition RL | Prompt search between blocks of weight updates |
| SIA [155] | Update $\theta$ and scaffold $h$; freeze the other at each step | Trajectories and reward dynamics select scaffold editing or RL | Feedback-driven choice of update type |
| VERA [110] | Update $\theta$ and skills in $h$; tools and execution rules fixed | Stage rubrics guide training; paired runs assess skill edits | Alternate model and skill rounds with the other component frozen |

### 5.4.1 Harness-Aware Model Training

**Learning within the deployed interface.** A training harness defines what the policy can observe, request, and change. Spreadsheet-RL trains with a concrete spreadsheet toolset and routing rules, connecting learned decisions to workbook operations [105]. UI-Copilot separates detailed observations from concise progress summaries and provides retrieval and calculation through a copilot. Its optimization separates single-turn tool choice on supplied histories from multi-turn action prediction on self-generated histories; the copilot is invoked at execution time [148]. In both cases, training learns a division of work between the policy and supplied operations. Changing that division at deployment changes the decision problem: a policy trained to request a computed result faces a different task when given only raw records.

**Learning to extend the action space.** SMITH trains a single policy on interleaved tool-building and tool-using tasks. Generated tools enter a pool for subsequent use, while rewards evaluate schema validity, executable code, and downstream outcomes [156]. The gradient update targets the policy, including its ability to construct tools; the resulting tools become external resources for later execution. This route is useful when many tasks share operations that can be packaged into executable interfaces. Its training signal must cover both whether a tool runs and whether using it improves the work, since syntactic validity alone does not establish a useful operation.

### 5.4.2 Learned Harness Design

**Amortizing harness construction.** A learned designer transfers optimization effort across tasks by predicting configurations for a downstream executor. JIT-Agent trains $D_\phi$ to generate compositional harnesses covering memory, planning, action, and capability orchestration. Supervised synthesis, preference learning, repair trajectories, and online optimization use execution reward, latency, and cost from a frozen executor [149]. The executor supplies the test of a design; learning resides in the generator. This mechanism can reduce repeated search when tasks share useful control structures, while its deployment cost still includes harness generation, repair, and execution.

**Learning what experience to preserve.** Skill designers optimize a narrower external object. Skill-$\alpha$ learns progressive edits from documents or execution experience, using a rollback reward that compares downstream runs with the original and revised skill on an anchored query [157]. CODESKILL trains a skill-management policy to extract, revise, merge, and maintain coding skills, combining rubric feedback with verified execution by a frozen downstream agent [150]. DMRL extends skill editing to advertising-system parameter adjustment: an upper-level editor revises a document, a frozen task agent executes the procedure, and policy optimization uses experimental feedback and a predictor of longer-term outcomes [151]. These methods learn external instructions through their consequences for another agent. Their transfer therefore depends on both the work distribution and the receiving executor's ability to follow those instructions.

### 5.4.3 Joint Model--Harness Optimization

**Coordinating updates across timescales.** Joint optimization makes external adaptation and executor learning respond to one another. Fast-Slow Training alternates reflective optimization of a prompt population with reinforcement learning of model weights. Prompts are evaluated under the current policy, then held fixed for a block of policy updates before search resumes [154]. Across its reasoning tasks, this combination reaches comparable reward with lower policy divergence from the base model, measured by KL, than RL alone, supporting a division between rapid contextual adaptation and more persistent parameter learning. Optimas addresses credit assignment in compound systems through component-specific reward functions aligned with global performance; prompts, hyperparameters, and trainable model components can then use different optimizers [158]. The shared challenge is to refresh feedback as the rest of the system changes: a prompt or local reward fitted to an earlier policy may guide its successor differently.

External experience can participate in the same loop. UI-Mem extracts workflows, skills, and failure patterns during GUI reinforcement learning, varies memory guidance across rollout groups, and reduces guidance as the policy improves [152]. SkillRL updates a hierarchical skill library while training the policy with retrieved skills [153]. The changing library alters exploration, and new trajectories alter both the library and the policy. This creates two routes for retaining experience. Comparing the original and trained policies under the same harness assesses training gains, while evaluating a fixed trained policy with and without the updated library assesses the contribution of library access.

**Choosing updates from work failures.** SIA uses execution trajectories and reward dynamics to select between scaffold editing and reinforcement learning of executor weights. Its combined system outperforms harness-only iteration on the three studied tasks, showing that some residual failures remain responsive to parameter updates after scaffold improvement [155]. VERA makes the choice at the level of work stages: rubric evidence identifies execution failures and missing procedures, model rounds train with rubric rewards, and harness rounds add, merge, or remove skills. Skill proposals are tested with paired runs under the same frozen model before acceptance [110]. Its model-only and harness-only ablations underperform the combined loop on the evaluated co-work and medical-research workflows. Its 4B and 9B agents lose performance on several general-capability tests after specialization, while the 27B agents retain broader capabilities more effectively. Target-workflow improvement and capability retention therefore require separate assessment.

**Allocating capability from matched evidence.** Component Routing for Self-Improving GUI Agents provides a direct comparison of where experience should reside. It holds the extracted experience and evaluation tasks constant while routing locators, procedures, state facts, and lessons to context or weights. Across the tested backbones and environments, recurring locators favor weights, whereas procedures and facts

whose usefulness depends on the current state favor retrieved context. Procedure transfer across models also differs by route: useful contextual guidance can become harmful when distilled into a policy with different action preferences [159]. The allocation unit is therefore the reusable component of experience, together with the conditions under which it applies.

These results support a conditional allocation rule. Recurrent interpretation, localization, or action errors across tasks favor parameter learning when training examples cover those decisions. Missing evidence, stale state, and failures tied to a particular procedure favor changes to retrieval, interfaces, or explicit skills. Frequent reuse can amortize training cost; rapidly changing facts and rules benefit from external revision without retraining. Current permissions, artifact versions, and outstanding commitments require external records that can be inspected and updated even when the policy has learned how to use them. The gains from a stronger model and a larger skill library consequently need not be additive: redundant guidance consumes context, and guidance learned under another policy can conflict with current behavior.

For diagnosis, a useful extension of the paired comparisons above is to cross the original and revised models with the original and revised harnesses. With compatible interfaces, testing $(M_0, H_0)$, $(M_1, H_0)$, $(M_0, H_1)$, and $(M_1, H_1)$ on matched tasks, initial states, verifiers, and execution budgets separates model gains, harness gains, and their interaction. Accounting for training, search, inference, and review costs then determines whether the improvement yields more accepted work per unit resource. For continuing responsibilities, the revised pair must also be checked against retained workspace state and still-valid commitments; this is a further requirement beyond performance on newly reset tasks. Section 6 examines the evidence supporting capability claims, while Section 7 develops the adaptation and verification problems that arise when capability updates occur during ongoing work.

# 6 Evaluation and Resources: Assessing Working Agents

Working-agent evaluation connects two questions: what work was delegated, and what evidence establishes that it was completed? An answer, a saved workbook, and an updated business record each require a different account of success. As work spans applications and dependent objects, assessment must also establish whether the results agree, the required conditions were respected, and unfinished responsibilities remain accounted for. The capabilities in Section 4 become measurable through these observable consequences of work.

This perspective connects evaluation to optimization. Section 5 describes how experience improves the model and harness; evaluation determines which improvements produce better work under the intended conditions. We organize this chapter around four parts of that connection. Section 6.1 examines how benchmarks construct tasks and workspaces. Section 6.2 compares the evidence and procedures used to judge completion. Section 6.3 explains how those judgments become measures of quality, reliability, and cost. Section 6.4 examines the environments, harnesses, and evaluation infrastructure that make these measurements reproducible.

## 6.1 Benchmarks and Task Construction

Benchmark design has two connected dimensions: how tasks are constructed and which work domains they represent. Task construction determines the source of a request, the available environment, and how difficulty can be varied. Domain coverage determines the work objects, professional knowledge, and acceptance criteria. Together they connect a benchmark's tasks to the capabilities and outcomes it measures.

### 6.1.1 Task Construction Strategies

Three recurring approaches connect benchmark tasks to work: expert-authored problems, tasks grounded in existing requests and applications, and the reconstruction of professional activities. Table 12 compares representative resources by construction approach, task source, and input–output form. These describe the work made available for evaluation; they do not yet specify which properties are checked.

**Expert-authored and curated problems.** This approach starts from a reasoning or information need and constructs a question with an assessable answer. GAIA combines information gathering, tool use, and reasoning in questions with concise reference answers; FinQA pairs financial documents with expert-written

questions and reasoning programs; LegalBench assembles legal reasoning tasks through collaboration with legal professionals [10, 23, 160]. Such construction makes it possible to vary the knowledge, evidence, or inference required while keeping the expected result explicit. These resources are useful for isolating the informational demands of work: interpreting a financial table, selecting a relevant rule, or combining evidence from several sources.

**Tasks grounded in existing requests and applications.** A second approach preserves the objects and operations through which work is performed. SWE-bench derives repository changes from GitHub issues and associated fixes, while SpreadsheetBench curates spreadsheet requests and files from user forums [33, 64]. WebArena, OSWorld, and AndroidWorld instead construct tasks within browser, desktop, and mobile environments, respectively [3, 4, 57]. The distinction matters for task design: mining a request supplies an authentic work objective, whereas an executable environment supplies the state transitions and feedback needed to pursue it. Combining the two allows evaluation of how an agent discovers information, acts, and responds to the consequences of its actions.

**Professional activities and workspace construction.** A third approach begins with an activity that produces a usable professional result. GDPval uses tasks developed by experienced professionals and evaluates their deliverables against professional work [53]. Workspace-Bench constructs workspaces in which tasks depend on discovering and relating files, while StartupBench derives deliverable-oriented tasks from adopted AI-product workflows, product research, and user interviews [19, 161]. Here task construction must preserve the relationship between the request, supporting materials, and acceptance criteria. A report requires decisions about relevant sources and assumptions; a delivery package can require figures and claims to remain consistent across its constituent files.

**Table 12** Task construction, sources, and input–output forms in representative benchmarks and an associated agent system.

| Benchmark / system | Construction approach | Task source | Input form | Output form |
|---|---|---|---|---|
| GAIA [160] | Expert-designed questions | Cross-domain information and reasoning problems | Text prompt; optional files and external information | Concise text answer |
| DocVQA [9] | Dataset-based annotation | Questions over document images | Document image and question | Text answer |
| FinQA [10] | Expert-designed questions | Financial reports and tables | Question, report text, and tables | Numerical answer and reasoning program |
| LegalBench [23] | Expert-designed and curated tasks | Legal cases, rules, and documents | Task instruction and legal text | Task-specific label or answer |
| SWE-bench [33] | Real-world issue mining | GitHub issues and associated repository fixes | Issue description and codebase | Code patch |
| SWE-agent (system) [8] | Uses existing software tasks | Repository issues, including SWE-bench | Issue and executable repository environment | Code patch |
| WebArena [3] | Realistic web-task design | Functional web applications | Request and browser environment | Answer or target website state |
| VisualWebArena [55] | Visually grounded web-task design | Visual websites and user requests | Request, browser, and visual content | Answer or target website state |
| OSWorld [4] | Realistic computer-task design | Desktop applications and workflows | Request and initialized desktop | Changed environment state or files |
| AndroidWorld [57] | Parameterized task templates | Android applications in an emulator | Request and initialized mobile environment | Changed application state |
| SpreadsheetBench [64] | Real-user artifact-task curation | Spreadsheet requests from user forums | Instruction and workbook test cases | Modified workbooks |

Continued on next page

Table 12 (continued)

| Benchmark / system | Construction approach | Task source | Input form | Output form |
|---|---|---|---|---|
| PPTC [37] | Multi-turn artifact-task design | Presentation creation and editing sessions | Instructions and presentation state | Created or modified slide deck |
| PresentBench [38] | Rubric-based artifact-task design | Presentation requests and background materials | Prompt and task-specific materials | Generated slide deck |
| GDPval [53] | Expert-authored occupational tasks | Professional work across occupations | Request and reference materials | Professional deliverables |
| Workspace-Bench [19] | Role-specific workspace construction | Dependency-driven professional tasks | Request and multi-file workspace | Workspace artifacts |
| StartupBench [161] | Market-validated, expert-built tasks | Adopted AI-product workflows | Request and workspace materials | Final workspace deliverables |

*Notes.* Construction labels summarize how tasks are obtained, not mutually exclusive benchmark categories. SWE-agent is an agent system rather than a separately constructed benchmark; its tasks and acceptance criteria come from the evaluation suite used. Input forms include supplied materials and accessible environments, while output forms identify submitted or changed objects, not a guarantee that every property of those objects is evaluated.

These approaches support different forms of control. Expert-authored questions can isolate particular reasoning demands; executable environments expose action consequences; professional tasks preserve the context that gives a deliverable its purpose. They can also be combined. A task mined from an existing request can be instantiated with new inputs, and a professional workflow can run in a controlled application environment. The useful design choice is which work relationships to preserve and which conditions to vary.

### 6.1.2 Domain Coverage and Work Requirements

Task sources alone do not determine capability coverage. Table 13 relates representative domains and interaction settings to their work materials, objectives, and capability demands. Its capability column describes requirements suggested by the work, not proof that a benchmark independently measures each capability in Section 4.

**Financial and legal analysis.** These tasks connect professional documents to a decision or conclusion. FinQA evaluates numerical reasoning over financial reports, and FinanceBench studies evidence-grounded financial question answering; LegalBench and LawBench cover several forms of legal understanding and reasoning [10, 23, 70, 162]. Financial work requires interpreting quantities, units, and assumptions before computing a result; legal work requires identifying applicable rules and relating them to the facts. Evaluation therefore connects answer correctness to the evidence and domain conditions that justify the answer.

**Software engineering and scientific work.** Here work produces executable changes and reproducible results. SWE-bench pairs repository issues with tests of the resulting code, while DS-1000 evaluates data-science code [33, 163]. ScienceAgentBench and PaperBench extend the task toward scientific implementations and research replication [36, 164]. The environment supplies code, data, dependencies, and execution feedback; assessment connects the submitted implementation to its behavior and, where required, the findings it reproduces.

**Enterprise operations.** Business tasks connect a request to records and actions across application state. WorkArena, CRMArena, AppWorld, and the $\tau$-bench family instantiate different combinations of enterprise interfaces, business data, and user interaction [7, 13, 15, 16, 44]. The work can involve locating the correct entity, applying a policy, updating a record, and completing dependent operations. Its acceptance criteria concern the intended business effect together with the constraints on producing it.

**Office and professional delivery.** OfficeBench, SpreadsheetBench, PPTC, and PresentBench involve native documents, workbooks, slides, and cross-application tasks [37, 38, 42, 64]. GDPval broadens this setting to occupational deliverables judged against professional work [53]. The output must combine correct content with the structure, calculation behavior, and presentation needed for its use. A reporting task, for

example, can require extracting source data, updating a workbook, and communicating the result in a report or presentation.

**Table 13** Representative benchmarks and systems by work domain or interaction setting. Capability demands are tied to work objectives rather than inferred solely from benchmark scores.

| Domain / setting | Representative benchmarks / systems | Workspace / materials | Typical work objective | Main capability focus |
|---|---|---|---|---|
| General knowledge | GAIA; DocVQA [9, 160] | Documents and webpages | Retrieve information and answer questions | Understanding, perception, and reasoning |
| Finance | FinQA; FinanceBench; GDPval [10, 53, 162] | Financial reports, spreadsheets, and business records | Financial analysis and decision support | Numerical reasoning and evidence grounding |
| Law | LegalBench; LawBench [23, 70] | Legal documents, regulations, and cases | Legal analysis and compliance judgment | Domain reasoning and rule application |
| Software engineering | SWE-bench; SWE-agent (system) [8, 33] | Code repositories and tests | Debug and implement changes | Code reasoning, execution, and testing |
| Data science / research | PaperBench [164] | Papers, code, and datasets | Reproduce and analyze research | Scientific reasoning and experimental verification |
| Office knowledge work | OfficeBench; SpreadsheetBench; PPTC; PresentBench [37, 38, 42, 64] | Documents, spreadsheets, and presentations | Create and revise artifacts | Artifact production and cross-application execution |
| Enterprise operations | WorkArena; EnterpriseOps-Gym [7, 165] | Enterprise systems and records | Complete business workflows | State management and policy compliance |
| Customer service | $\tau$-bench; $\tau^2$-bench; CRMArena [15, 16, 44] | Service applications, CRM records, and user requests | Resolve customer tasks | Interaction and constraint following |
| Computer use | OSWorld; AndroidWorld [4, 57] | Desktop and mobile applications | Complete user-interface operations | Grounded perception and interaction |
| Tool ecosystem | API-Bank; ToolLLM / ToolBench; ToolSandbox [2, 12, 166] | APIs, tools, and associated state | Select tools and execute operations | Tool reasoning and state-aware execution |

*Notes.* The rows are non-exclusive: computer use and tool ecosystems are cross-domain interaction settings, and resources such as GDPval span several domains. SWE-agent is a system evaluated on software tasks; ToolLLM is the framework associated with ToolBench. Domain representation does not establish complete coverage of work conditions, capability requirements, or professional acceptance criteria.

**Workspace construction and dependencies.** A work request may arrive with selected files, an application environment, or a broader workspace whose relevant materials must be discovered. Table 14 complements the domain overview by comparing these settings. Workspace-Bench makes file dependencies explicit, distinguishing supporting context, files that supply results, semantic relations, and file lineage [19]. These relations connect task construction to source selection, information integration, and the propagation of changes across work objects.

**Table 14** Workspace construction and dependency-related work in representative benchmarks. Each row connects the supplied setting to the work required and its assessment.

| Benchmark | Supplied setting | Relations involved in the work | Assessment |
|---|---|---|---|
| SWE-bench [33] | Issue description and repository state | Issue requirements, relevant code, and executable behavior | Tests on the patched repository |

| Benchmark | Supplied setting | Relations involved in the work | Assessment |
|---|---|---|---|
| OSWorld [4] | Desktop applications and task-specific initial state | Application interactions and work across files or programs | Task-specific execution-based checks |
| OfficeBench [42] | Office applications, messages, and files | Information transfer and dependent operations across applications | Task completion in the office environment |
| GDPval [53] | Occupational requests and reference materials | Source information, professional requirements, and deliverables | Blinded expert comparison of work products |
| TheAgentCompany [43] | Simulated company with documents, services, and coworkers | Workplace information, collaboration, and intermediate task goals | Checkpoints and final completion |
| StartupBench [161] | Workflow-derived requests and task materials | Related materials and deliverables satisfying professional requirements | Weighted rubric judgments using tool-assisted inspection |
| Workspace-Bench [19] | Role-specific file workspaces and dependency-driven tasks | Supporting context, result sources, semantic relations, and file lineage | Rubric scores and trajectory-derived dependency graph matching |

The construction strategy and domain jointly determine the difficulty and interpretation of a task. Varying source availability changes the demand for discovery; varying application state changes the required actions; varying acceptance criteria changes which properties of a deliverable matter. Evaluation methods then establish whether the agent met those requirements through its answer, its effects on the environment, or the work products it delivered.

## 6.2 Evaluation Methodologies

An evaluation method connects requirements to evidence and then applies a judgment procedure. These are separate choices. The evaluator may inspect an answer, execute a changed repository, query application state, open a deliverable, or examine events across a workflow. It may then use programmatic assertions, expert review, or model judgments. Keeping the observation and judgment choices explicit makes it possible to explain what a reported result establishes about working-agent capabilities.

### 6.2.1 Evaluation Targets and Observable Evidence

Table 16 connects task types and output objects to the protocols and evidence used to judge them. Unlike the input–output inventory in Table 12, this comparison concerns what an evaluator actually observes.

**Answer-level evaluation.** When the requested work closes with an informational response, the final answer is the main evaluation object. GAIA scores concise answers obtained through information gathering and reasoning; DocVQA and FinQA assess answers grounded in documents and financial material; LegalBench evaluates domain-specific judgments [9, 10, 23, 160]. Answer matching, human assessment, or model-based judgment can determine whether the response satisfies the question. Source and reasoning checks further connect the conclusion to its supporting evidence.

**State-based evaluation.** For actions that change an environment, completion is established by inspecting the resulting state or behavior. SWE-bench executes tests against the modified repository; AppWorld and CRMArena assess application outcomes or business records; OSWorld and WorkArena use task-specific checks of work in their application environments [4, 7, 13, 33, 44]. The $\tau$-bench protocol compares the resulting database state with the expected goal state, and EnterpriseOps-Gym uses SQL checks for required outcomes, integrity constraints, and specified side effects [15, 165]. These procedures permit different action sequences to reach a valid result. Their diagnostic resolution depends on the checked properties: a goal-state assertion identifies an achieved update, while preservation checks determine whether other relevant records remain valid.

**Artifact-based evaluation.** Native work products expose several complementary kinds of evidence. A workbook supports inspection of values, formulas, and object structure; a rendered slide exposes layout and readability; source materials support checks of factual fidelity. Existing benchmarks select different parts of this evidence. SpreadsheetBench applies a generated transformation to multiple workbook cases and checks the resulting answers. PPTC extracts PowerPoint attributes and checks their content and positional relationships. PresentBench evaluates generated decks through task-specific binary checklist items, providing decks to its model judge in PDF form [37, 38, 64]. GDPval adds professional review of occupational deliverables [53]. These designs connect the work product to the property being judged: calculation behavior calls for execution or input variation, editability calls for native-object inspection, and presentation quality calls for rendered views.

**Workflow-level evaluation.** Workflow tasks connect completion to dependencies among actions, objects, and intermediate states. EnterpriseOps-Gym and $\tau$-bench assess workflows through business-state outcomes, while TheAgentCompany includes intermediate checkpoints and ToolSandbox evaluates state milestones and specified minefields [12, 15, 43, 165]. Workspace-Bench and StartupBench extend this assessment to work involving related files and deliverables [19, 161]. The protocol can combine final results with intermediate observations: the former establish what was delivered, while the latter establish requirements such as completing prerequisites or obtaining approval before publication.

Consider a monthly revenue report. Answer evaluation checks a requested total; state evaluation checks that the correct report was published; artifact evaluation checks the workbook and narrative; workflow evaluation can additionally check whether the published version received approval. A combined protocol connects the calculations, deliverables, and publication process to the requirements of the reporting task.

### 6.2.2 Judging Procedures

Once the evaluation object is defined, a judge determines whether it meets the task requirements. Working-agent benchmarks use rule-based checks, rubric-based assessment, LLM-as-a-Judge, and Agent-as-a-Judge in complementary ways. Rubrics organize the criteria; programs, humans, and models apply them; an agentic judge can acquire additional evidence before deciding. Table 15 compares how these choices operate on work outcomes.

**Rule-based evaluation.** A rule-based judge executes explicit predicates against the answer, artifact, or environment state. Common implementations include exact-answer matching, repository tests, database assertions, and native-file property checks. SWE-bench tests the submitted patch; EnterpriseOps-Gym queries final database state; PPTC compares extracted PowerPoint attributes and positional relations [33, 37, 165]. SpreadsheetBench checks the results of a transformation across multiple workbook inputs [64]. These methods are repeatable and give precise feedback about the checked property. Their effectiveness depends on expressing the work requirement as an executable condition: a numerical assertion checks a value, while varying inputs tests the behavior of the transformation that produced it.

**Rubric-based assessment.** A rubric decomposes a broad work objective into explicit criteria. A report can be assessed for source fidelity, completeness, analytical soundness, and presentation; a research implementation can be assessed for required components, successful execution, and reproduced findings. PaperBench organizes replication requirements hierarchically, and PresentBench uses task-specific binary checklists for slide decks [38, 164]. Each criterion can be assigned to a suitable judge: a program can check a required file, a model can assess source-supported content, and an expert can assess professional appropriateness. Rubrics thereby connect open-ended requests to interpretable judgments and expose which requirements a delivery satisfies.

**LLM-as-a-Judge.** Here a language or multimodal model receives the request, candidate output, and relevant evaluation criteria or reference material, then returns a score, preference, or criterion-level decision. MT-Bench studies model judging through rating and pairwise comparison [167]. In working-agent evaluation, PresentBench applies a model judge to individual checklist items using the generated deck in PDF form [38]. This approach accommodates semantic and visual properties that are difficult to encode as exact predicates, including whether an explanation addresses the request or a slide communicates the required information. Source fidelity requires access to the relevant sources or reference-derived criteria, while visual judgments

require an appropriate rendered view. The judge's input is therefore part of the evaluation method.

**Agent-as-a-Judge.** An agentic judge turns assessment into an evidence-gathering process. Given a requirement, it can locate files, inspect an execution record, invoke tools, and combine the resulting observations before issuing a judgment. Agent-as-a-Judge develops this approach for evaluating agent-produced work through environment interaction and requirement-level evidence [168]. StartupBench uses separate judge sessions for rubric items, with extracted text and rendered pages supporting navigation and original files available for inspection; Workspace-Bench supplies the judge with input and output files, rubrics, and the execution trajectory [19, 161]. Active inspection is particularly useful for a workspace result whose correctness depends on several objects or on information that cannot be presented conveniently in one judge prompt. Its additional tool calls and execution time also enter the cost of assessment.

**Table 15** Judging procedures and the role of rubrics in working-agent evaluation.

| Approach | How it operates | Typical assessment | Representative resources |
|---|---|---|---|
| Rule-based | Executes tests, assertions, or attribute comparisons | Code behavior, database effects, numerical results, native-object properties | SWE-bench; SpreadsheetBench; PPTC; EnterpriseOps-Gym [33, 37, 64, 165] |
| Rubric-based | Decomposes requirements into criteria evaluated separately | Completeness, fidelity, functional requirements, and professional quality | PaperBench; PresentBench [38, 164] |
| LLM-as-a-Judge | Applies a model to supplied evidence and criteria | Semantic adequacy, visual quality, scores, and preferences | MT-Bench; PresentBench [38, 167] |
| Agent-as-a-Judge | Uses tools to gather and inspect evidence before judging | Multi-file results and requirements needing active investigation | Agent-as-a-Judge; StartupBench [161, 168] |

Professional review provides both direct assessment and a reference for automated judges. GDPval uses blinded comparisons by occupational experts, while PaperBench's JudgeEval compares automated decisions with expert labels [53, 164]. A practical evaluation can combine these procedures: programmatic checks establish executable properties, rubric-guided model judgments assess content and presentation, and tool-assisted or expert review resolves requirements needing further investigation. The resulting criterion decisions and preferences become the inputs to evaluation metrics.

**Table 16** Task types, output objects, and evaluation protocols in representative benchmarks. Each judgment is limited to the evidence and criteria exposed by its protocol.

| Benchmark | Task type | Output object | Evaluation protocol | Main judgment basis |
|---|---|---|---|---|
| GAIA [160] | Knowledge task | Text answer | Answer-level matching | Correctness against normalized reference answers |
| DocVQA [9] | Document QA | Text answer | Answer-level similarity | Average normalized Levenshtein similarity (ANLS) to reference answers |
| LegalBench [23] | Legal reasoning | Label or answer | Task-specific reference scoring | Agreement with expert-designed task labels or references |
| FinQA [10] | Financial reasoning | Program and numerical answer | Answer and program evaluation | Executed numerical result and program equivalence |
| API-Bank [2] | Tool use | Tool calls and responses | Call-level execution and response evaluation | API selection, arguments, execution results, and response quality |

Continued on next page

Table 16 (continued)

| Benchmark | Task type | Output object | Evaluation protocol | Main judgment basis |
|---|---|---|---|---|
| ToolLLM / ToolBench [166] | Tool use | Tool-use trajectory and answer | ToolEval model-based assessment | Task pass judgment and pairwise solution preference |
| WebArena [3] | Web interaction | Answer or website state | Answer and state-based checks | Reference answer, target page, or required website content |
| VisualWebArena [55] | Visual web task | Answer or website state | Answer and state-based checks | Required response or website outcome, including visual targets |
| OSWorld [4] | Computer use | Environment state and files | Task-specific execution checks | Required desktop, application, or file properties |
| AndroidWorld [57] | Mobile interaction | Application state | State-based checks | Task-specific changes in app databases or system state |
| SWE-bench [33] | Software engineering | Patched repository | Test execution | Required fixes pass and designated regression tests remain passing |
| SpreadsheetBench [64] | Spreadsheet task | Workbook | Multi-case artifact checks | Target-cell results across workbook test cases |
| PPTC [37] | Presentation task | Slide deck | PPTX-Match artifact comparison | Extracted slide attributes and positional relations |
| PresentBench [38] | Presentation generation | Slide deck | Model-judged binary checklist | Content and visual requirements inspected through rendered PDF |
| GDPval [53] | Professional work | Deliverables | Blinded expert comparison | Occupational-expert preference between matched work products |
| Workspace-Bench [19] | Workspace task | Workspace artifacts | Agent-as-a-Judge and trace analysis | Rubric satisfaction and recovered file-dependency relations |
| $\tau$-bench [15] | Service interaction | Application state | Goal-state comparison after dialogue | Required database changes against reference state |
| CRMArena [44] | CRM workflow | Business records or answer | Task-specific state and answer checks | Required business result or reference answer |
| StartupBench [161] | Professional workflow | Workspace deliverables | Item-wise, tool-assisted judging | Weighted rubric requirements and inspected deliverable evidence |

*Notes.* The table summarizes representative published protocols, not every later variant. Expert-designed tasks do not imply expert grading of each run: LegalBench primarily uses task-specific reference scoring. ToolLLM names the framework associated with ToolBench. Final-state agreement alone does not establish dialogue-wide policy compliance, and selected artifact checks do not establish every aspect of usability, provenance, or structural correctness. GDPval refers here to its original occupational-expert comparison protocol.

## 6.3 Outcome-Aligned Metrics

Judging produces decisions about properties of work; metrics specify how those decisions are summarized. A check-level score measures satisfied criteria, a task-level score measures completed work, and a repeated-run measure describes reliability under a defined protocol. Workspace-Bench illustrates this distinction through rubric pass rates, threshold-based task completion rates, and node and edge F1 scores for dependency graphs extracted from execution traces [19]. These measure requirement satisfaction, completion under a chosen threshold, and recovered file relations, respectively. Together with resource and human-effort measurements, they describe both the work achieved and what it took to achieve it. Table 18 summarizes the corresponding units of measurement.

### 6.3.1 Task Success and Outcome Quality

Task success expresses whether the acceptance conditions were met. In executable environments, it is often defined through a conjunction of checks. EnterpriseOps-Gym, for example, counts a task as successful when all outcome verifications pass [165]. This makes preservation and side-effect conditions part of completion. Partial-credit metrics instead retain information about progress: PaperBench aggregates weighted rubric judgments through a hierarchy, while StartupBench combines item-level decisions using importance weights [161, 164]. A high partial score and an accepted delivery describe different achievements.

The appropriate aggregation follows the task's acceptance logic. A workbook with a central calculation error can satisfy many presentation requirements while failing its intended use. When a requirement is essential to acceptance, it should act as a gate; graded scores can then describe quality among acceptable results or diagnose unsuccessful attempts. This preserves the distinction between completing the work and making progress on it. It also prevents tasks with numerous minor criteria from appearing stronger solely because those criteria dominate an average.

Outcome quality is best represented through properties relevant to the work object. For documents and presentations, these can include source fidelity, completeness, readability, and professional appropriateness. For workbooks and code, they can include calculation behavior, executable correctness, and the ability to reuse the result. For a collection of outputs, agreement among related values and claims becomes an additional criterion. GDPval's professional comparisons and PresentBench's dimension-specific grading illustrate complementary ways to assess this broader quality [38, 53]. The reported score should retain the criterion or dimension that gives it meaning.

Table 17 makes these distinctions concrete by pairing work objects with reported metrics and the judges that produce their underlying decisions. A judge type is not itself a metric: an automated test can support task success, a model can produce criterion decisions, and an expert comparison can support a preference rate.

**Table 17** Work objects, core evaluation metrics, and judge types in representative benchmarks and an associated agent system. Metrics summarize only the requirements covered by the stated checks or rubrics.

| Benchmark / system | Work object | Core evaluation metrics | Judge type |
|---|---|---|---|
| SWE-bench [33] | Code repository | Issue-resolution rate under required fix and regression tests | Automated test execution |
| SWE-agent (system) [8] | Software repository | Issue-resolution rate on the evaluated suite; inherits its acceptance tests | Benchmark-specific execution evaluator |
| SpreadsheetBench [64] | Workbook | Execution accuracy across workbook test cases, based on required cell results | Programmatic workbook checker |
| FinQA [10] | Financial answer and program | Execution accuracy and program accuracy | Rule-based program execution and equivalence checking |
| LegalBench [23] | Legal label or answer | Task-specific accuracy or balanced accuracy against references | Programmatic scoring of expert-designed tasks |
| PPTC [37] | Presentation | Instruction- and session-level accuracy under slide-attribute and relation matching | PPTX-Match rule-based evaluator |
| PresentBench [38] | Presentation | Binary-checklist satisfaction, with content and visual dimension breakdowns | Multimodal model judge; human validation of judging |
| GDPval [53] | Professional deliverables | Pairwise preference, win and tie rates against professional reference work | Blinded occupational-expert comparison |
| Workspace-Bench [19] | Workspace artifacts | Rubric pass rate, thresholded task completion, and dependency-graph node/edge F1 | Agent-as-a-Judge and trace-derived graph matching |
| CRMArena [44] | CRM records and answers | Task success against specified business outcomes or reference answers | Task-specific state and answer checker |

Continued on next page

Table 17 (continued)

| Benchmark / system | Work object | Core evaluation metrics | Judge type |
|---|---|---|---|
| $\tau$-bench [15] | Service workflow | Goal-state success and pass$^k$ across repeated attempts | Rule-based database-state comparison |
| AppWorld [13] | Application state | Task goal completion, including preservation of unrelated state | Programmatic goal and collateral-state checks |

*Notes.* SWE-agent inherits evaluation metrics from the benchmark used rather than defining an independent measurement standard. Result matching does not certify all formulas or workbook structure; passing selected tests does not establish general regression freedom. Terminal-state checks establish policy compliance only where policy requirements are encoded in those checks. Human validation of a model judge, as in PresentBench, is distinct from human grading of every agent output. GDPval's original expert protocol is distinguished from automated grading variants.

These benchmark-specific measures instantiate the metric families in Table 18. Keeping the work object, aggregation unit, and judge explicit shows why superficially similar success rates need not support the same capability claim.

**Table 18** Metric families, aggregation units, and interpretation

| Metric family | Unit and aggregation | What it estimates | Required qualification |
|---|---|---|---|
| Single-attempt success / pass@1 | Binary task outcome averaged over attempts or tasks | Completion under one stated configuration | State the task distribution, budget, assistance, and success predicate |
| pass@$k$ | Whether at least one of $k$ attempts succeeds, under the stated estimator | Success with a multi-attempt opportunity | Not ordinary one-attempt performance; candidate selection may require an oracle |
| pass$^k$ | Whether all $k$ attempts succeed, under the stated estimator | Repeat reliability; used in the $\tau$-bench family [15, 16] | Distinguish actual grouped trials from an independence-based estimate |
| Cell / object / criterion agreement | Matched units divided by tested units | Local correctness over the inspected units | A high average may conceal one task-invalidating defect [37, 64] |
| Weighted rubric score | Weighted aggregation of requirement judgments | Quality under an explicit rubric | Report weights and critical gates; easy criteria cannot compensate for forbidden effects [164] |
| Pairwise preference / win rate | Preferred outputs over comparable judgments | Relative quality against a stated comparator | Tie handling, judge blinding, and evidence access matter [53, 167] |
| Judge agreement | Agreement with an independent reference assessment | Reliability of the evaluator | Not the acting agent's success rate [168] |
| Cost, latency, and human effort | Resources per attempted or accepted task | Efficiency under a quality constraint | Expert preference alone is not measured economic return; report the denominator |

Aggregation across tasks introduces another choice. A micro-average pools checks, giving greater weight to tasks with more criteria; a macro-average first scores each task and then combines task scores. Reporting task-level distributions and domain breakdowns helps reveal whether an overall gain comes from broadly improved delivery or a narrow subset of work. Human preference scores likewise depend on the compared submissions and evaluation criteria. They describe relative quality within that comparison.

## 6.4 Open-Source Frameworks and Execution Environments

Assessment depends on the software that executes work and records its effects. An agent platform supplies the execution tools and runtime, while an evaluation harness initializes tasks, collects evidence, and scores outcomes. OpenHands integrates code, terminal, browser, and sandbox into an execution surface [81]; BrowserGym unifies observation, action, and evaluation interfaces across web environments [169]. End-to-end performance therefore reflects the model, its execution configuration, and the evaluation procedure together.

**Representative systems.**

Deployed products show how these mechanisms combine in working systems. Table 19 summarizes the documented configurations of Codex, Claude Code, and Microsoft 365 Copilot, using the official documentation accessed on 20 September 2026 [170–172].

**Table 19** Documented product configurations as working-agent systems

| Product / configuration | Documented mechanisms | Workspace and outcome | Implication for this survey |
|---|---|---|---|
| Codex | Repository guidance through AGENTS.md; skills; MCP; sandbox and approval settings; testing and review workflows | Repositories, files, commands, connected external tools; patches and checked results | Separates learned policy from configurable context, procedures, execution controls, and review |
| Claude Code | Project instructions; file edits and commands; MCP; reusable skills; lifecycle hooks | Codebases and connected tools; modified files and execution results | Makes procedure reuse and pre/post-action automation explicit |
| Microsoft 365 Copilot | Application-integrated grounding through Microsoft Graph and the signed-in user's access permissions | Documents, email, chats, and application context | Shows that organizational context and permission-scoped retrieval are central to workspace access |

These configurations connect model decisions to different workspaces. Repository agents expose file operations, commands, and reusable procedures, while Microsoft 365 Copilot grounds responses in organizational content under the user's access permissions. Connectors supply access; maintaining consistency and enforcing transaction or policy constraints require corresponding state and control mechanisms.

Codex illustrates how the same model can receive different repository guidance, tools, skills, permissions, and acceptance instructions. Its behavior is a property of this configured system. Testing and review supply feedback during execution; a held-out grader supplies external evaluation. Scheduled execution triggers repeated work, while adaptation and recovery determine how the system handles change and interruption. Extending the configuration to office work also requires artifact-specific operations and validators for spreadsheets, slides, and documents.

**Execution configurations.**

Table 20 organizes 43 benchmark configurations by domain, evaluation harness or runtime, and scoring procedure. The entries span interactive environments, execution runners, and answer or artifact graders. Together, they show how the work setting and the evidence available to the evaluator shape reported performance. A comparison therefore needs to identify both the actor's execution configuration and the evaluator used to assess its outputs.

**Table 20** Benchmark × evaluation harness, runtime, and scoring

| Domain | Benchmark | Evaluation harness / runtime | Scoring system / description |
|---|---|---|---|
| Finance | SpreadsheetBench | Execution verifier (openpyxl / LibreOffice recalculation) | Cell match; harness-independent (source label) |
| Finance | BankerToolBench | Dedicated tool environment | Rubric verifier |
| Finance | Finance Agent v2 | Vals evaluation framework (agent, tools, web) | Gold comparison |
| Finance/Law | PRBench | Response evaluation | Expert rubric; harness-independent (source label) |
| Law | LegalBench | Evaluation scripts | Label comparison; harness-independent (source label) |

| Domain | Benchmark | Evaluation harness / runtime | Scoring system / description |
|---|---|---|---|
| Law | Harvey LAB | Sandbox documents to deliverables; AA version uses AA harness | Single-LLM rubric judge, criterion by criterion |
| Law | Legal Research (vals) | Vals evaluation framework | Judging / comparison |
| Healthcare | MedCode | Vals evaluation framework | Comparison with expert codes |
| Healthcare | MedScribe | Vals evaluation framework | 100 rubric criteria |
| Healthcare | PSEBench | Dedicated evaluation | Policy judgment; harness-independent (source label) |
| Healthcare | MLCR-AA | AA long-context evaluation | Task criteria and faithfulness |
| Office | OfficeQA | Dedicated evaluation | Gold comparison |
| Office | PresentBench | Dedicated evaluation and checklists | LLM / human criterion checks |
| Office | PPTC | PPTX-Match system | System matching |
| Office | OmegaUse-OfficeVal | OfficeVal harness (GUI, scripts, or API) | Office artifact inspection |
| Office | OfficeBench | Multi-application OfficeBench environment | State validation |
| Office | MMLongBench-Doc | Dedicated evaluation | QA comparison |
| Office | GDP.pdf | AA evaluation | Task-specific criteria |
| Office | PPTEval/PPTAgent | PPTEval system | Multidimensional LLM judge |
| General | GDPval | Original: shell and web; AA v2: Stirrup harness | Blind pairwise / Elo; economic-value label in compilation |
| General | OmniaBench | Huawei verifiable environment | Environment state |
| General | Workspace-Bench | ByteDance agentic judge and file-dependency graph | Agentic verifier |
| General | JOBBench | World-state environment | Chained rubric |
| General | AutomationBench | Zapier SaaS sandbox (AA-AA version) | Goal completion without violations |
| General | APEX-Agents | Mercor multi-application world; AA version uses AA harness | Rubric verifier; pass@1 |
| General | $\tau^2$-bench | Sierra $\tau^2$ environment: user simulator, tool APIs, database | Terminal database and policy; pass$^k$ |
| General | EnterpriseOps-Gym | ServiceNow gym servers and MCP | Terminal database state |
| General | OSWorld | OSWorld VM / GUI environment | Execution-based judgment |
| General | WorkArena | BrowserGym and ServiceNow instance | Application state |
| General | TheAgentCompany | Self-hosted GitLab / Plane / RocketChat; OpenHands-like actor | Checkpoints |
| General | AppWorld | AppWorld engine (9 apps / 457 APIs) | Final state and collateral effects |
| General | MCP-Atlas | Real MCP servers (36/220, as supplied) | Tool execution and response |
| General | Remote Labor Index | End-to-end paid remote work; custom agents allowed | Expert / employer acceptance; automation rate |
| General | WildClawBench | CLI harnesses: OpenClaw / Claude Code / Codex / Hermes | Rules, state, and LLM/VLM judges |
| General | $OneMillion-Bench | Dedicated evaluation with tools / retrieval | Four-dimensional rubric |
| General | AgentIF-OneDay | Dedicated evaluation | Instance rubric and LLM validation (Gemini-3-Pro) |
| General | CentaurBench | Dedicated automate / augment comparison framework | Two evaluation modes |
| Business | CRMArena | Salesforce sandbox | Environment state and response |
| Business | CRMArena-Pro | Multi-turn Salesforce sandbox | State, dialogue, and confidentiality |
| Business | EnterpriseBench | Enterprise sandbox with fragmented data and permissions | State and answers |
| Business | CI-Work | Enterprise information-flow environment | Completion and leakage checks |

| Domain | Benchmark | Evaluation harness / runtime | Scoring system / description |
|---|---|---|---|
| Field | FieldWorkArena | Dedicated evaluation using field images / video | Grounded judgment |
| E-commerce | E-Commerce Bench | E-commerce system environment | Final state |

The "harness-independent" label in the table describes scoring an answer or artifact through a separate grader. The actor's context, tools, memory, and control policy still shape what is submitted for scoring. Comparisons should therefore report the model and runtime versions, available tools and permissions, task initialization, budgets, reset behavior, and evaluator. Reset behavior specifies which context, memory, files, and application state carry into subsequent runs.

This infrastructure completes the connection from benchmark construction to measured work outcomes. Tasks define the intended work, judging procedures inspect its results, metrics summarize performance, and the execution configuration establishes the conditions under which that performance was obtained. Section 7 develops how these systems can extend from completing assigned tasks to carrying responsibility as workspaces and requirements change.

## 7 Frontiers: Scaling Working Agents Beyond Bounded Tasks

The preceding chapters examine how working agents acquire capabilities and what evidence establishes their performance. A central frontier is to extend capable execution from software-oriented workflows to people across occupations and disciplines, including users who do not program. This requires competence in their subject matter, conventions, and deliverables, together with ways to delegate work without designing its technical execution. Broad access depends on both the range of work an agent can perform and the effort its users must supply.

The next research question is how the constraints on useful work change as those capabilities improve. An agent that succeeds in a clean workspace can still struggle with old artifacts in an unfamiliar occupation. A capable executor can miss a need that no new request announces. A highly rated presentation can still serve its actual audience poorly. These situations expose limits in the experience available for learning, the continuity of assistance, and the connection between assessed quality and useful outcomes. Further progress depends on identifying which constraint limits a particular work process and how improving it changes the work that can be delivered.

We develop four research positions around this problem. **Environment** concerns the training and evaluation infrastructure needed for generalization across domains, occupations, and workflows. **Persistence** concerns anticipating needs and fulfilling delegated responsibilities through events, interruptions, and capability updates. **Verification** concerns judging open-ended work, generalizing those judgments, and connecting quality to value in use. **Work-Centric Systems** concerns the system structure and division of labor through which these capabilities produce value. The persistence and openness dimensions in Section 2 determine how strongly each constraint is encountered; their relative importance can change within the same work process. Table 21 summarizes the resulting agenda.

**Table 21** Shifting bottlenecks and research directions for working agents. Each direction connects a limitation of current work execution to a prospective mechanism and evidence of progress.

| Frontier | Emerging bottleneck | Research direction and evidence |
|---|---|---|
| **7.1 Environment**: support generalization in realistic work | | |
| Coverage and transfer (§7.1.1) | Available experience covers a narrow range of professional work. | Build from domains through occupations to workflows; test transfer at each level. |
| Realism and noise (§7.1.2) | Clean workspaces omit obsolete versions, irrelevant material, and conflicting state. | Preserve realistic history and ambiguity; test source selection and consequence tracking. |
| Learning and evaluation (§7.1.3) | Easy or repetitive tasks reveal little about transferable capability. | Build curricula and independent, discriminative tests for SFT, RL, and deployment. |
| **7.2 Persistence**: fulfill responsibilities within agreed bounds | | |
| Proactive work (§7.2.1) | Useful work can require action before a new request arrives. | Infer needs from history and current events; assess useful anticipation and intervention burden. |
| Delayed effects (§7.2.2) | Local recovery can repeat effects or leave dependent work inconsistent. | Reconcile consequences before resumption; measure duplicate effects and downstream repair. |
| Adaptation and commitments (§7.2.3) | Capability updates can change decisions governing unfinished work. | Condition migration on current commitments; test improvement and continuity together. |
| **7.3 Verification**: connect contextual quality to value in use | | |
| Subjective quality (§7.3.1) | Several valid outputs can differ in usefulness and professional quality. | Combine grounded checks with contextual pairwise judgments and expert comparisons. |
| Generalizable criteria (§7.3.2) | Criteria and judge preferences transfer unevenly across work settings. | Learn shared principles with contextual requirements; test held-out occupations and audiences. |
| Downstream value (§7.3.3) | A preferred artifact's effect on decisions or returns is uncertain. | Connect judgments to controlled use outcomes; estimate incremental value and total cost. |
| **7.4 Work-Centric Systems**: organize capability around useful delivery | | |
| Work as a system object (§7.4.1) | Dynamic decomposition can lose requirements and responsibility. | Preserve work identity through structural changes; test reassignment and state accuracy. |
| Human–agent organization (§7.4.2) | Continuing goals and temporary execution require different forms of coordination. | Study persistent coordination with replaceable specialists; assess handoffs and human burden. |
| Net productivity (§7.4.3) | Gains in execution expose limits in review, integration, or attention. | Locate the limiting stage through controlled interventions and longitudinal work studies. |

## 7.1 Environment: Infrastructure for Generalization in Realistic Work

Generalization depends on which parts of human work an agent encounters during learning and evaluation. Future environment infrastructure needs broad occupational coverage, realistic workspaces, and tasks difficult enough to distinguish capabilities. These requirements connect the training environments in Section 5.1 to the benchmarks in Section 6.1: supervised fine-tuning needs informative demonstrations, reinforcement learning needs executable interaction and feedback, and evaluation needs unfamiliar work that reveals what transfers. The central question is which coverage and realism produce competence beyond the environments used to acquire it.

### 7.1.1 Coverage across Domains, Occupations, and Workflows

Environment construction can proceed from domain context, through occupational responsibilities, to executable workflows. Domain coverage introduces different knowledge and business rules; occupational coverage introduces different duties and standards of acceptable work; workflow coverage introduces different sequences, dependencies, and uses of tools. A finance analyst, a teacher, and a student preparing a disciplinary project can all need document production while requiring different knowledge and judgments. Supporting these users requires learning the work behind the shared file format.

Occupational resources provide a concrete basis for planning that coverage. O*NET describes occupations through tasks, work activities, knowledge, skills, and work context [173]. GDPval uses O*NET task and activity information in constructing professional work across 44 occupations in nine sectors [53]; commissioned tasks in the Remote Labor Index provide another source of work requirements [174]. Future environment builders could connect these descriptions to executable workflows, reference materials, and acceptance conditions. Disciplinary coursework and educational projects would supply complementary coverage of how students use similar tools for different learning goals.

Current synthesis approaches supply complementary ingredients. Repository environments recover consequences from code and tests [175–177]; EnvScaler and Agent World Model generate executable tool interactions; AgentMercury connects shared entities across services [97, 99, 112]. GraphForge organizes workspace synthesis through evidence relations among files [95]. These mechanisms could support a common construction process that instantiates occupational work in persistent workspaces, with expert examples grounding the responsibilities and synthetic variation expanding the situations in which they arise.

Coverage becomes useful when it exposes transferable structure. Financial reporting and procurement analysis can both require reconciling records, identifying an authoritative version, and propagating a revision, while assigning different meanings to the underlying objects. Generators should vary both these shared operations and the domain-specific conditions that govern them. Transfer can then be tested by withholding domains, occupations, or workflow structures separately and in combination. This would distinguish reuse of an operation in a new setting from adaptation to an unfamiliar responsibility and show where additional training coverage has the greatest value.

### 7.1.2 Realistic Workspaces and Decision-Relevant Noise

Real workspaces contain traces of earlier work: obsolete presentations, competing file versions, superseded instructions, partially updated records, and material unrelated to the current request. This noise is often meaningful evidence about history and authority. An old spreadsheet can be useful for explaining a change while being unsuitable as the source for the current report. Workspace-Bench's file dependencies illustrate the importance of organizing evidence across work objects [19]. Future environments should preserve the history that makes selecting and interpreting those objects difficult.

A useful construction strategy is to generate workspaces through successive episodes of human and agent activity. Earlier tasks would leave artifacts, revisions would create competing versions, and unrelated workflows would add background material. Each current task would require locating the relevant state within that accumulated workspace. Difficulty could be varied through source ambiguity, version overlap, dependency depth, and the amount of plausible but irrelevant information. The generator would retain enough ground

truth to determine which sources and effects should govern the present task. This ties noise to an interpretable work problem.

Realism also concerns execution. A colleague can change a record during a task, a permission can expire, or a tool can return late after committing an operation. Shared-state and asynchronous environments offer starting points [16, 178, 179]; simulation studies show that even the behavior of a simulated user can change measured outcomes [180, 181]. Calibration should target discrepancies that alter source selection, action, or recovery. Comparing agents in clean and historically accumulated workspaces would reveal how much apparent capability depends on having already organized the task for them.

### 7.1.3 Training Curricula and Discriminative Evaluation

The same environment infrastructure can support different learning and evaluation needs. For SFT, demonstrations should show how competent execution selects current evidence and handles conflicting material. For RL, resettable state and varied interactions should expose the consequences of decisions, with feedback linked to work outcomes. For evaluation, independently constructed workspaces and held-out workflow families should test the resulting generalization. Shared software can support these uses while keeping the evaluation instances, generation templates, and acceptance references separate from training.

The challenge is to make difficulty informative. Extending a task or adding files can increase cost without revealing a new capability boundary. More discriminative environments vary a consequential requirement: an outdated source that must be rejected, a dependency that changes the valid plan, or a missing fact that requires a query. Skill2Env's feedback-driven revision provides a starting point for directing synthesis toward observed failures [100]. Expert completion and graded variations of the same work can establish whether a challenge is realistic and where agents cease to handle it reliably.

Research should compare broader coverage, richer workflow dependencies, and more realistic noise under matched training budgets, then examine performance on held-out domains, occupations, and workflows. Changes in interfaces and business meaning should also be varied separately, building on work on evolving tools and schemas [182–184]. These comparisons would guide investment in environment infrastructure by the generalization it enables. As environments retain history and allow new events to arrive during unfinished work, they also create the conditions needed to study persistent and proactive assistance.

## 7.2 Persistence: Sustaining Delegated Responsibilities

Persistent work requires an agent to know what it still owes, anticipate what may be needed next, and choose when to act. A maintenance agreement can remain open after a report is delivered, while a recurring pattern in the user's history can make a new query or preparatory task useful before another request arrives. The relevant state includes current goals, deadlines, permissions, closure conditions, and effects awaiting confirmation. Context management and progress records support continuity [83, 185, 186]; the research objective is to turn that continuity into useful assistance as circumstances and capabilities change.

### 7.2.1 Proactive and Event-Driven Work

Proactivity introduces a decision before task execution: which work is worth initiating now? User history can reveal recurring reporting cycles, unresolved questions, preferred ways of working, and dependencies between requests. Combined with current events, it can guide queries for missing information or preparation for an approaching need. An agent supporting a monthly review might notice that new cost data have arrived, query the relevant records, and prepare an updated comparison within its existing delegation. Proactive-agent research begins to connect such anticipation with the timing and depth of intervention [187, 188].

Future systems need to connect inferred needs to the state of continuing responsibilities. A new event may advance several obligations or invalidate a stored plan; a historical preference may need confirmation after the user's goals change. Some relevant events are never announced. VibeLifeBench exposes this challenge through simulated multi-week tasks whose world requires renewed inspection [189]. An agent therefore needs policies for retrieving history, inspecting current state, querying the user when intent is unclear, and choosing whether to initiate, resume, or defer work.

Scheduling these choices requires an account of both usefulness and intervention cost. Asynchronous execution and temporal task environments supply mechanisms for managing waits and concurrent demands [190–193]. Each responsibility could retain conditions for starting, waiting, escalating, renewing, and closing, while a learned policy allocates attention across them. Evaluation should include useful work initiated before an explicit request, needs that were missed, unnecessary queries or interruptions, and obligations fulfilled on time. Histories and event streams with changed user goals would test whether proactivity adapts rather than repeatedly acting on an obsolete pattern. The resulting system would reduce the user's burden of remembering and coordinating work while retaining an accurate account of its consequences.

### 7.2.2 Reliability under Delayed and Irreversible Effects

Continuing responsibilities make recovery a problem of reconciling consequences. A local checkpoint can restore an agent's context after an interruption while an external transaction has already committed [194–196]. Retrying can duplicate the effect; proceeding from an assumed failure can leave later work inconsistent. The distinction among attempted action, accepted call, committed effect, and verified effect in Section 4 becomes essential to deciding what should happen next.

Provenance, idempotent operations, and compensating transactions provide complementary support [197–200]. Their use depends on knowing which consequence is unresolved. An uncertain payment calls for a state query; an incorrect but confirmed payment may require compensation; a report derived from the payment may need revision even after the transaction is repaired. The appropriate unit of recovery is therefore the affected set of work obligations and dependencies. Failure-attribution methods supply evidence about where an error arose [201, 202]; continuing work additionally requires identifying what now needs to be repaired.

A promising training direction is to vary feedback delay, partial completion, concurrent edits, and irreversibility while preserving a record of the true effects. Policies could learn which observation resolves uncertainty and which intervention preserves useful progress. Comparing systems resumed from the same interrupted state would expose duplicate effects, omitted obligations, and repair propagated to the wrong objects. This would connect recovery competence to the continuity of actual work, including cases in which a consequence must be acknowledged and carried forward because it cannot be undone.

### 7.2.3 Continual Adaptation without Losing Work Commitments

Learning during ongoing work introduces two distinct objectives: improving future decisions and preserving commitments that still govern unfinished work. Memory and skill methods retain useful experience, while model and harness updates change the procedures used to act on it [78, 92, 144, 203, 204]. An audit of self-evolution in simulated e-banking reports capability gains alongside increased unauthorized state changes in some configurations [205]. This motivates examining the effects of an update on the particular work already underway.

A reusable lesson and an agreed requirement have different update rules. A reporting procedure can be improved through experience; an agreed statistical definition remains applicable until an authorized revision changes it. Conventional backward-transfer measures concern retained competence [206]. Commitment compatibility concerns whether a revised policy still honors decisions, permissions, and pending effects belonging to a specific responsibility. Temporal memory and versioned records provide useful ingredients [207, 208], but the agent must learn to distinguish evidence about a better procedure from evidence that the requirement itself has changed.

One direction is to associate open work with the decisions and procedure versions under which it was last authorized. A revised skill could be introduced selectively; a broader model update could first serve new work while ongoing tasks are checked for compatibility. The model–harness comparisons in Section 5.4 can then be extended to retained workspace state: resume the same partially completed work under the original and updated configurations, including outstanding approvals and uncertain effects. Capability gains and commitment violations would become separately observable.

Changing circumstances can also justify revising the commitment. A withdrawn authorization or an obsolete business definition requires coordinated changes to the affected goals, results, and evidence. The useful

learning target is the ability to identify what remains valid, propose what must change, and preserve the grounds for that distinction. An agent could then improve its methods throughout a service interval while maintaining an accurate account of what the user is still entitled to expect. Fulfilling those expectations also requires judgments of quality that remain appropriate as the work and its intended use evolve.

## 7.3 Verification: From Subjective Quality to Value in Use

Many working-agent outputs admit several reasonable solutions whose quality depends on their intended use. Two slide decks can contain the same correct facts yet differ in clarity, emphasis, visual organization, and usefulness to a particular audience. A professional preference between them can be measured, but relating that preference to a better decision or a financial return requires further evidence. The frontier is to develop judgments that generalize across work settings and remain connected to the outcomes that make the work valuable. This extends the evaluation procedures in Section 6.2 into three questions: how to compare subjective quality, how to transfer the criteria, and how to establish their relationship to value in use.

### 7.3.1 Subjective Quality and Comparative Judgment

Work quality combines properties with different grounds for assessment. Source fidelity and numerical consistency can often be checked directly. Narrative emphasis, professional appropriateness, and persuasive organization require judgment in context. PresentBench supplies instance-specific checklists for slide generation, while GDPval uses expert pairwise comparisons of occupational deliverables [38, 53]. These approaches provide complementary evidence about whether requirements are satisfied and which acceptable output better serves the task.

Pairwise judging is particularly useful when an absolute score has no clear interpretation. Given the same brief and materials, a judge can compare which of two presentations better supports a stated audience and purpose. Future protocols should make that context explicit and combine the comparison with grounded checks of the claims being presented. The research problem is to learn a preference that responds to substantive usefulness while remaining robust to output order, verbosity, stylistic familiarity, and the judge's own model preferences [167, 209, 210]. Blinded comparisons, stable reference outputs, and dimension-specific judgments can make the basis of a ranking more interpretable.

Professional disagreement also contains information. A technical reviewer and a sales manager may prefer different presentations because they use them differently. Preserving the audience, decision context, and reasons for a preference would allow a system to distinguish legitimate differences in purpose from inconsistent judging. Experiments could compare holistic pairwise preferences, explicit rubrics, and their combination against assessments from the intended users. Progress would mean predicting those assessments on unfamiliar work while identifying which criteria account for disagreements. This creates a learning problem for contextual quality judgments, extending verification beyond a single fixed answer or universal style preference.

### 7.3.2 Generalizable Criteria for Evolving Work

A useful evaluation standard must transfer without assuming that every task values the same properties equally. Factual support, internal coherence, and satisfaction of the requested scope recur across many work products. Their expression and relative importance depend on the occupation, audience, and workflow: an executive presentation may prioritize a decision and its consequences, while a training presentation needs an explanation that supports learning. The opportunity is to learn shared quality principles together with a reliable way to derive the requirements of the current work.

Rubric construction can combine the request, source materials, professional conventions, and examples of accepted work. Evidence-based criteria in GraphForge and workspace inspection in Agent-as-a-Judge provide ingredients for grounding those requirements [95, 168]. Future research should test how much expert input is needed to adapt a judge or rubric to a new domain, occupation, or artifact type. Held-out work settings would reveal whether the system has learned transferable evaluation principles or mainly reproduced the conventions of its training examples.

Generalization requires both consistency and appropriate sensitivity. A cosmetic change that preserves the relevant content should leave a factual judgment intact; a change in the intended audience may justifiably change which presentation is preferred. Paired tests can vary these factors separately, with experts establishing which judgments should remain stable and which should change. This would distinguish a robust standard from a judge that merely produces the same ranking everywhere. The same principle applies over time: criteria and supporting evidence should remain tied to the current request, source versions, and authorized requirements. A revised business definition can invalidate a previously acceptable conclusion even when its file is unchanged, motivating validity tracking of the kind explored by StateGuard [211].

Such a standard could support both evaluation and the reward signals of Section 5.1.3. Its value would depend on preserving professional judgment across unfamiliar settings and changing requirements. Reusable criteria and automated checks could lower the workload of expert review, while independent judgments on new work would establish whether the resulting efficiency preserves the quality of the assessment.

### 7.3.3 Connecting Deliverable Quality to Downstream Value

The further question is whether a better-rated work product produces a better outcome when used. A slide deck may improve audience comprehension, support a more accurate decision, or contribute to a sale. These effects connect artifact quality to value through different mechanisms. A promising research direction is to identify which judged properties predict which use outcomes, for which audiences and work conditions. The target is an empirical relationship between evaluation and usefulness that can be tested beyond the examples used to define the rubric.

Controlled studies could begin with outputs from different model–harness configurations given comparable briefs and source materials. Expert rankings would be collected alongside audience comprehension, decision quality, correction effort, or task completion after using the artifact. Where deployment permits, randomized assignment of deliverables to comparable use settings could estimate their incremental effect. For a sales presentation, the offer, presenter, audience, and timing also influence revenue; a study must account for those factors to attribute an observed difference to the generated work. Longer-term follow-up could then examine whether immediate gains persist through actual adoption and use.

Comparing the return on investment of outputs from model A and model B requires this link to a counterfactual: what changes when otherwise comparable work uses one configuration rather than the other? Incremental benefits must be considered alongside generation, review, revision, integration, and maintenance costs. A stronger model could produce a preferred presentation yet require more adaptation before use; a modest quality improvement could have substantial value when the artifact supports a consequential recurring decision. Studies across occupations would show where a preference score predicts those differences and where further evidence is needed.

This also changes how verifier improvement should be assessed. Optimization can increase rubric scores while exploiting what the rubric omits [107, 212]. Co-evolving grader research likewise shows that downstream agent gains can coexist with a verifier that has lost its ability to reject defective outputs [213]. Future evaluation should follow quality judgments and use outcomes through model, harness, and judge updates. Establishing when an improvement in assessed quality leads to an improvement in actual work would make verification a stronger guide to both learning and deployment, and provide the evidence needed to compare alternative organizations of work.

## 7.4 Work-Centric Systems: From Task Automation to Scalable Work Delivery

Working agents may change the organization of digital work as well as its execution. Goals can outlast any particular model, specialist, or application; capable executors can be brought in for one stage and replaced for the next. These different lifetimes suggest a system organized around persistent work and adaptable participation. We consider three connected questions: how work retains its identity as its structure changes, how continuing coordination relates to specialized execution, and how the resulting organization changes the limits on useful output.

### 7.4.1 Work as a First-Class System Object

Work identity should survive changes in execution. A report can be decomposed into different tasks, reassigned to another agent, or revised under an upgraded model while retaining its purpose and outstanding obligations. Agent loops and workflow graphs organize execution [75, 79, 214]; case handling provides a precedent for organizing activity around an enduring work object [215]. The distinctive challenge for working agents is that language models can discover and revise the work structure itself from incomplete information.

A durable *work instance* could retain the goal, requirements, dependencies, decisions, outcomes, unresolved obligations, and acceptance evidence. Maintaining such a record becomes difficult when a model infers a previously unknown dependency, merges two apparent tasks, or concludes that a requirement has already been met. The research problem is to make these structural changes accurate, inspectable, and usable by other participants. Inferred descriptions of the workspace, proposed plans, and binding work decisions need distinguishable status, so a tentative interpretation can be revised without erasing an established requirement.

One direction is to represent decomposition and reassignment as explicit changes to shared work state. Splitting a task would preserve where each requirement is addressed; replacing an executor would preserve confirmed effects and pending obligations; changing a requirement would identify the decisions and outcomes it affects. These updates could be checked against current evidence before becoming the basis for subsequent action. The consequence records in Section 7.2.2 and the evidence tied to requirements and versions in Section 7.3.2 would support those checks, while the work instance would preserve their connection to the delegated goal.

This makes continuity across executors a concrete systems objective. Studies could vary decomposition, model versions, and participant availability during the same body of work, then examine lost obligations, inconsistent updates, and the cost of maintaining an accurate record. A useful representation would retain enough structure to support reliable coordination while allowing routine details to remain local to an executor. It would give people and agents a stable object through which to divide work even as the means of execution change.

### 7.4.2 Human--Agent Work Organization and Accountability

Continuing goals and specialized execution may support different organizational roles. Personal-agent research brings recurring user needs and the timing of assistance into focus [187–189]; multi-agent frameworks organize collaboration among executors with different roles [79, 80, 216]. A promising system direction is a persistent coordination layer that maintains goals, priorities, and unfinished responsibilities, coupled to specialist executors called upon for coding, analysis, documents, or business operations. The coordinator would retain continuity as the available specialists and their capabilities change.

This division is especially useful to investigate when user goals remain stable across heterogeneous tasks and executors are frequently replaced. Its benefit depends on whether retained context reduces repeated explanation and lost responsibility enough to offset coordination overhead. A coordinator that misunderstands a goal can propagate that mistake to several specialists. Comparisons with a single continuing executor and with distributed coordination should therefore examine both task completion and the effort required to establish and correct the shared understanding of work.

Delegation would need to transfer a bounded responsibility together with its evidence and authority. Authenticated delegation provides mechanisms for scoped permissions [217], while studies of handoffs show how governing constraints can disappear from otherwise informative summaries [218]. A recipient should be able to identify what has been accepted, what remains unresolved, which actions are permitted, and when responsibility returns to another participant. Personal context and organizational records also need distinct access boundaries: a specialist should receive the information needed for its work under the applicable permissions. User preferences can rank permitted choices; current delegation determines the authority to act.

Human participation would concentrate on defining goals and authorization boundaries, resolving conflicting priorities, and making judgments that require their expertise or decision rights. Human–agent teaming treats this as a joint activity [219]. Its design must account for the cost and timing of intervention: Proactivity-Gym's

human study finds that assistance can damage trust through poorly aligned intervention even when its output is correct [188]. Learning when to continue, hand off, or seek a decision could reduce repeated orchestration while keeping unresolved responsibilities visible.

This division of labor also determines who can use the system. A domain expert should be able to delegate an analysis without configuring a runtime or translating the goal into tool calls. A student tackling a disciplinary project may need help expressing requirements and interpreting the result. Studies should vary domain expertise and technical familiarity separately, measuring successful delegation, correction, and use alongside the effort of operating the agent. Broad adoption would then be assessed through the range of people who can obtain useful work with manageable assistance, connecting occupational coverage to accessible execution.

### 7.4.3 Net Productivity and Shifting Bottlenecks

Capability growth can change which part of a work process limits useful output. When execution is unreliable, improving completion directly expands what can be delivered. When usable drafts become plentiful, review and integration can become limiting. When several specialists execute in parallel, selecting worthwhile goals, allocating responsibility, and obtaining timely human decisions can consume the scarce capacity. Which constraint dominates depends on the work, its acceptance conditions, and the surrounding organization.

Existing productivity studies motivate examining that dependence. Generative AI improved speed and quality in a study of professional writing [220], while early-2025 AI tools slowed experienced developers completing tasks in their own repositories [221]. These settings expose different relationships between model assistance and the surrounding work. Organizational complements and the redistribution of human effort have long been central to understanding automation [222–224]. For working agents, the research opportunity is to identify how those relationships change as execution, verification, and coordination improve at different rates.

The relevant outcome is *accepted work*: results that meet the applicable requirements and are used downstream. The metrics in Section 6.3 describe quality and total effort; future studies should use them to locate the limiting stage. Controlled interventions could increase execution capacity, improve evidence supplied for review, or change the delegation process while holding work requirements comparable. The change in completed and used outcomes would reveal where an additional unit of capability has the greatest value. Longitudinal studies should repeat this diagnosis as both system capability and the mix of delegated work evolve, separating gains on a fixed workload from expansion into previously infeasible work or access by new user groups.

This perspective gives the optimization choices in Section 5.4 an organizational target. More training is valuable where decision errors constrain delivery; better evidence and incremental verification matter where review is limiting; persistent coordination matters where work is lost between participants. The same intervention can have a different value after another constraint is removed. Future working systems should therefore learn from the complete path through execution, acceptance, use, and maintenance, directing improvement toward the current limitation. Their value will lie in reducing the total burden of work and expanding the outcomes that can be usefully delivered under the available resources.

## 8 Conclusion

In this survey, we review working agents through a central thesis: reliable digital work is produced by a working system, in which a foundation model and a runtime harness act in a stateful workspace, and it is established by evidence that the resulting outcomes meet the requirements of the work. From this view, we develop the discussion through six aspects. **Foundation** defines a working agent through six elements, namely task, model, harness, workspace, work outcome, and verification, and describes working conditions along two independent dimensions: work persistence, which concerns how long delegated responsibility remains open, and work-world openness, which concerns how much of the task world can be specified before execution. **Work Forms** organizes delegated work by its scope, from response-level and atomic-action-level work through multi-step and workflow-level work to workspace-level work. **Capabilities** identifies six families of functions that must hold across the execution loop: workspace understanding, perception, reasoning and decision making, planning and coordination, execution and state management, and verification, recovery, and replanning. Reliable work requires the workspace model, observations, decisions, committed state, and verification evidence to remain in

agreement. **Optimization** follows work experience into system capability. Training environments, trajectories, and rewards make work learnable; model optimization internalizes capabilities through mid-training and supervised fine-tuning, agentic reinforcement learning, on-policy distillation, and self-improving training; harness optimization improves interfaces and tools, prompts and workflows, and memory and skills; and the coupling of model and harness decides whether a capability gap is best closed by learning, by external support, or by coordinated changes to both. **Evaluation and Resources** reads benchmarks through an evidence chain that runs from the requested work to the capability claim, and examines task construction, evaluation methodologies, outcome-aligned metrics, cross-benchmark coverage, and the frameworks and environments in which agents run. **Frontiers** consolidates open problems into four directions, namely environment, persistence, verification, and work-centric systems, and frames progress in terms of accepted work, meaning outcomes accepted against their requirements and used downstream, together with the net productivity of delivering them.

Three lessons recur across these aspects. First, capability belongs to a configuration rather than to a model. The same requirement can be met through training or through an interface, a memory, a skill, or a verifier, and end-to-end results describe the model, harness, workspace, and verification procedure together. Second, the decisive evidence lies in the workspace. Passing tests, recalculated workbooks, committed application state, and accepted artifacts establish what transcripts and declarations of completion cannot, and both optimization and evaluation are only as reliable as the evidence they inspect. Third, persistence and openness change what reliability requires. As responsibilities outlast a single execution and work worlds become less specified in advance, an agent must keep its commitments, its view of the current state, and the grounds for accepting its results aligned while both its environment and its own capabilities change.

We hope this survey offers a shared vocabulary and a useful reference for researchers and practitioners who build, train, and evaluate agents for digital work. As agents take on responsibilities that persist across time and organizations, the measure of progress will move from tasks completed to work delivered and accepted. Working agents will therefore remain a central problem for building AI systems that are useful, dependable, and accountable in practice.

## References


[1] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. arXiv preprint arXiv:2302.04761, 2023. URL https://arxiv.org/abs/2302.04761.

[2] Minghao Li, Yingxiu Zhao, Bowen Yu, et al. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. arXiv preprint, 2023. URL https://arxiv.org/abs/2304.08244.

[3] Shuyan Zhou, Frank F. Xu, Hao Zhu, et al. WebArena: A Realistic Web Environment for Building Autonomous Agents. arXiv preprint, 2023. URL https://arxiv.org/abs/2307.13854.

[4] Tianbao Xie, Danyang Zhang, Jixuan Chen, et al. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. arXiv preprint, 2024. URL https://arxiv.org/abs/2404.07972.

[5] Hongxin Li, Jingran Su, Yuntao Chen, et al. SheetCopilot: Bringing Software Productivity to the Next Level through Large Language Models. arXiv preprint, 2023. URL https://arxiv.org/abs/2305.19308.

[6] Hao Zheng, Xinyan Guan, Hao Kong, et al. PPTAgent: Generating and Evaluating Presentations Beyond Text-to-Slides. arXiv preprint, 2025. URL https://arxiv.org/abs/2501.03936.

[7] Alexandre Drouin, Maxime Gasse, Massimo Caccia, et al. WorkArena: How Capable Are Web Agents at Solving Common Knowledge Work Tasks? arXiv preprint, 2024. URL https://arxiv.org/abs/2403.07718.

[8] John Yang, Carlos E. Jimenez, Alexander Wettig, et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv preprint, 2024. URL https://arxiv.org/abs/2405.15793.

[9] Minesh Mathew, Dimosthenis Karatzas, and C. V Jawahar. DocVQA: A Dataset for VQA on Document Images. arXiv preprint, 2020. URL https://arxiv.org/abs/2007.00398.

[10] Zhiyu Chen, Wenhu Chen, Charese Smiley, et al. FinQA: A Dataset of Numerical Reasoning over Financial Data. arXiv preprint, 2021. URL https://arxiv.org/abs/2109.00122.

[11] Fengbin Zhu, Wenqiang Lei, Youcheng Huang, et al. TAT-QA: A Question Answering Benchmark on a Hybrid of Tabular and Textual Content in Finance. arXiv preprint, 2021. URL https://arxiv.org/abs/2105.07624.

[12] Jiarui Lu, Thomas Holleis, Yizhe Zhang, et al. ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. arXiv preprint, 2024. URL https://arxiv.org/abs/2408.04682.

[13] Harsh Trivedi, Tushar Khot, Mareike Hartmann, et al. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. arXiv preprint, 2024. URL https://arxiv.org/abs/2407.18901.

[14] Xingyao Wang, Zihan Wang, Jiateng Liu, Yangyi Chen, Lifan Yuan, Hao Peng, and Heng Ji. Mint: Evaluating llms in multi-turn interaction with tools and language feedback. arXiv preprint arXiv:2309.10691, 2023. URL https://arxiv.org/abs/2309.10691.

[15] Shunyu Yao, Noah Shinn, Pedram Razavi, et al. tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv preprint, 2024. URL https://arxiv.org/abs/2406.12045.

[16] Victor Barres, Honghua Dong, Soham Ray, et al. tauˆ2-Bench: Evaluating Conversational Agents in a Dual-Control Environment. arXiv preprint, 2025. URL https://arxiv.org/abs/2506.07982.

[17] Thomas Kwa, Ben West, Joel Becker, and Amy Deng. Measuring ai ability to complete long software tasks. arXiv preprint arXiv:2503.14499, 2025. URL https://arxiv.org/abs/2503.14499.

[18] Hjalmar Wijk, Tao Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Josh Clymer, Jai Dhyani, Elena Ericheva, Katharyn Garcia, Brian Goodrich, Nikola Jurkovic, Holden Karnofsky, Megan Kinniment, Aron Lajko, Seraphina Nix, Lucas Sato, William Saunders, Maksym Taran, Ben West, and Elizabeth Barnes. Re-bench: Evaluating frontier ai r&d capabilities of language model agents against human experts. arXiv preprint arXiv:2411.15114, 2024. URL https://arxiv.org/abs/2411.15114.

[19] Zirui Tang, Xuanhe Zhou, Yumou Liu, et al. Workspace-Bench 1.0: Benchmarking AI Agents on Workspace Tasks with Large-Scale File Dependencies. arXiv preprint, 2026. URL https://arxiv.org/abs/2605.03596.

[20] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. Squad: 100,000+ questions for machine comprehension of text, 2016. URL https://arxiv.org/abs/1606.05250.

[21] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering, 2018. URL https://arxiv.org/abs/1809.09600.

[22] Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. Chartqa: A benchmark for question answering about charts with visual and logical reasoning. arXiv preprint arXiv:2203.10244, 2022. URL https://arxiv.org/abs/2203.10244.

[23] Neel Guha, Julian Nyarko, Daniel E. Ho, et al. LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models. arXiv preprint, 2023. URL https://arxiv.org/abs/2308.11462.

[24] Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William W. Cohen, and Xinghua Lu. Pubmedqa: A dataset for biomedical research question answering, 2019. URL https://arxiv.org/abs/1909.06146.

[25] Pan Lu, Swaroop Mishra, Tony Xia, Liang Qiu, Kai-Wei Chang, Song-Chun Zhu, Oyvind Tafjord, Peter Clark, and Ashwin Kalyan. Learn to explain: Multimodal reasoning via thought chains for science question answering, 2022. URL https://arxiv.org/abs/2209.09513.

[26] Justin Payan, Swaroop Mishra, Mukul Singh, et al. InstructExcel: A Benchmark for Natural Language Instruction in Excel, 2023. URL https://aclanthology.org/2023.findings-emnlp.265/.

[27] Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In Forty-second International Conference on Machine Learning, 2025.

[28] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Li YanTao, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing GUI grounding for advanced visual GUI agents. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 9313–9332, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.505. URL https://aclanthology.org/2024.acl-long.505/.

[29] Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. Screenspot-pro: Gui grounding for professional high-resolution computer use. arXiv preprint arXiv:2504.07981, 2025. URL https://arxiv.org/abs/2504.07981.

[30] Qianhui Wu, Kanzhi Cheng, Rui Yang, Chaoyun Zhang, Jianwei Yang, Huiqiang Jiang, Jian Mu, Baolin Peng, Bo Qiao, Reuben Tan, Si Qin, Lars Liden, Qingwei Lin, Huan Zhang, Tong Zhang, Jianbing Zhang, Dongmei Zhang, and Jianfeng Gao. Gui-actor: Coordinate-free visual grounding for gui agents, 2025. URL https://arxiv.org/abs/2506.03143.

[31] Alex Egg, Martin Iglesias Goyanes, Friso Kingma, Andreu Mora, Leandro von Werra, and Thomas Wolf. Dabstep: Data agent benchmark for multi-step reasoning. arXiv preprint arXiv:2506.23719, 2025. URL https://arxiv.org/abs/2506.23719.

[32] Zhenting Wang, Qi Chang, Hemani Patel, Shashank Biju, Cheng-En Wu, Quan Liu, Aolin Ding, Alireza Rezazadeh, Ankit Shah, Yujia Bao, and Eugene Siow. Mcp-bench: Benchmarking tool-using llm agents with complex real-world tasks via mcp servers, 2025. URL https://arxiv.org/abs/2508.20453.

[33] Carlos E. Jimenez, John Yang, Alexander Wettig, et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv preprint, 2023. URL https://arxiv.org/abs/2310.06770.

[34] Liqiang Jing, Zhehui Huang, Xiaoyang Wang, et al. DSBench: How Far Are Data Science Agents from Becoming Data Science Experts? arXiv preprint, 2024. URL https://arxiv.org/abs/2409.07703.

[35] Dan Zhang, Sining Zhoubian, Min Cai, et al. DataSciBench: An LLM Agent Benchmark for Data Science. arXiv preprint, 2025. URL https://arxiv.org/abs/2502.13897.

[36] Ziru Chen, Shijie Chen, Yuting Ning, et al. ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery. arXiv preprint, 2024. URL https://arxiv.org/abs/2410.05080.

[37] Yiduo Guo, Zekai Zhang, Yaobo Liang, et al. PPTC Benchmark: Evaluating Large Language Models for PowerPoint Task Completion. arXiv preprint, 2023. URL https://arxiv.org/abs/2311.01767.

[38] Xin-Sheng Chen, Jiayu Zhu, Pei-lin Li, et al. PresentBench: A Fine-Grained Rubric-Based Benchmark for Slide Generation. arXiv preprint, 2026. URL https://arxiv.org/abs/2603.07244.

[39] Guancheng Zeng, Xueyi Chen, Jiawang Hu, Shaohua Qi, Yaxuan Mao, Zhantao Wang, Yifan Nie, Shuang Li, Qiuyang Feng, Pengxu Qiu, Yujia Wang, Wenqiang Han, Linyan Huang, Gang Li, Jingjing Mo, and Haowen Hu. Routine: A structural planning framework for llm agent system in enterprise, 2025. URL https://arxiv.org/abs/2507.14447.

[40] Yibin Chen, Yifu Yuan, Zeyu Zhang, Yan Zheng, Jinyi Liu, Fei Ni, Jianye Hao, Hangyu Mao, and Fuzheng Zhang. Sheetagent: Towards a generalist agent for spreadsheet reasoning and manipulation via large language models, 2025. URL https://arxiv.org/abs/2403.03636.

[41] Léo Boisvert, Megh Thakkar, Maxime Gasse, et al. WorkArena++: Towards Compositional Planning and Reasoning-based Common Knowledge Work Tasks. arXiv preprint, 2024. URL https://arxiv.org/abs/2407.05291.

[42] Zilong Wang, Yuedong Cui, Li Zhong, et al. OfficeBench: Benchmarking Language Agents across Multiple Applications for Office Automation. arXiv preprint, 2024. URL https://arxiv.org/abs/2407.19056.

[43] Frank F. Xu, Yufan Song, Boxuan Li, et al. TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks. arXiv preprint, 2024. URL https://arxiv.org/abs/2412.14161.

[44] Kung-Hsiang Huang, Akshara Prabhakar, Sidharth Dhawan, et al. CRMArena: Understanding the Capacity of LLM Agents to Perform Professional CRM Tasks in Realistic Environments. arXiv preprint, 2024. URL https://arxiv.org/abs/2411.02305.

[45] Kung-Hsiang Huang, Akshara Prabhakar, Onkar Thorat, et al. CRMArena-Pro: Holistic Assessment of LLM Agents Across Diverse Business Scenarios and Interactions. arXiv preprint, 2025. URL https://arxiv.org/abs/2505.18878.

[46] Jian Zhu, Yuzheng Zhang, Zeyao Ma, Bohan Zhang, Armin Schoepf, Daniel Woloch, Peter Yiliu Wang, Guangyu Robert Yang, Samuel Jacob, Siddharth Nagisetty, Abhiram Chundru, Jean Lin, Spencer Mateega, and Jing Zhang. Spreadsheetbench 2: Evaluating agents on end-to-end business spreadsheet workflows, 2026. URL https://arxiv.org/abs/2606.29955.

[47] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, et al. AFlow: Automating Agentic Workflow Generation. arXiv preprint, 2024. URL https://arxiv.org/abs/2410.10762.

[48] Hanchao Liu, Rongjun Li, Weimin Xiong, Ziyu Zhou, and Wei Peng. Workteam: Constructing workflows from natural language with multi-agents, 2025. URL https://arxiv.org/abs/2503.22473.

[49] Houxing Ren, Mingjie Zhan, Zimu Lu, Ke Wang, Yunqiao Yang, Haotian Hou, and Hongsheng Li. Towards robust real-world spreadsheet understanding with multi-agent multi-format reasoning, 2026. URL https://arxiv.org/abs/2604.12282.

[50] Chenglin Yu, Yuchen Wang, Songmiao Wang, Hongxia Yang, and Ming Li. Infiagent: An infinite-horizon framework for general-purpose autonomous agents, 2026. URL https://arxiv.org/abs/2601.03204.

[51] Haoyu Dong, Jianbo Zhao, Yuzhang Tian, et al. SpreadsheetLLM: Encoding Spreadsheets for Large Language Models. arXiv preprint, 2024. URL https://arxiv.org/abs/2407.09025.

[52] Kunlun Zhu, Hongyi Du, Zhaochen Hong, Xiaocheng Yang, Shuyi Guo, Zhe Wang, Zhenhailong Wang, Cheng Qian, Xiangru Tang, Heng Ji, and Jiaxuan You. MultiAgentBench: Evaluating the collaboration and competition of LLM agents. arXiv preprint arXiv:2503.01935, 2025. URL https://arxiv.org/abs/2503.01935.

[53] Tejal Patwardhan, Rachel Dias, Elizabeth Proehl, et al. GDPval: Evaluating AI Model Performance on Real-World Economically Valuable Tasks, 2025. URL https://cdn.openai.com/pdf/d5eb7428-c4e9-4a33-bd86-86dd4bcf12ce/GDPval.pdf.

[54] Olly Styles, Sam Miller, Patricio Cerda-Mardini, Tanaya Guha, Victor Sanchez, and Bertie Vidgen. Workbench: a benchmark dataset for agents in a realistic workplace setting. arXiv preprint arXiv:2405.00823, 2024. URL https://arxiv.org/abs/2405.00823.

[55] Jing Yu Koh, Robert Lo, Lawrence Jang, et al. VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. arXiv preprint, 2024. URL https://arxiv.org/abs/2401.13649.

[56] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, et al. Windows Agent Arena: Evaluating Multi-Modal OS Agents at Scale. arXiv preprint, 2024. URL https://arxiv.org/abs/2409.08264.

[57] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, et al. AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents. arXiv preprint, 2024. URL https://arxiv.org/abs/2405.14573.

[58] Fangyu Lei, Jixuan Chen, Yuxiao Ye, et al. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. arXiv preprint arXiv:2411.07763, 2024. URL https://arxiv.org/abs/2411.07763.

[59] Yixing Jiang, Kameron C. Black, Gloria Geng, Danny Park, James Zou, Andrew Y. Ng, and Jonathan H. Chen. MedAgentBench: A Realistic Virtual EHR Environment to Benchmark Medical LLM Agents. arXiv preprint arXiv:2501.14654, 2025. URL https://arxiv.org/abs/2501.14654.

[60] Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. OmniParser for Pure Vision Based GUI Agent. arXiv preprint arXiv:2408.00203, 2024. URL https://arxiv.org/abs/2408.00203.

[61] Saaket Agashe, Kyle Wong, Vincent Tu, Jiachen Yang, Ang Li, and Xin Eric Wang. Agent S2: A Compositional Generalist-Specialist Framework for Computer Use Agents. arXiv preprint arXiv:2504.00906, 2025. URL https://arxiv.org/abs/2504.00906.

[62] Linke Ouyang, Yuan Qu, Hongbin Zhou, Jiawei Zhu, Rui Zhang, Qunshu Lin, Bin Wang, Zhiyuan Zhao, Man Jiang, Xiaomeng Zhao, Jin Shi, Fan Wu, Pei Chu, Minghao Liu, Zhenxiang Li, Chao Xu, Bo Zhang, Botian Shi, Zhongying Tu, and Conghui He. Omnidocbench: Benchmarking diverse pdf document parsing with comprehensive annotations. arXiv preprint arXiv:2412.07626, 2024. URL https://arxiv.org/abs/2412.07626.

[63] Yubo Ma, Yuhang Zang, Liangyu Chen, Meiqi Chen, Yizhu Jiao, Xinze Li, Xinyuan Lu, Ziyu Liu, Yan Ma, Xiaoyi Dong, Pan Zhang, Liangming Pan, Yu-Gang Jiang, Jiaqi Wang, Yixin Cao, and Aixin Sun. Mmlongbench-doc: Benchmarking long-context document understanding with visualizations. arXiv preprint arXiv:2407.01523, 2024. URL https://arxiv.org/abs/2407.01523.

[64] Zeyao Ma, Bohan Zhang, Jing Zhang, et al. SpreadsheetBench: Towards Challenging Real World Spreadsheet Manipulation. arXiv preprint, 2024. URL https://arxiv.org/abs/2406.14991.

[65] Tianyang Liu, Canwen Xu, and Julian McAuley. Repobench: Benchmarking repository-level code auto-completion systems. arXiv preprint arXiv:2306.03091, 2023. URL https://arxiv.org/abs/2306.03091.

[66] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous Program Improvement. arXiv preprint arXiv:2404.05427, 2024. URL https://arxiv.org/abs/2404.05427.

[67] Sirui Hong, Yizhang Lin, Bang Liu, et al. Data Interpreter: An LLM Agent For Data Science. arXiv preprint arXiv:2402.18679, 2024. URL https://arxiv.org/abs/2402.18679.

[68] Zhiyu Chen, Shiyang Li, Charese Smiley, et al. ConvFinQA: Exploring the Chain of Numerical Reasoning in Conversational Finance Question Answering. arXiv preprint, 2022. URL https://arxiv.org/abs/2210.03849.

[69] Antoine Bigeard, Langston Nashold, Rayan Krishnan, and Shirley Wu. Finance Agent Benchmark: Benchmarking LLMs on Real-world Financial Research Tasks. arXiv preprint arXiv:2508.00828, 2025. URL https://arxiv.org/abs/2508.00828.

[70] Zhiwei Fei, Xiaoyu Shen, Dawei Zhu, Fengzhe Zhou, Zhuo Han, Songyang Zhang, Kai Chen, Zongwen Shen, and Jidong Ge. Lawbench: Benchmarking legal knowledge of large language models. arXiv preprint arXiv:2309.16289, 2023. URL https://arxiv.org/abs/2309.16289.

[71] Qianchu Liu, Sheng Zhang, Guanghui Qin, et al. HealthAgentBench: A Unified Benchmark Suite of Realistic Agentic Healthcare Environments for Challenging Frontier AI Agents. arXiv preprint arXiv:2606.31179, 2026. URL https://arxiv.org/abs/2606.31179.

[72] Bodhisattwa Prasad Majumder, Harshit Surana, Dhruv Agarwal, Bhavana Dalvi Mishra, Abhijeetsingh Meena, Aryan Prakhar, Tirth Vora, Tushar Khot, Ashish Sabharwal, and Peter Clark. Discoverybench: Towards data-driven discovery with large language models. arXiv preprint arXiv:2407.01725, 2024. URL https://arxiv.org/abs/2407.01725.

[73] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, et al. Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. arXiv preprint arXiv:2601.11868, 2026. URL https://arxiv.org/abs/2601.11868.

[74] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better llm agents. arXiv preprint arXiv:2402.01030, 2024. URL https://arxiv.org/abs/2402.01030.

[75] Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv preprint, 2022. URL https://arxiv.org/abs/2210.03629.

[76] Karthik Valmeekam, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change. arXiv preprint arXiv:2206.10498, 2022. URL https://arxiv.org/abs/2206.10498.

[77] Jian Xie, Kai Zhang, Jiangjie Chen, Tinghui Zhu, Renze Lou, Yuandong Tian, Yanghua Xiao, and Yu Su. Travelplanner: A benchmark for real-world planning with language agents. arXiv preprint arXiv:2402.01622, 2024. URL https://arxiv.org/abs/2402.01622.

[78] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. arXiv preprint arXiv:2409.07429, 2024. URL https://arxiv.org/abs/2409.07429.

[79] Qingyun Wu, Gagan Bansal, Jieyu Zhang, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv preprint, 2023. URL https://arxiv.org/abs/2308.08155.

[80] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, et al. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. arXiv preprint, 2023. URL https://arxiv.org/abs/2308.00352.

[81] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for ai software developers as generalist agents. arXiv preprint arXiv:2407.16741, 2024. URL https://arxiv.org/abs/2407.16741.

[82] Chaoyun Zhang, He Huang, Chiming Ni, et al. UFO2: The Desktop AgentOS. arXiv preprint arXiv:2504.14603, 2025. URL https://arxiv.org/abs/2504.14603.

[83] Charles Packer, Sarah Wooders, Kevin Lin, et al. MemGPT: Towards LLMs as Operating Systems. arXiv preprint, 2023. URL https://arxiv.org/abs/2310.08560.

[84] Prateek Chhikara, Dev Khant, Saket Aryan, et al. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv preprint, 2025. URL https://arxiv.org/abs/2504.19413.

[85] Di Wu, Hongwei Wang, Wenhao Yu, et al. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. arXiv preprint, 2024. URL https://arxiv.org/abs/2410.10813.

[86] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, et al. Evaluating Very Long-Term Conversational Memory of LLM Agents. arXiv preprint, 2024. URL https://arxiv.org/abs/2402.17753.

[87] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, et al. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. arXiv preprint, 2024. URL https://arxiv.org/abs/2406.13352.

[88] Aman Madaan, Niket Tandon, Prakhar Gupta, et al. Self-Refine: Iterative Refinement with Self-Feedback. arXiv preprint, 2023. URL https://arxiv.org/abs/2303.17651.

[89] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In International Conference on Learning Representations, 2024. URL https://arxiv.org/abs/2305.11738.

[90] Hao Zheng, Guozhao Mo, Xinru Yan, et al. DeepPresenter: Environment-Grounded Reflection for Agentic Presentation Generation. arXiv preprint arXiv:2602.22839, 2026. URL https://arxiv.org/abs/2602.22839.

[91] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying LLM-Based Software Engineering Agents. Proceedings of the ACM on Software Engineering, 2(FSE):801–824, 2025. doi: 10.1145/3715754.

[92] Noah Shinn, Federico Cassano, Edward Berman, et al. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv preprint, 2023. URL https://arxiv.org/abs/2303.11366.

[93] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning acting and planning in language models. arXiv preprint arXiv:2310.04406, 2023. URL https://arxiv.org/abs/2310.04406.

[94] Jiarong Zhao, Zhikai Lei, Zhiheng Xi, Rui Zheng, Hang Yan, Jie Zhou, Qin Chen, and Liang He. NexForge: Scaling Agent Capabilities through Requirement-Driven Task Synthesis for LLMs. arXiv preprint arXiv:2607.14186, 2026. URL https://arxiv.org/abs/2607.14186.

[95] Qisheng Su, Hanchen Wang, Guanru Zhu, Huicheng Jiang, Qiuyinzhe Zhang, Kou Shi, Zhen Fang, Ziao Zhang, Qingnan Ren, Honglin Guo, Zehui Chen, Tao Gui, and Feng Zhao. GraphForge: Training Working Agents with Graph-Anchored Workspace Synthesis. arXiv preprint arXiv:2609.38923, 2026. URL https://arxiv.org/abs/2609.38923.

[96] Xinyu Zhu, Fenyi Liu, Yuzhu Cai, Shuo Tang, Rui Ye, Linfeng Zhang, and Siheng Chen. WorkGenesis: Building the Worlds That Teach Agents to Work. arXiv preprint arXiv:2609.39325, 2026. URL https://arxiv.org/abs/2609.39325.

[97] Zhaoyang Wang, Canwen Xu, Boyi Liu, Yite Wang, Siwei Han, Zhewei Yao, Huaxiu Yao, and Yuxiong He. Agent World Model: Infinity Synthetic Environments for Agentic Reinforcement Learning. arXiv preprint arXiv:2602.10090, 2026. URL https://arxiv.org/abs/2602.10090.

[98] Yirong Zeng, Shen You, Jinhang Feng, Yufei Liu, Xiao Ding, Yutai Hou, Hao Cong, Yuxian Wang, Wu Ning, Wang Xu, and Bibo Cai. EnvCraft: Synthesizing Executable Environments in Agentic RL for Claw-like Agent. arXiv preprint arXiv:2609.05576, 2026. URL https://arxiv.org/abs/2609.05576.

[99] Minbyul Jeong and Chanwoong Yoon. AgentMercury: Your Agent Can Synthesize Verifiable Environments for Business Scenarios at scale. arXiv preprint arXiv:2608.20634, 2026. URL https://arxiv.org/abs/2608.20634.

[100] Weiyi Xu, Xiaowen Yang, Wen Da, Hang Xu, Canwei Li, Hongjie You, Pusen Dong, Yucheng Zeng, Zhaokai Luo, and Mu Chuan. Skill2Env: Capability-Oriented Environment Synthesis from Skills for General Agents. arXiv preprint arXiv:2609.33772, 2026. URL https://arxiv.org/abs/2609.33772.

[101] Ankush Agarwal, Harsh Vishwakarma, Suraj Nagaje, and Chaitanya Devaguptapu. EnterpriseLab: A Full-Stack Platform for developing and deploying agents in Enterprises. arXiv preprint arXiv:2603.21630, 2026. URL https://arxiv.org/abs/2603.21630.

[102] Zhuohang Fan, Beichen Zhang, Yuanfa Li, Changqiao Wu, Wei Liu, Jian Luan, and Weigang Zhang. SEE: Structure-aware Exploring & Exploiting for Long-horizon GUI Agent Trajectory Synthesis. arXiv preprint arXiv:2607.18046, 2026. URL https://arxiv.org/abs/2607.18046.

[103] Negin Raoof, Richard Zhuang, Marianna Nezhurina, Etash Guha, Atula Tejaswi, Ryan Marten, Charlie F. Ruan, Tyler Griggs, Alexander Glenn Shaw, Hritik Bansal, E. Kelly Buchanan, Artem Gazizov, Reinhard Heckel, Chinmay Hegde, Sankalp Jajee, Daanish Khazi, Emmanouil Koukoumidis, Xiangyi Li, Hange Liu, Shlok Natarajan, Harsh Raj, Nicholas Roberts, Ethan Shen, Nishad Singhi, Michael Siu, Ashima Suvarna, Hanwen Xing, Patrick Yubeaton, Robert Zhang, Leon Liangyu Chen, Xiaokun Chen, Steven Dillmann, Saadia Gabriel, Xunyi Jiang, Anurag Kashyap, Boxuan Li, Yein Park, Minh Pham, Sujay Sanghavi, Lin Shi, Ke Sun, Yixin Wang, Zhiwei Xu, Erica Zhang, Siyan Zhao, Wanjia Zhao, Jenia Jitsev, Alex Dimakis, Benjamin Feuer, and Ludwig Schmidt. OpenThoughts-Agent: Data Recipes for Agentic Models. arXiv preprint arXiv:2606.24855, 2026. URL https://arxiv.org/abs/2606.24855.

[104] Dingzirui Wang, Xuanliang Zhang, Keyan Xu, Qingfu Zhu, and Wanxiang Che. CurateEvo: Data-Curation Evolving for Agentic Post-Training. arXiv preprint arXiv:2607.06140, 2026. URL https://arxiv.org/abs/2607.06140.

[105] Banghao Chi, Yining Xie, Mingyuan Wu, et al. Spreadsheet-RL: Advancing Large Language Model Agents on Realistic Spreadsheet Tasks via Reinforcement Learning. arXiv preprint, 2026. URL https://arxiv.org/abs/2605.22642.

[106] Maksim Ivanov and Abhijay Rana. Anchor: Mitigating Artifact Drift in Agent Benchmark Generation. arXiv preprint arXiv:2605.26321, 2026. URL https://arxiv.org/abs/2605.26321.

[107] Yiming Pan, Chengwei Hu, Xuancheng Huang, et al. AeSlides: Incentivizing Aesthetic Layout in LLM-Based Slide Generation via Verifiable Rewards. arXiv preprint, 2026. URL https://arxiv.org/abs/2604.22840.

[108] Karthik Ragunath Ananda Kumar and Subrahmanyam Arunachalam. Learning to Present: Inverse Specification Rewards for Agentic Slide Generation. arXiv preprint, 2026. URL https://arxiv.org/abs/2603.16839.

[109] Shangqing Tu, Daniel Zhang-Li, Yucheng Wang, Shiyu Gan, Yanpeng Wang, Huiqiang Rong, Mofei Chen, Shen Yang, Yini Chen, Yinuo Duan, Binglin Liu, Ye He, Danqi Zheng, Zhanxin Hao, Yuxuan Wu, Mengting Tao, Yuqiu Liu, Jifan Yu, Juanzi Li, Bin Xu, Lei Hou, Huiqin Liu, and Yu Zhang. CogEvol: Towards Efficient and Reliable Learning Environment Generation. arXiv preprint arXiv:2608.30968, 2026. URL https://arxiv.org/abs/2608.30968.

[110] Junqi Liu, Yongyang Pan, Zhuosong Jiang, Dongbai Li, Bo Zhang, Xitong Ling, Sheng Wang, Hanrong Ye, Yufan He, Can Zhao, Pengfei Guo, Dong Yang, Andriy Myronenko, Yuyin Zhou, Tianyu Liu, Daguang Xu, and Yucheng Tang. VERA: Scaling Verifiable Environments for Agentic co-Evolution. arXiv preprint arXiv:2610.05923, 2026. URL https://arxiv.org/abs/2610.05923.

[111] Dayuan Fu, Shenyu Wu, Yunze Wu, Zerui Peng, Yaxing Huang, Jie Sun, Ji Zeng, Mohan Jiang, Lin Zhang, Yukun Li, Jiarui Hu, Liming Liu, Jinlong Hou, and Pengfei Liu. daVinci-Env: Open SWE Environment Synthesis at Scale. arXiv preprint arXiv:2603.13023, 2026. URL https://arxiv.org/abs/2603.13023.

[112] Xiaoshuai Song, Haofei Chang, Guanting Dong, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. Envscaler: Scaling tool-interactive environments for llm agent via programmatic synthesis. arXiv preprint arXiv:2601.05808, 2026. URL https://arxiv.org/abs/2601.05808.

[113] Guhong Chen, Chenghao Sun, Cheng Fu, Qiyao Wang, Zhihong Huang, Chaopeng Wei, Guangxu Chen, Feiteng Fang, Ahmadreza Argha, Bing Zhao, Xander Xu, Qi Han, Hamid Alinejad-Rokny, Qiang Qu, Binhua Li, Shiwen Ni, Min Yang, Hu Wei, and Yongbin Li. Beyond Quantity: Trajectory Diversity Scaling for Code Agents. arXiv preprint arXiv:2602.03219, 2026. URL https://arxiv.org/abs/2602.03219.

[114] Ji Zeng, Dayuan Fu, Tiantian Mi, Yumin Zhuang, Yaxing Huang, Xuefeng Li, Lyumanshan Ye, Muhang Xie, Qishuo Hua, Zhen Huang, Mohan Jiang, Hanning Wang, Jifan Lin, Yang Xiao, Jie Sun, Yunze Wu, and Pengfei Liu. daVinci-Dev: Agent-native Mid-training for Software Engineering. arXiv preprint arXiv:2601.18418, 2026. URL https://arxiv.org/abs/2601.18418.

[115] Miao Peng, Qintong Zhang, Nuo Chen, Yuhan Li, Guochen Yan, Xinran Gu, Hongqiu Wu, Hai Wang, Lydell Huang, Wentao Zhang, and Jia Li. TrajLong: Co-Designing Agentic and Long-Context Supervision for Mid-Training. arXiv preprint arXiv:2610.04973, 2026. URL https://arxiv.org/abs/2610.04973.

[116] Congmin Zheng, Xiaoyun Mo, Xinbei Ma, Qiqiang Lin, Yin Zhao, Jiachen Zhu, Xingyu Lou, Jun Wang, Zhaoxiang Wang, Weiwen Liu, Zhuosheng Zhang, Yong Yu, and Weinan Zhang. Adaptive Milestone Reward for GUI Agents. arXiv preprint arXiv:2602.11524, 2026. URL https://arxiv.org/abs/2602.11524.

[117] Jiangweizhi Peng, Yuanxin Liu, Ruida Zhou, Charles Fleming, Zhaoran Wang, Alfredo Garcia, and Mingyi Hong. HiPER: Hierarchical Reinforcement Learning with Explicit Credit Assignment for Large Language Model Agents. arXiv preprint arXiv:2602.16165, 2026. URL https://arxiv.org/abs/2602.16165.

[118] Jiaqi Wang, Wenhao Zhang, Weijie Shi, Yaliang Li, and James Cheng. TCOD: Exploring Temporal Curriculum in On-Policy Distillation for Multi-turn Autonomous Agents. arXiv preprint arXiv:2604.24005, 2026. URL https://arxiv.org/abs/2604.24005.

[119] Yubin Wu, Zicheng Cai, Liping Ning, Hua Wang, Zhi Chen, Yaohua Tang, and Hao Chen. LiteGUI: Lightweight GUI Agents via Multi-Solution Guided Distillation and Dual-Level Reinforcement Learning. arXiv preprint arXiv:2605.07505, 2026. URL https://arxiv.org/abs/2605.07505.

[120] Weiwei Li, Junzhuo Liu, Tong Chu, Hengfu Yu, and Wen Li. The Next Screenshot Knows: Gated Hindsight Distillation for Mobile GUI Agents. arXiv preprint arXiv:2608.06065, 2026. URL https://arxiv.org/abs/2608.06065.

[121] Jonas Hübotter, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, and Andreas Krause. Reinforcement Learning via Self-Distillation. In International Conference on Machine Learning, 2026. URL https://arxiv.org/abs/2601.20802.

[122] Xinbei Ma, Congmin Zheng, Jiyang Qiu, Jiale Hong, Yao Yao, Xiangmou Qu, Jiaxin Yin, Xingyu Lou, Jun Wang, Weiwen Liu, Weinan Zhang, Zhuosheng Zhang, Hai Zhao, and Bryan Hooi. Retrospective Progress-Aware Self-Refinement for LLM Agent Training. arXiv preprint arXiv:2606.14302, 2026. URL https://arxiv.org/abs/2606.14302.

[123] Chuan Xiao, Zhengbo Jiao, Shaobo Wang, Wei Wang, Bing Zhao, Hu Wei, Linfeng Zhang, and Lin Qu. Socratic-SWE: Self-Evolving Coding Agents via Trace-Derived Agent Skills. arXiv preprint arXiv:2606.07412, 2026. URL https://arxiv.org/abs/2606.07412.

[124] Yubo Wang, Jiarong Liang, Yuxuan Zhang, Xuye Liu, Cong Wei, Yuyu Zhang, Ping Nie, and Wenhu Chen. Function-Aware Fill-in-the-Middle as Mid-Training for Coding Agent Foundation Models. arXiv preprint arXiv:2607.12463, 2026. URL https://arxiv.org/abs/2607.12463.

[125] Zihan Ding, Longxu Dou, Qi Gao, Xiangwu Guo, Shengchao Hu, Zilong Huang, Zihang Jiang, Lei Ke, Mengcheng Lan, Weixian Lei, Hanxuan Li, Honglin Li, Xiyun Li, Zaitang Li, Leowei Liang, Xin Luo, Haozhe Ma, Jiayi Mao, Zhoujie Pan, Can Qin, Tianyuan Qu, Weiqi Wang, Wenkai Wang, Yonglin Wang, Yuxin Wang, Chenxu Wu, Yingchen Yu, Chenyu Zhang, and Yuhao Zheng. UI-Mate: Advancing Open-Weight Foundation GUI Agents with In-Context Demonstrations. arXiv preprint arXiv:2608.15930, 2026. URL https://arxiv.org/abs/2608.15930.

[126] Zihao Lin, Wanrong Zhu, Jiuxiang Gu, Jihyung Kil, Christopher Tensmeyer, Lin Zhang, Shilong Liu, Ruiyi Zhang, Lifu Huang, Vlad I. Morariu, and Tong Sun. MiLDEdit: Reasoning-Based Multi-Layer Design Document Editing. arXiv preprint arXiv:2601.04589, 2026. URL https://arxiv.org/abs/2601.04589.

[127] Changhao Li, Rushi Qiang, Jiawei Huang, Chenxiao Gao, Chao Zhang, Niao He, and Bo Dai. Revisiting DAgger in the Era of LLM-Agents. arXiv preprint arXiv:2605.12913, 2026. URL https://arxiv.org/abs/2605.12913.

[128] Gengsheng Li, Mao Zheng, Mingyang Song, Ruiqi Liu, Tianyu Yang, Jie Sun, Qiyong Zhong, Haiyun Guo, Junfeng Fang, Dan Zhang, and Jinqiao Wang. On-Policy Distillation with Curriculum Turn-level Guidance for Multi-turn Agents. arXiv preprint arXiv:2606.15912, 2026. URL https://arxiv.org/abs/2606.15912.

[129] Qiyong Zhong, Mao Zheng, Mingyang Song, Xin Lin, Jie Sun, Houcheng Jiang, Xiang Wang, and Junfeng Fang. SOD: Step-wise On-policy Distillation for Small Language Model Agents. arXiv preprint arXiv:2605.07725, 2026. URL https://arxiv.org/abs/2605.07725.

[130] Hao Wang, Guozhi Wang, Han Xiao, Yufeng Zhou, Yue Pan, Jichao Wang, Ke Xu, Yafei Wen, Xiaohu Ruan, Xiaoxin Chen, and Honggang Qi. Skill-SD: Skill-Conditioned Self-Distillation for Multi-turn LLM Agents. arXiv preprint arXiv:2604.10674, 2026. URL https://arxiv.org/abs/2604.10674.

[131] Haoxiang Zhang, Qinglin Chen, Hiroaki Hayashi, Zhuofeng Li, Siming Zhang, Jiaxin Zhang, Jixuan Chen, Fang Wu, Pan Lu, Silvio Savarese, Julian McAuley, and Chien-Sheng Wu. Self-Retrospection Distillation:

Turning Post-hoc Experiences into Prior Foresight. arXiv preprint arXiv:2610.08077, 2026. URL https://arxiv.org/abs/2610.08077.

[132] Jincheng Ren, Siwei Wu, Yizhi Li, Kang Zhu, Shu Xu, Boyu Feng, Ruibin Yuan, Wei Zhang, Riza Batista-Navarro, Jian Yang, and Chenghua Lin. A Self-Evolving Framework for Efficient Terminal Agents via Observational Context Compression. arXiv preprint arXiv:2604.19572, 2026. URL https://arxiv.org/abs/2604.19572.

[133] Sandip Ghoshal, Anshul Mittal, Jyotika Singh, Miguel Ballesteros, Weiyi Sun, Fang Tu, Shailender Singh, Yassine Benajiba, Fahad Shah, Sujeeth Bharadwaj, Sujith Ravi, and Dan Roth. JTPRO: A Joint Tool-Prompt Reflective Optimization Framework for Language Agents. arXiv preprint arXiv:2604.19821, 2026. URL https://arxiv.org/abs/2604.19821.

[134] Cen Mia Zhao, Haibo Ruan, Wenjie Chen, Pei-fen Tu, Usman Abbasi, and Joel Hesch. Beyond Prompts: Measuring and Optimizing LLM Tool-Agent Harnesses. arXiv preprint arXiv:2609.05736, 2026. URL https://arxiv.org/abs/2609.05736.

[135] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, et al. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. In International Conference on Learning Representations, 2026. URL https://arxiv.org/abs/2507.19457.

[136] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-Harness: End-to-End Optimization of Model Harnesses. arXiv preprint arXiv:2603.28052, 2026. URL https://arxiv.org/abs/2603.28052.

[137] Yaxin Luo, Haobin Jiang, Jialv Zou, Xu Huang, Wenhao Yan, Haodong Li, Zhengrong Yue, Jing Li, Xiaofu Chen, Xiaohan Zhao, Jiacheng Liu, Jiacheng Cui, Zhiqiang Shen, and Xiaotong Li. AutoDesign: Meta-Harness Optimization for Long-Horizon Agentic Design. arXiv preprint arXiv:2608.13560, 2026. URL https://arxiv.org/abs/2608.13560.

[138] Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, et al. ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory. In International Conference on Learning Representations, 2026. URL https://arxiv.org/abs/2509.25140.

[139] Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, et al. SkillOpt: Executive Strategy for Self-Evolving Agent Skills. arXiv preprint arXiv:2605.23904, 2026. URL https://arxiv.org/abs/2605.23904.

[140] Yangbo Wei, Zhen Huang, Shaoqiang Lu, Junhong Qian, Qifan Wang, Chen Wu, and Lei He. SkillSmith: Co-Evolving Skills and Tools for Self-Improving Agent Systems. arXiv preprint arXiv:2606.01314, 2026. URL https://arxiv.org/abs/2606.01314.

[141] Ruocheng Guo, Kaiwen Dong, Xiang Gao, and Kamalika Das. Learning to Rewrite Tool Descriptions for Reliable LLM-Agent Tool Use. arXiv preprint arXiv:2602.20426, 2026. URL https://arxiv.org/abs/2602.20426.

[142] Zhiwei Shang, Yu Huo, Mingrong Gong, An Yan, Zikun Qu, Junhao Dong, Bryan Kian Hsiang Low, Chenglin Wu, and Zhongxiang Dai. MESH-Harness: Self-Improving Agent Harnesses via Bandit-Guided Compositional Evolution. arXiv preprint arXiv:2610.05300, 2026. URL https://arxiv.org/abs/2610.05300.

[143] Wenbo Pan, Shujie Liu, Chin-Yew Lin, Jingying Zeng, Xianfeng Tang, Xiangyang Zhou, Yan Lu, and Xiaohua Jia. Evolving Agents in the Dark: Retrospective Harness Optimization via Self-Preference. arXiv preprint arXiv:2606.05922, 2026. URL https://arxiv.org/abs/2606.05922.

[144] Zekai Wang, Yingqiang Ge, Zekun Wang, Hai Wang, Yuhui Xu, Joshua Frandsen, Shancong Fu, Ashia C. Wilson, and Chandan K. Reddy. VERSE: Verified self-evolving optimizer for agent harnesses. arXiv preprint arXiv:2610.02616, 2026. URL https://arxiv.org/abs/2610.02616.

[145] Sungho Park, Wonjoong Kim, Jue Zhang, Wook-Shin Han, Pengfei Gao, Chanyoung Park, Yongqiang Yao, Rao Fu, Elsie Nallipogu, Qingwei Lin, and Victor Rühle. ActiveSaddler: Automated Curriculum Learning for Agent Harness Optimization. arXiv preprint arXiv:2610.00906, 2026. URL https://arxiv.org/abs/2610.00906.

[146] Zewen Liu, Zhan Shi, Yisi Sang, Bing He, Minhua Lin, Tianxin Wei, Dakuo Wang, Benoit Dumoulin, Wei Jin, and Hanqing Lu. Adaptive Auto-Harness: Sustained Self-Improvement for Agentic System Deployment on Open-Ended Task Streams. arXiv preprint arXiv:2606.01770, 2026. URL https://arxiv.org/abs/2606.01770.

[147] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, et al. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. In International Conference on Learning Representations, 2026. URL https://arxiv.org/abs/2510.04618.

[148] Zhengxi Lu, Fei Tang, Guangyi Liu, Kaitao Song, Xu Tan, Jin Ma, Wenqi Zhang, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. UI-Copilot: Advancing Long-Horizon GUI Automation via Tool-Integrated Policy Optimization. arXiv preprint arXiv:2604.13822, 2026. URL https://arxiv.org/abs/2604.13822.

[149] Guibin Zhang, Leo Lu, Fangzhou Xie, Kang Zhu, Junhao Wang, Zhifei Xie, Zhaochen Yu, Zihang Liu, Zhongxiang Sun, Qiankun Li, Yue Liao, Heng Chang, Xiaobin Hu, Qibing Ren, Wangchunshu Zhou, Chuanrui Hu, Yafeng Deng, and Shuicheng Yan. JIT-Agent: Scaling Harness Intelligence via Just-in-Time Harness Evolution. arXiv preprint arXiv:2608.25593, 2026. URL https://arxiv.org/abs/2608.25593.

[150] Yanzhou Li, Yiran Zhang, Xiaoyu Zhang, Xiaoxia Liu, and Yang Liu. CODESKILL: Learning Self-Evolving Skills for Coding Agents. arXiv preprint arXiv:2605.25430, 2026. URL https://arxiv.org/abs/2605.25430.

[151] Wei Zhang, Hongji Li, Song Sun, Peng Yu, Xue Yang, Lei Zhao, and Peng Jiang. DMRL: Document-Mediated Reinforcement Learning for Skill Optimization in Advertising Recommendation. arXiv preprint arXiv:2609.02170, 2026. URL https://arxiv.org/abs/2609.02170.

[152] Han Xiao, Guozhi Wang, Hao Wang, Shilong Liu, Yuxiang Chai, Yue Pan, Yufeng Zhou, Xiaoxin Chen, Yafei Wen, and Hongsheng Li. UI-Mem: Self-Evolving Experience Memory for Online Reinforcement Learning in Mobile GUI Agents. arXiv preprint arXiv:2602.05832, 2026. URL https://arxiv.org/abs/2602.05832.

[153] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. arXiv preprint arXiv:2602.08234, 2026. URL https://arxiv.org/abs/2602.08234.

[154] Rishabh Tiwari, Kusha Sareen, Lakshya A Agrawal, Joseph E. Gonzalez, Matei Zaharia, Kurt Keutzer, Inderjit S Dhillon, Rishabh Agarwal, and Devvrit Khatri. Learning, Fast and Slow: Towards LLMs That Adapt Continually. arXiv preprint arXiv:2605.12484, 2026. URL https://arxiv.org/abs/2605.12484.

[155] Prannay Hebbar, Yogendra Manawat, Samuel Verboomen, Alesia Ivanova, Selvam Palanimalai, Kunal Bhatia, and Vignesh Baskaran. SIA: Self Improving AI with Harness & Weight Updates. arXiv preprint arXiv:2605.27276, 2026. URL https://arxiv.org/abs/2605.27276.

[156] Zhi Rui Tam, Chieh-Yen Lin, Yun-Nung Chen, Shao-Hua Sun, and Hung-yi Lee. Joint Optimization of Tool Creation and Use for Large Language Model Agents. arXiv preprint arXiv:2608.24571, 2026. URL https://arxiv.org/abs/2608.24571.

[157] Junhao Shen, Zhanqiu Zhang, Yiwen Guo, and Hong Cheng. Progressive Agent Skill Generation via Reinforcement Learning. arXiv preprint arXiv:2608.01678, 2026. URL https://arxiv.org/abs/2608.01678.

[158] Shirley Wu, Parth Sarthi, Shiyu Zhao, Aaron Lee, Herumb Shandilya, Adrian Mladenic Grobelnik, et al. Optimas: Optimizing Compound AI Systems with Globally Aligned Local Rewards. In International Conference on Learning Representations, 2026. URL https://arxiv.org/abs/2507.03041.

[159] Beining Wu, Zihao Ding, and Jun Huang. Not All Experience Belongs in the Weights: Component Routing for Self-Improving GUI Agents. arXiv preprint arXiv:2610.01787, 2026. URL https://arxiv.org/abs/2610.01787.

[160] Grégoire Mialon, Clémentine Fourrier, Craig Swift, et al. GAIA: a benchmark for General AI Assistants. arXiv preprint, 2023. URL https://arxiv.org/abs/2311.12983.

[161] Liya Zhu, Xin Ma, Tao Liu, et al. StartupBench: Benchmarking General-Purpose Agents on Market-Validated End-to-End Workflows. arXiv preprint, 2026. URL https://arxiv.org/abs/2608.17800.

[162] Pranab Islam, Anand Kannappan, Douwe Kiela, Rebecca Qian, Nino Scherrer, and Bertie Vidgen. FinanceBench: A New Benchmark for Financial Question Answering. arXiv preprint, 2023. URL https://arxiv.org/abs/2311.11944.

[163] Yuhang Lai, Chengxi Li, Yiming Wang, et al. DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation. arXiv preprint, 2022. URL https://arxiv.org/abs/2211.11501.

[164] Giulio Starace, Oliver Jaffe, Dane Sherburn, et al. PaperBench: Evaluating AI's Ability to Replicate AI Research. arXiv preprint, 2025. URL https://arxiv.org/abs/2504.01848.

[165] Shiva Krishna Reddy Malay, Shravan Nayak, Jishnu Sethumadhavan Nair, et al. EnterpriseOps-Gym: Environments and Evaluations for Stateful Agentic Planning and Tool Use in Enterprise Settings. arXiv preprint, 2026. URL https://arxiv.org/abs/2603.13594.

[166] Yujia Qin, Shihao Liang, Yining Ye, et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. arXiv preprint, 2023. URL https://arxiv.org/abs/2307.16789.

[167] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. arXiv preprint, 2023. URL https://arxiv.org/abs/2306.05685.

[168] Mingchen Zhuge, Changsheng Zhao, Dylan Ashley, et al. Agent-as-a-Judge: Evaluate Agents with Agents. arXiv preprint, 2024. URL https://arxiv.org/abs/2410.10934.

[169] Thibault Le Sellier De Chezelles, Maxime Gasse, Alexandre Drouin, Massimo Caccia, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omidi Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Quentin Cappart, Graham Neubig, Ruslan Salakhutdinov, Nicolas Chapados, and Alexandre Lacoste. The browsergym ecosystem for web agent research. arXiv preprint arXiv:2412.05467, 2024. URL https://arxiv.org/abs/2412.05467.

[170] OpenAI. Codex Manual: Best Practices and Runtime Configuration, 2026. URL https://developers.openai.com/codex/codex-manual.md.

[171] Anthropic. Claude Code Overview, 2026. URL https://code.claude.com/docs/en/overview.

[172] Microsoft. Microsoft 365 Copilot Architecture and How It Works, 2026. URL https://learn.microsoft.com/en-us/copilot/microsoft-365/microsoft-365-copilot-architecture.

[173] National Center for O*NET Development. The O*NET content model. URL https://www.onetcenter.org/content.html. Accessed 13 October 2026.

[174] Mantas Mazeika, Alice Gatti, Cristina Menghini, Udari Madhushani Sehwag, Shivam Singhal, Yury Orlovskiy, Steven Basart, Manasi Sharma, Denis Peskoff, Elaine Lau, Jaehyuk Lim, Lachlan Carroll, Alice Blair, Vinaya Sivakumar, Sumana Basu, Brad Kenstler, Yuntao Ma, Julian Michael, Xiaoke Li, Oliver Ingebretsen, Aditya Mehta, Jean Mottola, John Teichmann, Kevin Yu, Zaina Shaik, Adam Khoja, Richard Ren, Jason Hausenloy, Long Phan, Ye Htet, Ankit Aich, Tahseen Rabbani, Vivswan Shah, Andriy Novykov, Felix Binder, Kirill Chugunov, Luis Ramirez, Matias Geralnik, Hernán Mesura, Dean Lee, Ed-Yeremai Hernandez Cardona, Annette Diamond, Summer Yue, Alexandr Wang, Bing Liu, Ernesto Hernandez, and Dan Hendrycks. Remote labor index: Measuring ai automation of remote work. arXiv preprint arXiv:2510.26787, 2025. URL https://arxiv.org/abs/2510.26787.

[175] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with swe-gym. arXiv preprint arXiv:2412.21139, 2024. URL https://arxiv.org/abs/2412.21139.

[176] Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. R2e-gym: Procedural environments and hybrid verifiers for scaling open-weights swe agents. arXiv preprint arXiv:2504.07164, 2025. URL https://arxiv.org/abs/2504.07164.

[177] John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. Swe-smith: Scaling data for software engineering agents. arXiv preprint arXiv:2504.21798, 2025. URL https://arxiv.org/abs/2504.21798.

[178] Yijia Shao, Vinay Samuel, Yucheng Jiang, John Yang, and Diyi Yang. Collaborative gym: A framework for enabling and evaluating human-agent collaboration. arXiv preprint arXiv:2412.15701, 2024. URL https://arxiv.org/abs/2412.15701.

[179] Romain Froger, Pierre Andrews, Matteo Bettini, Amar Budhiraja, Ricardo Silveira Cabral, Virginie Do, Emilien Garreau, Jean-Baptiste Gaya, Hugo Laurençon, Maxime Lecanu, Kunal Malkan, Dheeraj Mekala, Pierre Ménard, Gerard Moreno-Torres Bertran, Ulyana Piterbarg, Mikhail Plekhanov, Mathieu Rita, Andrey Rusakov, Vladislav Vorotilov, Mengjue Wang, Ian Yu, Amine Benhalloum, Grégoire Mialon, and Thomas Scialom. ARE: Scaling up agent environments and evaluations. arXiv preprint arXiv:2509.17158, 2025. URL https://arxiv.org/abs/2509.17158.

[180] Xuhui Zhou, Weiwei Sun, Qianou Ma, Yiqing Xie, Jiarui Liu, Weihua Du, Sean Welleck, Yiming Yang, Graham Neubig, Sherry Tongshuang Wu, and Maarten Sap. Mind the Sim2Real gap in user simulation for agentic tasks. In Conference on Language Modeling, 2026. URL https://arxiv.org/abs/2603.11245.

[181] Preethi Seshadri, Samuel Cahyawijaya, Ayomide Odumakinde, Sameer Singh, and Seraphina Goldfarb-Tarrant. Lost in simulation: LLM-simulated users are unreliable proxies for human users in agentic evaluations. arXiv preprint arXiv:2601.17087, 2026. URL https://arxiv.org/abs/2601.17087.

[182] Zeyu Leo Liu, Shrey Pandit, Xi Ye, Eunsol Choi, and Greg Durrett. CodeUpdateArena: Benchmarking knowledge editing on API updates. arXiv preprint arXiv:2407.06249, 2024. URL https://arxiv.org/abs/2407.06249.

[183] Tianshu Zhang, Kun Qian, Siddhartha Sahai, Yuan Tian, Shaddy Garg, Huan Sun, and Yunyao Li. EvoSchema: Towards text-to-SQL robustness against schema evolution. In International Conference on Very Large Data Bases, 2025. URL https://arxiv.org/abs/2603.10697.

[184] Jundong Xu, Qingchuan Li, Jiaying Wu, Yihuai Lan, Shuyue Stella Li, Huichi Zhou, Bowen Jiang, Lei Wang, Jun Wang, Anh Tuan Luu, Caiming Xiong, Hae Won Park, Bryan Hooi, and Zhiyuan Hu. EvoArena: Tracking memory evolution for robust LLM agents in dynamic environments. arXiv preprint arXiv:2606.13681, 2026. URL https://arxiv.org/abs/2606.13681.

[185] Anthropic. Effective Harnesses for Long-Running Agents, 2025. URL https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents.

[186] Guanting Dong, Xiaoshuai Song, Yuyang Hu, et al. Towards Long-Horizon Agents: A Survey — Foundation, Evolution, Harness, Optimization, Application, and Frontier, 2026. URL https://github.com/RUC-NLPIR/Awesome-Long-Horizon-Agents.

[187] Yaxi Lu, Shenzhi Yang, Cheng Qian, Guirong Chen, Qinyu Luo, Yesai Wu, Huadong Wang, Xin Cong, Zhong Zhang, Yankai Lin, Weiwen Liu, Yasheng Wang, Zhiyuan Liu, Fangming Liu, and Maosong Sun. Proactive agent: Shifting LLM agents from reactive responses to active assistance. In International Conference on Learning Representations, 2025. URL https://arxiv.org/abs/2410.12361.

[188] Jio Oh, Seunghyun Do, Young-Jun Lee, Steven Euijong Whang, and Dongyeop Kang. Foundations of proactive agents: Principles, technical layers, and Proactivity-Gym. arXiv preprint arXiv:2609.37267, 2026. URL https://arxiv.org/abs/2609.37267.

[189] Xiaohongshu Dots Studio and Evolvent AI. VibeLifeBench: Can your life agent be proactive and persistent in a living world? arXiv preprint arXiv:2608.10875, 2026. URL https://arxiv.org/abs/2608.10875.

[190] In Gim, Seung seob Lee, and Lin Zhong. Asynchronous LLM function calling. arXiv preprint arXiv:2412.07017, 2024. URL https://arxiv.org/abs/2412.07017.

[191] Kou Shi, Ziao Zhang, Shiting Huang, Avery Nie, Zhen Fang, Qiuchen Wang, Lin Chen, Huaian Chen, Zehui Chen, and Feng Zhao. AsyncTool: Evaluating the asynchronous function calling capability under multi-task scenarios. arXiv preprint arXiv:2605.27995, 2026. URL https://arxiv.org/abs/2605.27995.

[192] Yikai Zhang, Siyu Yuan, Caiyu Hu, Kyle Richardson, Yanghua Xiao, and Jiangjie Chen. TimeArena: Shaping efficient multitasking language agents in a time-aware simulation. In Annual Meeting of the Association for Computational Linguistics, 2024. URL https://arxiv.org/abs/2402.05733.

[193] Fangru Lin, Emanuele La Malfa, Valentin Hofmann, Elle Michelle Yang, Anthony Cohn, and Janet B. Pierrehumbert. Graph-enhanced large language models in asynchronous plan reasoning. In International Conference on Machine Learning, 2024. URL https://arxiv.org/abs/2402.02805.

[194] Isham Kalappurackal Mansoor, Abhishek Phadke, and Pratip Rana. Verified tool calls improve LLM agent reliability under non-atomic failures. arXiv preprint arXiv:2608.02645, 2026. URL https://arxiv.org/abs/2608.02645.

[195] Pat Helland. Life beyond distributed transactions. ACM Queue, 14(5):69–98, 2016. doi: 10.1145/3012426.3025012.

[196] Yusheng Zheng, Yiwei Yang, Wei Zhang, and Andi Quinn. ACRFence: Preventing semantic rollback attacks in agent checkpoint-restore. arXiv preprint arXiv:2603.20625, 2026. URL https://arxiv.org/abs/2603.20625.

[197] James Cheney, Laura Chiticariu, and Wang-Chiew Tan. Provenance in databases: Why, how, and where. Foundations and Trends in Databases, 1(4):379–474, 2009. doi: 10.1561/1900000006.

[198] Hector Garcia-Molina and Kenneth Salem. Sagas. In Proceedings of the ACM SIGMOD International Conference on Management of Data, pages 249–259, 1987. doi: 10.1145/38713.38742.

[199] Edward Y. Chang and Longling Geng. SagaLLM: Context management, validation, and transaction guarantees for multi-agent LLM planning. arXiv preprint arXiv:2503.11951, 2025. URL https://arxiv.org/abs/2503.11951.

[200] Bardia Mohammadi, Nearchos Potamitis, Lars Klein, Akhil Arora, and Laurent Bindschaedler. Atomix: Timely, transactional tool use for reliable agentic workflows. arXiv preprint arXiv:2602.14849, 2026. URL https://arxiv.org/abs/2602.14849.

[201] Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. In International Conference on Machine Learning, 2025. URL https://arxiv.org/abs/2505.00212.

[202] Mengzhuo Chen, Junjie Wang, Fangwen Mu, Yawen Wang, Zhe Liu, Huanxiang Feng, and Qing Wang. Seeing the whole elephant: A benchmark for failure attribution in LLM-based multi-agent systems. In Annual Meeting of the Association for Computational Linguistics, 2026. URL https://arxiv.org/abs/2604.22708.

[203] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. arXiv preprint arXiv:2308.10144, 2023. URL https://arxiv.org/abs/2308.10144.

[204] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv preprint, 2023. URL https://arxiv.org/abs/2305.16291.

[205] Jialong Li and Jialing Zhu. Auditing self-evolution in financial agents: Capability gains, security drift, and execution-interface mismatch. arXiv preprint arXiv:2608.17684, 2026. URL https://arxiv.org/abs/2608.17684.

[206] David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. In Advances in Neural Information Processing Systems, 2017. URL https://arxiv.org/abs/1706.08840.

[207] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. arXiv preprint arXiv:2501.13956, 2025. URL https://arxiv.org/abs/2501.13956.

[208] Yongye Su, Wujiang Xu, Chaoji Zuo, and Elisa Bertino. ChronoMem: Version control and semantic rollback for large language model agent memory. arXiv preprint arXiv:2607.27773, 2026. URL https://arxiv.org/abs/2607.27773.

[209] Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations. In Advances in Neural Information Processing Systems, 2024. URL https://arxiv.org/abs/2404.13076.

[210] Shashwat Goel, Joschka Struber, Ilze Amanda Auzina, Karuna K. Chandra, Ponnurangam Kumaraguru, Douwe Kiela, Ameya Prabhu, Matthias Bethge, and Jonas Geiping. Great models think alike and this undermines AI oversight. In International Conference on Machine Learning, 2025. URL https://arxiv.org/abs/2502.04313.

[211] Wenle Liao, Zhao Wang, Jingchao Zhang, Jiajie Jin, Yimeng Xu, and Zhicheng Dou. StateGuard: Analytical-state management with validity-aware intervention for long-horizon data agents. arXiv preprint arXiv:2609.34134, 2026. URL https://arxiv.org/abs/2609.34134.

[212] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In International Conference on Machine Learning, 2023. URL https://arxiv.org/abs/2210.10760.

[213] Xing Zhang, Guanghui Wang, Yanwei Cui, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He. Who verifies the verifier? co-evolving inspectable graders with self-improving agents. arXiv preprint arXiv:2610.11464, 2026. URL https://arxiv.org/abs/2610.11464.

[214] LangChain. LangGraph Overview, 2026. URL https://docs.langchain.com/oss/python/langgraph/overview.

[215] Wil M. P. van der Aalst, Mathias Weske, and Dolf Grünbauer. Case handling: A new paradigm for business process support. Data & Knowledge Engineering, 53(2):129–162, 2005. doi: 10.1016/j.datak.2004.07.003.

[216] Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang Zhu, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, Peter Chang, Ricky Loynd, Robert

West, Victor Dibia, Ahmed Awadallah, Ece Kamar, Rafah Hosn, and Saleema Amershi. Magentic-One: A generalist multi-agent system for solving complex tasks. arXiv preprint arXiv:2411.04468, 2024. URL https://arxiv.org/abs/2411.04468.

[217] Tobin South, Samuele Marro, Thomas Hardjono, Robert Mahari, Cedric Deslandes Whitney, Dazza Greenwood, Alan Chan, and Alex Pentland. Authenticated delegation and authorized AI agents. arXiv preprint arXiv:2501.09674, 2025. URL https://arxiv.org/abs/2501.09674.

[218] Yian Wang, Agam Goyal, Eshwar Chandrasekharan, and Hari Sundaram. Facts without rules: Boundary metadata collapse in multi-agent LLM handoffs. arXiv preprint arXiv:2608.29028, 2026. URL https://arxiv.org/abs/2608.29028.

[219] Gary Klein, David D. Woods, Jeffrey M. Bradshaw, Robert R. Hoffman, and Paul J. Feltovich. Ten challenges for making automation a "team player in joint human-agent activity. IEEE Intelligent Systems, 19(6):91–95, 2004. doi: 10.1109/MIS.2004.74.

[220] Shakked Noy and Whitney Zhang. Experimental evidence on the productivity effects of generative artificial intelligence. Science, 381(6654):187–192, 2023. doi: 10.1126/science.adh2586.

[221] Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein. Measuring the impact of early-2025 ai on experienced open-source developer productivity. arXiv preprint arXiv:2507.09089, 2025. URL https://arxiv.org/abs/2507.09089.

[222] Erik Brynjolfsson, Daniel Rock, and Chad Syverson. The productivity J-curve: How intangibles complement general purpose technologies. American Economic Journal: Macroeconomics, 13(1):333–372, 2021. doi: 10.1257/mac.20180386.

[223] Lisanne Bainbridge. Ironies of automation. Automatica, 19(6):775–779, 1983. doi: 10.1016/0005-1098(83)90046-8.

[224] Thomas W. Malone and Kevin Crowston. The interdisciplinary study of coordination. ACM Computing Surveys, 26(1):87–119, 1994. doi: 10.1145/174666.174668.